

LIMITBREAK · SECURITY REVIEW

LimitBreak Round Two.

A security review of Round Two.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
LimitBreak	Round Two	Guardian	November 24 – April 24, 2026	May 1, 2026

§ 00

Contents.

217 pages — overview, confidence ranking, scope, 115 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 08
	Main Review · 44 findings	P. 16
	Remediation Review 1 · 27 findings	P. 95
	Remediation Review 2 · 11 findings	P. 141
	Remediation Review 3 · 15 findings	P. 157
	Remediation Review 4 · 14 findings	P. 185
	Remediation Review 5 · 4 findings	P. 211
§ 05	Disclaimer	P. 216
§ 06	About Guardian	P. 217

Overview.

Guardian reviewed Round Two across 6 review rounds. The final report includes 2 Critical, 6 High, 45 Medium, 38 Low, 24 Informational findings.

AUDIT FIRM	REPOSITORY
Guardian	limit-break-inc/lbamm-core, limit-break-inc/lbamm-hooks-and-handlers, limit-break-inc/lbamm-pool-type-fixed, limit-break-inc/amm-pool-type-dynamic, limit-break-inc/tm-core-lib, limit-break-inc/secure-proxy
CLIENT	METHODOLOGY
LimitBreak	Smart contract · Manual · Main Review · Remediation Review 1 · Remediation Review 2 · Remediation Review 3 · Remediation Review 4 · Remediation Review 5
PROJECT	MAIN REVIEW COMMIT
Round Two	d5435e12c4ce
PREPARED BY	REMEDATION REVIEW 1 COMMIT
Guardian	35994740ba14
AUDIT WINDOW	REMEDATION REVIEW 2 COMMIT
November 24 – April 24, 2026	c956d55c8e98
FINAL REPORT	REMEDATION REVIEW 3 COMMIT
May 1, 2026	64ae27a2a9f3

1.1 Confidence Ranking

3 / 5

Moderate Confidence

Guardian assigns a Confidence Ranking of 3 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
Critical	2	0	2	0	0
High	6	0	6	0	0
Medium	45	0	37	0	8
Low	38	0	25	0	13
Informational	24	0	19	0	5

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence ASSIGNED	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Round Two in `limit-break-inc/lbamm-core`, `limit-break-inc/lbamm-hooks-and-handlers`, `limit-break-inc/lbamm-pool-type-fixed`, `limit-break-inc/amm-pool-type-dynamic`, `limit-break-inc/tm-core-lib`, `limit-break-inc/secure-proxy`. The scope includes 6,524 source lines across 61 files.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH

Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM

Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW

Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH

Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM

Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW

Unlikely to ever occur in production.

Files in Scope 1/2.

FILE	NSLOC	TOTAL
src/Constants.sol	71	134
src/DataTypes.sol	174	454
src/Errors.sol	55	164
src/LimitBreakAMM.sol	149	797
src/modules/AMMModule.sol	1,816	3,319
src/modules/ModuleAdmin.sol	97	289
src/modules/ModuleFeeCollection.sol	72	209
src/modules/ModuleLiquidity.sol	32	214
src/libraries/FeeHelper.sol	83	201
src/libraries/LBAMMStorage.sol	10	34
src/libraries/PoolDecoder.sol	12	46
src/interfaces/ILimitBreakAMM.sol	15	23
src/interfaces/ILimitBreakAMMFlashloanCallback.sol	4	31
src/interfaces/ILimitBreakAMMPoolType.sol	5	30
src/interfaces/ILimitBreakAMMTransferHandler.sol	5	33
src/interfaces/hooks/ILimitBreakAMMLiquidityHook.sol	5	35
src/interfaces/hooks/ILimitBreakAMMPoolHook.sol	5	25
src/interfaces/hooks/ILimitBreakAMMTokenHook.sol	5	29
src/interfaces/core/ILimitBreakAMMEvents.sol	68	108
src/interfaces/core/ILimitBreakAMMFees.sol	4	28
src/interfaces/core/ILimitBreakAMMFlashloan.sol	4	17
src/interfaces/core/ILimitBreakAMMLiquidity.sol	4	41
src/interfaces/core/ILimitBreakAMMProtocol.sol	4	17
src/interfaces/core/ILimitBreakAMMSwap.sol	4	56
src/interfaces/core/ILimitBreakAMMTokenSettings.sol	4	32
src/hooks/AMMStandardHook.sol	332	795
src/hooks/CreatorHookSettingsRegistry.sol	292	891
src/hooks/DataTypes.sol	28	68
src/hooks/Errors.sol	22	65
src/hooks/libraries/SqrtPriceCalculator.sol	51	120
src/hooks/interfaces/IAMMStandardHook.sol	39	80
src/hooks/interfaces/ICreatorHookSettingsRegistry.sol	38	81
src/handlers/permit/Constants.sol	17	50
src/handlers/permit/DataTypes.sol	29	64
src/handlers/permit/Errors.sol	11	32
src/handlers/permit/PermitTransferHandler.sol	260	464

Files in Scope 2/2.

FILE	NSLOC	TOTAL
src/handlers/interfaces/ITransferHandlerExecutorValidation.sol	3	25
src/Constants.sol	15	44
src/DataTypes.sol	99	244
src/Errors.sol	13	38
src/FixedPoolQuoter.sol	43	85
src/FixedPoolType.sol	173	421
src/interfaces/IFixedPoolType.sol	19	40
src/libraries/FixedHelper.sol	880	1,291
src/libraries/FixedPoolDecoder.sol	19	54
src/Constants.sol	16	49
src/DataTypes.sol	76	192
src/DynamicPoolType.sol	295	605
src/Errors.sol	18	53
src/interfaces/IDynamicPoolType.sol	25	49
src/libraries/BitMath.sol	32	67
src/libraries/DynamicHelper.sol	319	664
src/libraries/DynamicPoolDecoder.sol	10	44
src/libraries/LiquidityMath.sol	13	44
src/libraries/SqrtPriceMath.sol	177	410
src/libraries/SwapMath.sol	73	143
src/libraries/TickMath.sol	146	237
src/Constants.sol	17	44
src/DataTypes.sol	12	30
src/Errors.sol	8	23
src/SecureProxy.sol	197	456

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
C-01	Stale nextHeightAbove Causes Double Height Cross	Logical Error	CRITICAL	ACKNOWLEDGED	p. 16
H-01	LP Fees Permanently Locked in Fixed Pool	Logical Error	HIGH	ACKNOWLEDGED	p. 19
H-02	Missing Height Update Locks LP Funds	Logical Error	HIGH	ACKNOWLEDGED	p. 21
H-03	Add Liquidity To Height DoS	Logical Error	HIGH	ACKNOWLEDGED	p. 23
M-01	Empty-pool swaps set arbitrary price	Unexpected Behavior	MEDIUM	ACKNOWLEDGED	p. 26
M-02	Partial fills can zero LP/protocol fees	Rounding	MEDIUM	ACKNOWLEDGED	p. 28
M-03	Direct Swap Max-Price Bound Bypass via Overflow	Logical Error	MEDIUM	ACKNOWLEDGED	p. 31
M-04	FixedPoolType LP Sybil Attack	Rewards	MEDIUM	ACKNOWLEDGED	p. 32
M-05	directSwap returns inverted amounts	Unexpected Behavior	MEDIUM	ACKNOWLEDGED	p. 33
M-06	minimumProtocolFee Not Symmetric Across Flows	Math	MEDIUM	ACKNOWLEDGED	p. 35
M-07	minimumProtocolFee Not Reduced On Partial Fill	Math	MEDIUM	ACKNOWLEDGED	p. 36
M-08	Asymmetric Price Bounds Validation	Validation	MEDIUM	ACKNOWLEDGED	p. 37
M-09	Fixed swapByInput strands "change" in reserves	Unexpected Behavior	MEDIUM	ACKNOWLEDGED	p. 38

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
M-10	Fixed pool LP exit can revert on fee dust	DoS	MEDIUM	ACKNOWLEDGED	p. 40
M-11	Fixed pool reserve desync from split rounding	Unexpected Behavior	MEDIUM	ACKNOWLEDGED	p. 45
M-12	Partial-consumed height underpays pairValue	Rounding	MEDIUM	ACKNOWLEDGED	p. 48
M-13	Missing Token Flashloan Validation	Validation	MEDIUM	ACKNOWLEDGED	p. 50
M-14	Missing Collect Fees Hook	Logical Error	MEDIUM	ACKNOWLEDGED	p. 51
L-01	Executor-set fee-on-top not bound to permit	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 53
L-02	Role server outage bricks paused proxy	Warning	LOW	ACKNOWLEDGED	p. 55
L-03	SecureProxy doesn't enforce accountability	Trust Assumptions	LOW	ACKNOWLEDGED	p. 57
L-04	Partial withdraw wipes position on prec. loss	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 59
L-05	Pool rules bypass before hook config	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 61
L-06	Dynamic-Fee Hook May Silently Return 0 poolFee	Validation	LOW	ACKNOWLEDGED	p. 63
L-07	checkAMMExecutionState Return Can Be Wrong	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 64
L-08	Misleading Error	Error	LOW	ACKNOWLEDGED	p. 65
L-09	Fixed pool user loss via silent rounding to 1	Rounding	LOW	ACKNOWLEDGED	p. 66
L-10	FixedPool hook may cause a silent overflow	Math	LOW	ACKNOWLEDGED	p. 68
L-11	Wrong slippage parameter specs for direct swaps	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 70
L-12	feeGrowthOutside Not Initialized for New Heights	Warning	LOW	ACKNOWLEDGED	p. 72

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-13	Rounding Effect Causes Swap Reverts	Rounding	LOW	ACKNOWLEDGED	p. 75
L-14	Tokens Can't Disallow Pairs After Pool Creation	Validation	LOW	ACKNOWLEDGED	p. 77
L-15	Fee growth calc can overflow on output swaps	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 78
I-01	Brute-forceable pause codes	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 80
I-02	Pause codes can be reused in SecureProxy	Trust Assumptions	INFORMATIONAL	ACKNOWLEDGED	p. 82
I-03	Fee-on-top not prorated on partial fills	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 84
I-04	Wrong error used for sqrtPrice bounds	Best Practices	INFORMATIONAL	ACKNOWLEDGED	p. 85
I-05	Missing Zero Adress Checks In Constructors	Best Practices	INFORMATIONAL	ACKNOWLEDGED	p. 86
I-06	Missing Zero Adress Checks In Constructors	Best Practices	INFORMATIONAL	ACKNOWLEDGED	p. 87
I-07	Unused Code	Superfluous Code	INFORMATIONAL	ACKNOWLEDGED	p. 88
I-08	Redundant temporary vars in _poolSwapByInput	Superfluous Code	INFORMATIONAL	ACKNOWLEDGED	p. 90
I-09	Price Bounds isSet Can Not Be Reset	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 91
I-10	Unused Return Declaration In ammHandleTransfer	Best Practices	INFORMATIONAL	ACKNOWLEDGED	p. 92
I-11	Exact Out Slippage Failure	Logical Error	INFORMATIONAL	ACKNOWLEDGED	p. 93
REMEDIATION REVIEW 1					
C-01	Zero-Amount Cross Can Underflow Liquidity	DoS	CRITICAL	ACKNOWLEDGED	p. 95
H-01	Missing Hook In CLOB	Validation	HIGH	ACKNOWLEDGED	p. 99

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
H-02	increaseHeight Leaves Zero Remaining Mid-Range	Logical Error	HIGH	ACKNOWLEDGED	p. 100
H-03	Split Rounding Shifts Excess Output Causing DoS	DoS	HIGH	ACKNOWLEDGED	p. 102
M-01	Zero-Amount Orders Can DoS Fills	DoS	MEDIUM	ACKNOWLEDGED	p. 105
M-02	Missing tokenIn != tokenOut Validation	Unexpected Behavior	MEDIUM	ACKNOWLEDGED	p. 107
M-03	CLOB openOrder Reverts With AMM Hook	Unexpected Behavior	MEDIUM	ACKNOWLEDGED	p. 109
M-04	Remove hintSqrtPriceX96 Griefing Attack	Gas Griefing	MEDIUM	ACKNOWLEDGED	p. 110
M-05	Price Validation Fails If beforeSwap Disabled	DoS	MEDIUM	ACKNOWLEDGED	p. 111
M-06	Token Liquidity Hook Fees Ignored	Rewards	MEDIUM	ACKNOWLEDGED	p. 113
M-07	Price Bounds Bypass Via snapPrice	Logical Error	MEDIUM	ACKNOWLEDGED	p. 114
M-08	Current Height Manipulation Bypasses Protection	Validation	MEDIUM	ACKNOWLEDGED	p. 115
M-09	Input Swap Split Can Exceed Input	DoS	MEDIUM	ACKNOWLEDGED	p. 118
M-10	Stale Escalation Tier Blocks New Emergency Pause	DoS	MEDIUM	ACKNOWLEDGED	p. 120
L-01	Unbounded Fill Loop Enables Gas Griefing	DoS	LOW	ACKNOWLEDGED	p. 122
L-02	OrderBucket Prev Pointers Go Stale	Warning	LOW	ACKNOWLEDGED	p. 123
L-03	Zero Amount Deposits And Withdrawals Permitted	Validation	LOW	ACKNOWLEDGED	p. 125
L-04	Unsafe Pattern: Missing Tstorish Reset	Warning	LOW	ACKNOWLEDGED	p. 126

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-05	Zero-Liquidity Price Manipulation	Warning	LOW	ACKNOWLEDGED	p. 128
L-06	Token0 Not Restored After Precision Rounding	Rounding	LOW	ACKNOWLEDGED	p. 129
I-01	OrderBookFill Event May Report Wrong Nonce	Events	INFORMATIONAL	ACKNOWLEDGED	p. 131
I-02	Non-Canonical Token Ordering In Hook Context	Logical Error	INFORMATIONAL	ACKNOWLEDGED	p. 132
I-03	Unused excessAmountIn Field In FixedSwapCache	Superfluous Code	INFORMATIONAL	ACKNOWLEDGED	p. 134
I-04	swapByOutput Can Undercharge Input	Rounding	INFORMATIONAL	ACKNOWLEDGED	p. 135
I-05	Misleading Error On Invalid Tick	Error	INFORMATIONAL	ACKNOWLEDGED	p. 137
I-06	Tstore Activation EOA-Only Not Enforced	Documentation	INFORMATIONAL	ACKNOWLEDGED	p. 138
I-07	Fee Shortage Incorrectly Amplified In Outputs	Math	INFORMATIONAL	ACKNOWLEDGED	p. 139

REMEDIATION REVIEW 2

M-01	Zero Ratio Component Bricks Swaps At Min	DoS	MEDIUM	RESOLVED	p. 141
L-01	Split Rounding Can Overpay Output	Rounding	LOW	ACKNOWLEDGED	p. 143
L-02	Exact Output Splits Can Underfill	Unexpected Behavior	LOW	RESOLVED	p. 145
L-03	_crossHeight Guard Checks Wrong Value	Logical Error	LOW	RESOLVED	p. 147
L-04	Missing HANDLER_ORDER_VALIDATE Flag In Mask	Configuration	LOW	RESOLVED	p. 148
L-05	hopFeeBPS = MAX_BPS Blocks Output Swaps	Unexpected Behavior	LOW	RESOLVED	p. 149

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-06	Direct Swap Max Out Ignores Hook Fees	Unexpected Behavior	LOW	RESOLVED	p. 150
L-07	No Direct Way To Disable Hooks	Validation	LOW	RESOLVED	p. 151
L-08	CLOB Fills Let Executor Skim Maker-Funded Fees	Unexpected Behavior	LOW	RESOLVED	p. 152
I-01	Unused Error	Superfluous Code	INFORMATIONAL	RESOLVED	p. 154
I-02	Hook Validates Different Fee Than Charged	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 155

REMEDIATION REVIEW 3

M-01	Flashloan Cross-Token Fee Can Use Wrong Units	Unexpected Behavior	MEDIUM	ACKNOWLEDGED	p. 157
M-02	Hook Pricing Breaks On Partial Fills	Logical Error	MEDIUM	ACKNOWLEDGED	p. 159
M-03	Floor Math Arbitrage Extracts Reserves	Rounding	MEDIUM	RESOLVED	p. 161
M-04	Pool Has Reserves And swapByOutput Reverts	DoS	MEDIUM	RESOLVED	p. 164
M-05	Unbacked Output Becomes Unfunded Dust	Logical Error	MEDIUM	ACKNOWLEDGED	p. 166
M-06	Forced Top Up Makes Possible Swaps Revert	DoS	MEDIUM	RESOLVED	p. 168
M-07	Floor Rounding Stalls Height Consumption	DoS	MEDIUM	ACKNOWLEDGED	p. 170
M-08	SwapByInput Can DoS In Valid States	DoS	MEDIUM	ACKNOWLEDGED	p. 172
L-01	ISingleProviderPoolType Missing Inheritance	Best Practices	LOW	RESOLVED	p. 174
L-02	No Validation On Hook-Returned SqrtPrice	Validation	LOW	RESOLVED	p. 175

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-03	getCurrentPriceX96 Is Likely Wrong	Oracle	LOW	ACKNOWLEDGED	p. 177
L-04	Fixed Pool amountOut Exceeds Ceil Bound	Rounding	LOW	RESOLVED	p. 178
L-05	Zero Output Swap Becomes One Output	Rounding	LOW	RESOLVED	p. 181
I-01	Fixed Quoter NatSpec Mismatch	Documentation	INFORMATIONAL	RESOLVED	p. 183
I-02	NatSpec Claims Non-Existent Event	Documentation	INFORMATIONAL	RESOLVED	p. 184

REMEDIATION REVIEW 4

M-01	FixedPoolType LP Sybil Attack	Rewards	MEDIUM	ACKNOWLEDGED	p. 185
M-02	minimumProtocolFee Not Symmetric Across Flows	Math	MEDIUM	ACKNOWLEDGED	p. 186
M-03	Floor Math Arbitrage Extracts Reserves	Rounding	MEDIUM	RESOLVED	p. 187
M-04	Pool Has Reserves And swapByOutput Reverts	DoS	MEDIUM	RESOLVED	p. 190
M-06	Forced Top Up Makes Possible Swaps Revert	DoS	MEDIUM	RESOLVED	p. 192
M-07	Floor Rounding Stalls Height Consumption	DoS	MEDIUM	ACKNOWLEDGED	p. 194
M-08	SwapByInput Can DoS In Valid States	DoS	MEDIUM	ACKNOWLEDGED	p. 196
M-09	minimumProtocolFee Not Reduced On Partial Fill	Math	MEDIUM	ACKNOWLEDGED	p. 198
M-10	Missing Token Flashloan Validation	Validation	MEDIUM	ACKNOWLEDGED	p. 199
M-11	Missing Collect Fees Hook	Logical Error	MEDIUM	ACKNOWLEDGED	p. 200
L-01	Potential DoS on fillOrder	DoS	LOW	ACKNOWLEDGED	p. 202
L-02	Lacking Zero Validation	Warning	LOW	ACKNOWLEDGED	p. 204

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-04	Fixed Pool amountOut Exceeds Ceil Bound	Rounding	LOW	RESOLVED	p. 206
L-05	Zero Output Swap Becomes One Output	Rounding	LOW	RESOLVED	p. 209

REMEDIATION REVIEW 5

M-01	Hook Price Bounds Require Both Swap Hooks	Configuration	MEDIUM	ACKNOWLEDGED	p. 211
M-02	Hook call memory is underallocated	Unexpected Behavior	MEDIUM	RESOLVED	p. 212
I-01	Stale Natspec in AMMModule	Documentation	INFORMATIONAL	RESOLVED	p. 214
I-02	Wrong Error Description	Documentation	INFORMATIONAL	RESOLVED	p. 215

CRITICAL **C-01**

ACKNOWLEDGED

Stale nextHeightAbove Causes Double Height Cross

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/lbamm-pool-type-fixed-team1-1763932948844/blob/mediations/src/libraries/FixedHelper.sol#L686
REVIEW	Main Review

DESCRIPTION

When removing liquidity from the tail end of positions (i.e., when `endHeight == nextHeightAbove`) and the current height falls within this position's range, `_removeLiquidityFromHeight` performs special handling to move `currentHeight` down to `nextHeightBelow`:

```
// FixedHelper.sol#L674-L683
} else {
// End height is equal to next height above so this is the tail end of positions
// Update mapping below to reflect tail, move current height down to tail
mapBelow.nextHeightAbove = nextHeightAbove = nextHeightBelow;
if (nextHeightBelow < height.currentHeight) {
height.currentHeight = nextHeightBelow;
}
}
```

The height state sentinel is also updated to point to this same value:

```
if (height.nextHeightAbove == fromHeight) {
height.nextHeightAbove = nextHeightAbove;
}
```

This results in `height.nextHeightAbove == height.currentHeight`, both pointing to the same value.

Example scenario:

C-01 Stale nextHeightAbove Causes Double Height Cross

CONTINUED

1. A position exists from height 400 → 900, with currentHeight = 600
2. When this position is removed, `currentHeight` moves down to 400, and `nextHeightAbove` is also set to 400
3. Subsequently, new liquidity is added above this height (e.g., a new position from 400 → 1000)
4. The height mapping (linked list) correctly links to the new upper height, but the height state (`height.nextHeightAbove`) remains stale at 400

This occurs because `_addLiquidity` only updates `height.nextHeightAbove` when the condition `height.nextHeightAbove > endHeight` is true. Since `400 > 1000` is false, the update is skipped.

During a subsequent swap that increases height, the pool detects `currentHeight == nextHeightAbove` and triggers a height crossing at 400. However, this is a spurious crossing—the swap is already at height 400 and should not be “crossing” it. This causes `liquidityNet` at height 400 to be incorrectly added to the active liquidity, double-counting positions that start at this height.

In the attached POC, this erroneous liquidity increment causes a mismatch between collected position values and `position1ShareOf1`. When a user attempts to remove liquidity, the subtraction underflows, permanently blocking withdrawals and trapping user funds. The pool’s liquidity accounting becomes corrupted, affecting all subsequent operations.

— RECOMMENDATION

Update the condition in `_addLiquidity` to also refresh `height.nextHeightAbove` when it is stale (i.e., pointing at or below `currentHeight`):

```
if (currentHeight >= startHeight && currentHeight < endHeight) {
  ++height.liquidity;
  ++height.remainingAtHeight;
  if (height.nextHeightBelow < startHeight) {
    height.nextHeightBelow = startHeight;
  }
  // Fix: Also update if nextHeightAbove is stale (pointing to currentHeight or below)
  if (height.nextHeightAbove == 0 || height.nextHeightAbove > endHeight ||
  height.nextHeightAbove <= currentHeight) {
    height.nextHeightAbove = endHeight;
  }
} else {
  // ... existing code ...
}
```

C-01 Stale nextHeightAbove Causes Double Height Cross

CONTINUED

PROOF OF CONCEPT <https://github.com/GuardianOrg/lbamm-core-fuzz-1763932869200/blob/poc-double-crossing/test/fuzzing/FoundryPlayground.sol#L1738>

— RESOLUTION

Status: Acknowledged.

HIGH **H-01**

ACKNOWLEDGED

LP Fees Permanently Locked in Fixed Pool

CATEGORY Logical Error
LOCATION AMMModule.sol, FixedPoolType.sol
REVIEW Main Review

DESCRIPTION

A fee accounting desynchronization exists between the core `AMMModule` and `FixedPoolType` that can result in LP fees being permanently locked in the pool with no recovery mechanism.

When a swap occurs, fees are tracked in two separate locations:

- `AMMModule.sol`: Increments `poolState feeBalance0` with the collected fee
- `FixedHelper.sol`: Should update `feeGrowthGlobal0f0X128` to allow LPs to claim their share

Under certain edge conditions (particularly swaps with zero output), the `feeBalance` is incremented but `feeGrowthGlobal` remains at zero. This occurs because:

- In `_increaseHeight/_decreaseHeight`, the fee distribution loop (`while (remaining != 0)`) only executes when `amount > 0`
- When `amount = 0` but `feeAmount > 0`, the loop is skipped entirely
- The fee is collected at the `AMMModule` level but never recorded in the pool type's fee growth tracking.

The following accounting invariant is violated:

```
sum(claimable_fees_for_all_LPs) == poolState feeBalance0 + poolState feeBalance1
```

After all LPs collect their fees, `feeBalance0` and `feeBalance1` should be zero (or minimal rounding). In the discovered case:

```
feeBalance0 = 974,874,975 (stuck)
feeGrowthGlobal0f0X128 = 0 (no LP can claim)
```

RECOMMENDATION

Ensure fee accounting is consistent between `AMMModule` and `FixedPoolType`. Options include:

H-01 LP Fees Permanently Locked in Fixed Pool

CONTINUED

- Prevent zero-output swaps from collecting fees: If `amountOut = 0`, the swap should either revert or not collect LP fees
- Handle edge case in fee distribution: Modify `_increaseHeight/_decreaseHeight` to properly distribute fees even when `amount = 0`:

PROOF OF CONCEPT <https://github.com/GuardianOrg/lbamm-core-fuzz-1763932869200/blob/fuzz-giraffe/test/fuzzing/FoundryPlayground.sol#L2224>

— RESOLUTION

Status: Acknowledged.

HIGH **H-02**

ACKNOWLEDGED

Missing Height Update Locks LP Funds

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/lbamm-pool-type-fixed-team1-1763932948844/blob/mediations/src/libraries/FixedHelper.sol#L1334
REVIEW	Main Review

DESCRIPTION

In `_increaseHeight`, when `remaining <= heightRemainingLiquidity`, only `remainingAtHeight` is decremented but `currentHeight` is not advanced:

```
} else {  
    heightCache.remainingAtHeight -= uint128(remaining);  
    // currentHeight is NOT updated here!  
}
```

As a result, when removing liquidity this causes `_collectPositionSide` to miscalculate `pairValue`. The calculation `liquidity - sideValue` evaluates to zero because `sideValue = endHeight - currentHeight` uses the stale `currentHeight`:

```
sideValue = endHeight - currentHeight; // eg., 9.654e21 - 0 = 9.654e21  
pairValue = calculateFixedInput(liquidity - sideValue, sqrtPriceX96, sideZero);  
// calculateFixedInput(9.654e21 - 9.654e21, ...) = 0  
  
test_poc_staleHeight demonstrates this issue:
```

H-02 Missing Height Update Locks LP Funds

CONTINUED

1. A Fixed pool is created with a very high `sqrtPriceX96` (clamped near `MAX_SQRT_RATIO`).
2. An LP adds liquidity: `9.654e21 token0` (`height0: 0→9.654e21`) and `5.119e21 token1` (`height1: 0→5.119e21`).
3. A swap executes with `amount0out = 1` (`token0`), which requires `reserveAmountIn = 1.845e19 token1` due to the extreme price ratio. The swap correctly deposits `1.845e19 token1` into reserves.
4. In `_increaseHeight`, since `remaining (1) <= heightRemainingLiquidity (1)`, only `remainingAtHeight` is decremented to 0, but `currentHeight0` remains at 0.
5. When the LP removes all liquidity, `_collectPositionSide` calculates:
 - `sideValue = 9.654e21 - 0 = 9.654e21`
 - `pairValue = calculateFixedInput(9.654e21 - 9.654e21, ...) = 0`
 - After the `--sideValue` correction: LP receives `9.654e21 - 1 token0` but only their original `5.119e21 token1`.
6. The `1.845e19 token1` from the swap remains stuck in `reserve1`, failing the invariant check that reserves should be dust after all liquidity is removed.

Impact: Liquidity providers suffer direct fund loss equal to the swap input amounts that should have been credited to their positions. These tokens become permanently unrecoverable as they remain in reserves with no accounting mechanism to distribute them.

— RECOMMENDATION

Modify the `_increaseHeight` function to properly advance `currentHeight` when all remaining liquidity at the current height is consumed. See `[\[commit\]](https://github.com/GuardianOrg/lbamm-core-fuzz-1763932869200/commit/74a6e627ed045e06d5fcfc459ce520803bd39990)`.

Suggested fix: <https://github.com/GuardianOrg/lbamm-core-fuzz-1763932869200/commit/74a6e627ed045e06d5fcfc459ce520803bd39990>

PROOF OF CONCEPT <https://github.com/GuardianOrg/lbamm-core-fuzz-1763932869200/commit/b40d96b8fdb870dedf696022639fbc33f5abeb34>

— RESOLUTION

Status: Acknowledged.

HIGH **H-03**

ACKNOWLEDGED

Add Liquidity To Height DoS

CATEGORY Logical Error
LOCATION FixedHelper.sol
REVIEW Main Review

DESCRIPTION

The `_addLiquidityToHeight` function adds 1 liquidity unit to a specific `toHeight` by incrementing `heightInfo[toHeight].liquidityGross`. If this height was previously unused, it inserts `toHeight` into the sorted double linked list, `heightMap`, using `informationHeight` as a cursor. The `while(true)` in `_addLiquidityToHeight` only runs when `liquidityGross` transitions from 0 to 1. That means, the height was empty immediately before this call. The contract enters a loop to find a spot to insert a new height, and it handles all possible cases except one. There is no handling for: `toHeight == informationHeight`. When `toHeight == informationHeight` and `informationNextHeightBelow | informationNextHeightAbove != 0`, no other conditions execute, so the loop keeps running forever. This can be weaponized when we see how the code handles height 0. In a Fixed pool, when we remove all liquidity from a height, the contract deletes that node from the linked list so the chain stays clean. The code treats height 0 as a special anchor. Looking at `_removeLiquidityFromHeight`:

```
if (fromHeight != 0) {  
  // If it's not 0, clear the pointers (removing from list)  
  mapHeight.nextHeightBelow = 0;  
  mapHeight.nextHeightAbove = 0;  
}  
// If it is 0, we do nothing and leave the pointers alive.
```

Even if height 0 has 0 liquidity, it remains inside the linked list structure. When the missed branch state executes, the contract starts walking the linked list to find where to put height 0. Eventually, the the cursor position lands on 0. At this moment:

```
Target height 'toHeight' = 0  
Current cursor 'informationHeight' = 0  
  
while (true) {  
  // load next and previous pointers  
  // Case 1: Is the target HIGHER than where we are?  
  if (toHeight > informationHeight) {
```

H-03 Add Liquidity To Height DoS

CONTINUED

```
break; // insert here
}
// Case 2: Is the target LOWER than where we are?
else if (toHeight < informationHeight) {
break; // insert here
}
// Case 3: Navigate Down
else if (toHeight < informationHeight) {
// move cursor down
}
// Case 4: Navigate Up
else if (toHeight > informationHeight) {
// move cursor up
}
// @Audit: What if toHeight == informationHeight?
// The code does nothing. It loops back to the start.
// And it runs forever.
}
```

The code only expects the target to be greater than or less than the current node. It never expects to bump into the target itself. For example: The pool has liquidity at height 100, but height 0 was emptied earlier. Height 0 is now empty but still linked. A user calls `depositLiquidity` starting at height 0. The contract sees that height 0 is empty, so it enters the insertion loop infinitely. By exploiting this, an attacker can cause `depositLiquidity()` to become uncallable. This can only happen if it is called with a `startHeight` of 0. If the start height is higher, it will not enter the state that infinitely loops. If an attacker exploits this for any pool, they can cause a global DoS preventing any liquidity add calls, even if `addInRange = false` is used, it can still be forced to include a `startHeight` of 0. In `_calculateLiquidityStartAndEndHeights`, if `currentHeight == 0`, the code takes the early branch:

```
if (currentHeight % precision == 0) {
startHeight = currentHeight;
}
```

regardless of `addInRange` an attacker can cause `currentHeight = 0` if they consume all liquidity in that side via swaps, walking downward until no height remains. Or remove all liquidity positions below the current height, via `removeLiquidity` if they own all the positions. After this point no one would be able to add liquidity until a swap is performed to move `currentHeight` above 0.

H-03 Add Liquidity To Height DoS

CONTINUED

— RECOMMENDATION

Add a check inside the loop to handle the equality case. If we find that the node already exists, we should break the loop so the system can keep working with the next heights:

```
else if (toHeight == informationHeight) {  
  // The node already exists in the chain  
  // We don't need to link it again.  
  break;  
}
```

PROOF OF CONCEPT <https://gist.github.com/minato7namikazi/7ab68a15dc5060130f8daab10b45cb98>

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-01**

ACKNOWLEDGED

Empty-pool swaps set arbitrary price

CATEGORY Unexpected Behavior
LOCATION DynamicHelper.sol
REVIEW Main Review

DESCRIPTION

The swap entrypoints accept and process swaps when pool liquidity is zero. In `DynamicPoolType.swapByInput / swapByOutput`, pool state is loaded and passed directly into `computeSwap` without any zero-liquidity guard:

```
DynamicPoolState storage ptrPoolState = ammState.pools[poolId];
DynamicPoolState memory poolState = ptrPoolState;
swapCache.sqrtPriceCurrentX96 = poolState.sqrtPriceX96;
swapCache.tick = poolState.tick;
swapCache.liquidity = poolState.liquidity;
DynamicHelper.computeSwap(ammState, swapCache, poolState, uint16(poolFeeBPS),
DynamicHelper._swapByInputStep);
ptrPoolState.sqrtPriceX96 = swapCache.sqrtPriceCurrentX96;
ptrPoolState.tick = swapCache.tick;
```

`computeSwap` then iterates even when `swapCache.liquidity` is zero and advances price/tick to the limit, while fee updates are skipped only when liquidity is zero:

```
while (swapCache.amountSpecifiedRemaining != 0 && swapCache.sqrtPriceCurrentX96 !=
swapCache.sqrtPriceLimitX96) {
// swap step executes with zero liquidity, amounts stay zero
_swapStep(swapCache, step, poolFeeBPS);
swapCache.sqrtPriceCurrentX96 = swapCache.sqrtPriceNextX96;
if (swapCache.liquidity > 0) {
swapCache.feeGrowthGlobalX128 += FullMath.mulDiv(step.feeAmount, Q128, swapCache.liquidity);
}
swapCache.tick = swapCache.zeroForOne ? step.tickNext - 1 : step.tickNext;
}
```

Because the updated `sqrtPriceCurrentX96` and `tick` are written back unconditionally, anyone can call a swap on an empty pool and push the stored price/tick to an arbitrary bound without consuming input or providing liquidity. Impact: the first LP or integrator sees a

M-01 Empty-pool swaps set arbitrary price

CONTINUED

manipulated starting price; initial deposits may need skewed ratios or fail, and any quoting/routing that trusts the stored price is wrong until later activity repositions the pool. This is a price-tampering/DoS during bootstrapping.

— RECOMMENDATION

Revert early when a pool has zero liquidity before entering `computeSwap` (and when `amountSpecifiedRemaining` is zero). Additionally, avoid updating `sqrPrice/tick` when no liquidity was consumed in the swap path to prevent state changes on empty pools.

PROOF OF CONCEPT <https://gist.github.com/GuardianAudits/37877e98d3952cb7a575aec033f26d14>

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-02**

ACKNOWLEDGED

Partial fills can zero LP/protocol fees

CATEGORY Rounding
LOCATION AMModule.sol
REVIEW Main Review

DESCRIPTION

When a pool returns a partial fill in `_poolSwapByInput`, the AMM scales `expectedLPFee` and `expectedProtocolLPFee` by `actualAmountIn/originalAmountIn` using `FullMath.mulDiv`, which floors the result. For small partial fills this floor can drop both expectations to zero even though the configured `poolFeeBPS` is nonzero. `_validateProtocolFees` then sees an expected LP fee of zero and accepts whatever the pool reports, so a pool-type that returns `poolFeeOfAmountIn=0` and `poolProtocolFees=0` on a tiny partial fill will pass validation. That can make input swaps fee-free when the pool underfills, bypassing both LP and protocol fee collection despite a nonzero pool fee configuration.

Relevant code:

```
// AMModule.sol:1367-1420 (partial fill scaling)
(
    actualAmountIn,
    tmpSwapCache.amountOut,
    poolFeeOfAmountIn,
    poolProtocolFees
) = ILimitBreakAMMPoolType(PoolDecoder.getPoolType(tmpSwapCache.poolId)).swapByInput(
    tmpSwapCache.context,
    tmpSwapCache.poolId,
    tmpSwapCache.zeroForOne,
    tmpSwapCache.amountIn,
    poolFeeBPS,
    lpFeeBPS,
    tmpSwapHooksExtraData.poolType
);
uint256 originalAmountIn = tmpSwapCache.amountIn;
if (actualAmountIn != originalAmountIn) {
    // Adjust fees to compensate for partial fill (floored)
    tmpSwapCache.expectedLPFee = FullMath.mulDiv(tmpSwapCache.expectedLPFee, actualAmountIn,
    originalAmountIn);
```

M-02 Partial fills can zero LP/protocol fees

CONTINUED

```
tmpSwapCache.expectedProtocolLPFee = FullMath.mulDiv(tmpSwapCache.expectedProtocolLPFee,
actualAmountIn, originalAmountIn);
tmpSwapCache.amountIn = actualAmountIn;
uint256 amountInAdjustment = originalAmountIn - actualAmountIn;
uint256 exchangeFeeAdjustment = FullMath.mulDiv(tmpSwapCache.exchangeFeeAmount,
amountInAdjustment, originalAmountIn);
uint256 protocolExchangeFeeAdjustment =
FullMath.mulDiv(tmpSwapCache.protocolExchangeFeeAmount, amountInAdjustment, originalAmountIn);
tmpSwapCache.adjustedAmountSpecified = tmpSwapCache.adjustedAmountSpecified -
amountInAdjustment
- exchangeFeeAdjustment - protocolExchangeFeeAdjustment;
tmpSwapCache.exchangeFeeAmount -= exchangeFeeAdjustment;
tmpSwapCache.protocolFeeFromFees -= protocolExchangeFeeAdjustment;
}

// AMModule.sol:2594-2664 (expected fees initially rounded up)
uint256 expectedLPFee = FullMath.mulDivRoundingUp(swapAmountIn, poolFeeBPS, MAX_BPS);
uint256 expectedProtocolLPFee = FullMath.mulDiv(
expectedLPFee,
swapCache.protocolFeeStructure.lpFeeBPS,
MAX_BPS
);
// ...later floored by mulDiv in the partial-fill branch shown above

// FeeHelper.calculateAmountAfterFeesSwapByInput (fee-on-top/exchange computed once)
uint256 feeOnTopAmount = feeOnTop.amount;
if (feeOnTopAmount > 0) {
if (feeOnTopAmount > amountInAfterFees) revert LBAMM__FeeAmountExceedsInputAmount();
amountInAfterFees -= feeOnTopAmount;
(uint256 feeOnTopAmountToRecipient, uint256 protocolFeeOnTopAmount) =
_calculateFlatFeeWithRecipientAndProtocolFee(
feeOnTopAmount,
protocolFeeStructure.feeOnTopBPS
);
protocolFeesFromSwap = protocolFeeOnTopAmount;
swapCache.feeOnTopAmount = feeOnTopAmountToRecipient;
}

// AMModule._finalizeSwapCollectFundsAndDisburse: fee-on-top paid even after partial fill
uint256 feeOnTopAmount = swapCache.feeOnTopAmount;
```

M-02 Partial fills can zero LP/protocol fees

CONTINUED

```
if (feeOnTopAmount > 0) {  
  SafeERC20.safeTransfer(swapOrder.tokenIn, feeOnTop.recipient, feeOnTopAmount);  
  netAmountIn -= feeOnTopAmount;  
}
```

— RECOMMENDATION

Recompute the LP/protocol fee expectations on the pool-reported `actualAmountIn` using rounding up and require the pool-reported `poolFee0fAmountIn` and protocol share to be at least those expectations even after partial fills.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-03**

ACKNOWLEDGED

Direct Swap Max-Price Bound Bypass via Overflow

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/lbamm-hooks-and-handlers-team1-1763932904310/blob/remediations/src/hooks/AMMStandardHook.sol#L719
REVIEW	Main Review

DESCRIPTION

For direct swaps, `_validatePricingBounds` reconstructs a synthetic price in `afterSwap` using `SqrtPriceCalculator.computeRatioX96(amount1, amount0)`.

When the implied price is extremely large, `computeRatioX96` overflows the `uint160` return and deliberately returns 0. The subsequent bound checks only revert if `sqrtPriceX96 > maxSqrtPriceX96`; with `minSqrtPriceX96` unset (0), an overflowed 0 value skips the upper-bound check entirely, letting a direct swap execute at a price that should have been rejected.

RECOMMENDATION

Treat `computeRatioX96` returning 0 as an overflow and revert (or clamp to `type(uint160).max` before comparison).

RESOLUTION

Status: Acknowledged.

MEDIUM **M-04**

ACKNOWLEDGED

FixedPoolType LP Sybil Attack

CATEGORY	Rewards
LOCATION	FixedPoolType
REVIEW	Main Review

— DESCRIPTION

Swaps will consume the current height before moving on to the next one. Multiple users are able to open positions in the same height and only if all of them are consumed, will the system touch the liquidity of the next height.

This logic enables a sybil attack which allows a single LP to likely gain all the fees while passive LPs which deposited liquidity among multiple heights own a very capital inefficient position:

- Precision of the FixedPool is 100
- Alice deposits \$100k liquidity from height 0 to 100k
- Eve creates 1000 different accounts and deposits \$100 liquidity with every account from height 0 to 100
- Now Eve will earn fees on \$100k liquidity while Alice only earns fees on \$100 liquidity before the rest of Alice's liquidity is touched. And Eve is able to add more liquidity to her position before that happens, which means Alice will likely never earn more fees.

Therefore a sybil attack is way more profitable for LPs than using the system as intended.

— RECOMMENDATION

Be aware about this and consider to rethinking the FixedPool logic in general or to document the behavior.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-05**

ACKNOWLEDGED

directSwap returns inverted amounts

CATEGORY Unexpected Behavior
LOCATION LimitBreakAMM.sol
REVIEW Main Review

DESCRIPTION

The public `LimitBreakAMM.directSwap` API advertises `(amountIn, amountOut)` where `amountOut` is the "amount of output tokens transferred to recipient". In the implementation, `_finalizeDirectSwap` returns the maker's token sent to the executor (`tokenInToExecutor`) and `directSwap` assigns that value to `amountOut`, while `amountIn` is set to `swapCache.amountOut`, the tokens the executor paid to the recipient. The return tuple is inverted relative to the documented semantics: callers believing `amountOut` was delivered to the recipient will misaccount or make wrong settlement decisions. In a reproduction, `amountOut` (~9.3e20) never reached the recipient, because that value represents the opposite side of the trade. Any integration or off-chain logic that trusts the return values for payout/slippage checks will be working with inverted amounts and can double-charge or under-credit. On-chain execution still moves tokens correctly, but the documentation in the contract is incorrect and can break consumers.

In `directSwap` the executor (taker) supplies `tokenOut` directly to the recipient (maker) and receives `tokenIn` from the maker; fees are taken from the token-in side. Example with concrete numbers: Alice (executor) buys 100 USDC (tokenIn) from Bob (maker/recipient) paying 0.05 WETH (tokenOut). On-chain transfers: Alice pays Bob 0.05 WETH; Bob pays Alice 100 USDC minus fees. The function currently returns `(amountIn=0.05 WETH, amountOut=100 USDC)`, but the docstring says `amountOut` is the amount "transferred to recipient" (Bob actually received 0.05 WETH, which is in `amountIn`).

```
* @return amountIn Total amount of input tokens collected including fees.  
* @return amountOut Amount of output tokens transferred to recipient.
```

RECOMMENDATION

Align return values with the documented semantics. After `_finalizeDirectSwap`, set `amountIn = swapCache.amountIn` (input collected) and `amountOut = swapCache.amountOut` (tokens delivered to recipient), or adjust `_finalizeDirectSwap` to return the recipient transfer amount and mirror that in `directSwap`. Ensure NatSpec matches the actual values returned.

M-05 directSwap returns inverted amountsCONTINUED

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-06**

ACKNOWLEDGED

minimumProtocolFee Not Symmetric Across Flows

CATEGORY Math

LOCATION <https://github.com/GuardianOrg/lbamm-core-team1-1763932865865/blob/remediations/src/modules/AMMModule.sol>

REVIEW Main Review

DESCRIPTION

The `minimumProtocolFee` is calculated based on the `amountIn` in both the `SwapByInput` and `SwapByOutput` flow. However different fees and other factors adjust the `amountIn` before this calculation in different ways depending on the chose swap type:

- `SwapByInput`
- `amountIn` decreased by `exchangeFee` & `feeOnTop`
- `minimumProtocolFee` calculated based on `amountIn`
- `SwapByOutput`
- `amountOut` increased by `beforeSwapHookFees` (influencing needed `amountIn`)
- swap happens: `amountIn` may be decreased as only a partial fill was possible
- `minimumProtocolFee` calculated based on needed `amountIn`

As we can see the `minimumProtocolFee` charged may differ and therefore the users may pay more or less fees depending on if they use the `SwapByInput` or `SwapByOutput` function.

RECOMMENDATION

Consider to make adjust these flows so that the `minimumProtocolFee` is the same no matter if the user decides to use the `SwapByInput` or `SwapByOutput` function.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-07**

ACKNOWLEDGED

minimumProtocolFee Not Reduced On Partial Fill

CATEGORY	Math
LOCATION	https://github.com/GuardianOrg/lbamm-core-team1-1763932865865/blob/d5435e12c4ceeb6975468d41694eaf3d7e68525e/src/modules/AMMModule.sol#L1400-L1429
REVIEW	Main Review

— DESCRIPTION

In case only a partial fill was possible fees like for example the `exchangeFee` is adjusted to match the new swap amount.

However the `minimumProtocolFee` was already deducted from the `amountIn` before the swap happened in the `SwapByInput` flow and is not decremented in case of a partial fill. Therefore the user could pay way too much fees in that case.

— RECOMMENDATION

Consider to adjust this charged fee and give it back to the user.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-08**

ACKNOWLEDGED

Asymmetric Price Bounds Validation

CATEGORY Validation

LOCATION <https://github.com/GuardianOrg/lbamm-hooks-and-handlers-team1-1763932904310/blob/39b3b0e5122f5c75ad082aca52f97eca4212450a/src/hooks/AMMStandardHook.sol#L693-L740>

REVIEW Main Review

— DESCRIPTION

For normal pool swaps the `_validatePricingBounds` function uses the current `sqrtPrice` of the given pool.

But for direct swaps the function tries to calculate the price based on the given `amountIn` and `amountOut`.

However the given `amountIn` may or may not already be deducted by the `exchangeFee` and `feeOnTop` depending on if this is a `SwapByInput` or a `SwapByOutput`, this will therefore manipulate the price and with it the outcome of the `_validatePricingBounds` check.

Therefore a direct `SwapByInput` could revert due to the `_validatePricingBounds` check while a direct `SwapByOutput` does not (or the other way around).

— RECOMMENDATION

Consider to use the original `amountIn` & `amountOut` values for this validation or to document this behavior.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-09**

ACKNOWLEDGED

Fixed swapByInput strands "change" in reserves

CATEGORY Unexpected Behavior
LOCATION FixedHelper.sol
REVIEW Main Review

DESCRIPTION

The fixed pool's input-based swap path can accept more input than is actually needed to produce the discrete, integer `amountOut`, and the leftover "change" is neither refunded to the swapper nor attributed to LPs as fees. This creates protocol-level accounting drift where `PoolState.reserveIn` increases by the full post-fee input, while the fixed-pool internal accounting only "uses" the portion that corresponds to the floored `amountOut`. When all LP positions are later unwound, this remainder is not withdrawable and remains stuck in the pool as non-dust reserves.

In `FixedHelper.swapByInput`, the swap computes `amountInAfterFees` and then calculates the output using `calculateFixedInput` (which floors via `FullMath.mulDiv`):

```
(amountInAfterFees, lpFeeAmount, protocolFeeAmount) =  
_calculateInputLPAndProtocolFee(swapCache.amountIn, poolFeeBPS, swapCache.protocolFeeBPS);  
uint256 amountOut = swapCache.amountOut =  
calculateFixedInput(amountInAfterFees, swapCache.sqrtPriceCurrentX96, swapCache.zeroForOne);  
swapCache.feeAmount = lpFeeAmount;  
swapCache.protocolFee = protocolFeeAmount;  
_applySwapToLiquidity(ptrPoolState, swapCache, amountInAfterFees, amountOut, lpFeeAmount);
```

The issue is that `amountOut` is an integer that can be significantly smaller than the continuous-price expectation due to flooring, but `_applySwapToLiquidity` is still applied using the full `amountInAfterFees`. There is no subsequent step that computes the minimum reserve input required for that discrete `amountOut` and no adjustment to `swapCache.amountIn` (the "actualAmountIn" returned to the core) to enable refunding the unused remainder.

In the POC attached, the fixed pool price is extremely high (`sqrtPriceX96 = 340307949074909932165501021441157658275`), so swapping token1->token0 produces an `amountOut` that floors

M-09 Fixed swapByInput strands “change” in reserves

CONTINUED

to 1 while consuming a very large input amount. After the pool fee, the swap has `amountInAfterLPFee = 20380619700000000000`, but the fixed-price input required for `amountOut = 1` at that price is only 18449517799894629113. The difference is:

```
remainder = 20380619700000000000 - 18449517799894629113 = 1931101900105370887
```

That remainder ends up persisted in the core pool's `reserve1`, but it is not claimable by LPs upon full withdrawal, so after all liquidity is removed the pool still has `reserve1 = 1931101900105370887` and fails the “pool exited” invariant.

This is not merely an acceptable rounding dust effect. The stranded amount can be large (up to almost the per-unit “price” for one output unit in the relevant direction) and can accumulate across swaps, permanently locking user-paid value inside the pool and breaking full-exit expectations.

Impact: swappers can be overcharged (effectively donating unrefundable value), and the pool can retain significant, unclaimable reserves after all liquidity is removed, causing accounting drift and invariant failures.

— RECOMMENDATION

Make the fixed pool's `swapByInput` a true “consume up to input” path by returning an `actualAmountIn` that reflects only what is needed to produce the discrete `amountOut`. After computing `amountOut`, compute the minimum reserve input needed using `calculateFixedOutput(amountOut, sqrtPriceX96, zeroForOne)`, then recompute fees consistently so the returned `actualAmountIn` causes the core's existing partial-fill logic to refund the remainder.

A minimal sketch of the intent (not exact code) is:

```
uint256 reserveInNeeded = calculateFixedOutput(amountOut, sqrtPriceX96, zeroForOne);
uint256 actualAmountIn = reserveInNeeded + feeAmountForReserveInNeeded;
```

This ensures the core only transfers/credits what is actually consumed by the swap outcome, and prevents orphaned “change” from being trapped in `reserveIn`.

Suggested fix was added here: <https://github.com/GuardianOrg/lbamm-core-fuzz-1763932869200/commit/be6bb5b5278bcb90a18e76f91ec97f49ece907c3>

PROOF OF CONCEPT <https://github.com/GuardianOrg/lbamm-core-fuzz-1763932869200/blob/74a6e627ed045e06d5fcfc459ce520803bd39990/test/fuzzing/FoundryPlayground.sol#L1971-L1979>

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-10**

ACKNOWLEDGED

Fixed pool LP exit can revert on fee dust

CATEGORY DoS
LOCATION FixedHelper.sol; FixedPoolType.sol
REVIEW Main Review

DESCRIPTION

The fixed pool implementation uses two separate accounting buckets in the AMM: principal reserves (`reserve0` / `reserve1`) and accrued LP fees (`feeBalance0` / `feeBalance1`). On swaps, the AMM credits the exact LP fee amount into the `feeBalance` bucket:

```
if (swapCache.zeroForOne) {  
    ptrPoolState.reserve0 = _safeIncrementUint128(ptrPoolState.reserve0, reserveIn);  
    ptrPoolState.reserve1 = _safeDecrementUint128(ptrPoolState.reserve1, swapCache.amountOut);  
    ptrPoolState.feeBalance0 = _safeIncrementUint128(ptrPoolState.feeBalance0,  
    poolFeeOfAmountIn);  
} else {  
    ptrPoolState.reserve0 = _safeDecrementUint128(ptrPoolState.reserve0, swapCache.amountOut);  
    ptrPoolState.reserve1 = _safeIncrementUint128(ptrPoolState.reserve1, reserveIn);  
    ptrPoolState.feeBalance1 = _safeIncrementUint128(ptrPoolState.feeBalance1,  
    poolFeeOfAmountIn);  
}
```

At the fixed-pool type layer, however, per-position fees are tracked using Q128 “fee growth” accumulators. Q128 is defined as:

```
uint256 constant Q128 = 2 ** 128;
```

Fee growth is incremented by distributing fees across the active “liquidity” count at each height using integer division, which truncates remainders:

```
uint256 feeGrowthGlobalIncrement = UnsafeMath.simpleMulDiv(  
    feeDistributedToHeight,  
    Q128,  
    heightCache.liquidity  
);  
if (zeroForOne) {
```

M-10 Fixed pool LP exit can revert on fee dust

CONTINUED

```
heightCache.feeGrowthGlobal0f0X128 += feeGrowthGlobalIncrement;
} else {
heightCache.feeGrowthGlobal0f1X128 += feeGrowthGlobalIncrement;
}
```

When fees are later collected/realized for a position, the position's fee amount is also computed with truncation by dividing the fee-growth delta by `Q128`:

```
(uint256 feeGrowthInside0X128, uint256 feeGrowthInside1X128) = _getFeeGrowthInside(
heightInfo,
height,
startHeight,
endHeight,
currentHeight
);
fee0 = (feeGrowthInside0X128 - feeGrowthInside0LastX128) / Q128;
fee1 = (feeGrowthInside1X128 - feeGrowthInside1LastX128) / Q128;
```

This combination means swaps can create "fee dust" in `feeBalance0/1` that is not representable (or not perfectly distributable) via the `Q128` fee-growth accounting. In the POC provided, the first swap that generates token0 LP fees is:

```
emit Swap(
poolId: 0x000000000000000000000000bb44bb7dd22364c7cf5e2fb09c841100000202196d,
zeroForOne: true,
lpFeeAmount: 425635831915043762
)
```

Immediately after, the AMM pool state shows that exact fee credited to `feeBalance0`:

```
PoolState({
reserve0: 7894109463277092671500,
reserve1: 715694126042162472900,
feeBalance0: 425635831915043762,
feeBalance1: 1245250849440000000
})
```

Because the fixed pool's fee growth uses division by the active liquidity count, the amount that can be expressed/claimed via per-position fee growth is truncated, leaving a small remainder (in this POC, the remainder evolves from `2` wei after the first token0-fee swap to `1` wei after the next token0-fee swap with `lpFeeAmount: 949040`, which changes `feeBalance0` to `425635831915992802`).

M-10 Fixed pool LP exit can revert on fee dust

CONTINUED

The critical part is that the AMM enforces strict separation of buckets on liquidity removal. During `removeLiquidity`, the pool type returns `(withdraw0, withdraw1, fees0, fees1)` (computed by the fixed-pool logic, which itself uses the truncated fee calculation) and the AMM then decrements principal and fee buckets independently:

```
function withdrawAll(
bytes32 poolId,
FixedLiquidityWithdrawAllParams memory withdrawAllParams,
FixedPoolState storage ptrPoolState,
FixedPositionInfo storage position
) internal returns (uint256 withdraw0, uint256 withdraw1, uint256 fee0, uint256 fee1) {
    (withdraw0, withdraw1, fee0, fee1) = _collectPosition(ptrPoolState, position);
    if (withdraw0 < withdrawAllParams.minAmount0 || withdraw1 < withdrawAllParams.minAmount1) {
        revert FixedPool__InsufficientLiquidityForRemoval();
    }
    position.startHeight0 = 0;
    position.endHeight0 = 0;
    position.feeGrowthInside00f0LastX128 = 0;
    position.feeGrowthInside10f0LastX128 = 0;
    position.startHeight1 = 0;
    position.endHeight1 = 0;
    position.feeGrowthInside00f1LastX128 = 0;
    position.feeGrowthInside10f1LastX128 = 0;
}

if (withdraw0 > 0) {
    poolState.reserve0 = _safeDecrementUint128(poolState.reserve0, withdraw0);
}
if (withdraw1 > 0) {
    poolState.reserve1 = _safeDecrementUint128(poolState.reserve1, withdraw1);
}
if (fees0 > 0) {
    poolState.feeBalance0 = _safeDecrementUint128(poolState.feeBalance0, fees0);
}
if (fees1 > 0) {
    poolState.feeBalance1 = _safeDecrementUint128(poolState.feeBalance1, fees1);
}
```

The decrement helper reverts on underflow:

M-10 Fixed pool LP exit can revert on fee dust

CONTINUED

```
function _safeDecrementUint128(uint256 a, uint256 b) internal pure returns (uint128
difference) {
    assembly ("memory-safe") {
        difference := sub(a, b)
        if gt(difference, a) {
            mstore(0x00, 0x99EFB929) //SafeCast__Uint128overflow()
            revert(0x1C, 0x04)
        }
    }
}
```

In the POC, USER10 attempts to exit the fixed pool and the pool type returns values that are internally consistent in total, but mis-split by 1 wei between `withdraw0` and `fees0`:

```
FixedPositionInfo({
    startHeight0: 0,
    endHeight0: 4370000,
    startHeight1: 0,
    endHeight1: 715239630949762746700
})

← [Return] positionId,
withdraw0 = 235797060064557277,
withdraw1 = 715003833889702559422,
fees0 = 425225300332944329,
fees1 = 7880651
```

The intended total exit amounts for this position are `withdraw0 + fees0 = 661022360397501606` token0 and `withdraw1 + fees1 = 715003833889710440073` token1, but the AMM reverts before state updates and transfers, so the LP cannot exit at all. Right after, the pool's stored accounting buckets are:

```
PoolState({
    reserve0: 235797060064557276,
    reserve1: 715003833889702559424,
    feeBalance0: 425225300332944330,
    feeBalance1: 7880652
})
```

M-10 Fixed pool LP exit can revert on fee dust

CONTINUED

This makes `withdraw0 == reserve0 + 1` and `fees0 == feeBalance0 - 1` (the overall token0 sum still matches), so the AMM underflows when it tries to decrement `reserve0` by `withdraw0` and reverts:

```
← [Revert] SafeCast__Uint128overflow()
```

This can look confusing when inspecting the fixed pool's height state because the call trace also prints `FixedHeightState(... liquidity: 1, remainingAtHeight: 0, ...)`. That `liquidity: 1` is the active height-interval "liquidity" denominator used for fee-growth math at the current height, not the token reserve. The actual reserves are in `PoolState.reserve0/reserve1` above, and they are large; the exit fails solely because a 1-wei accounting misclassification causes a `uint128` underflow revert.

Impact: A 1 wei fee-growth rounding remainder can cause `removeLiquidity/withdrawAll` to revert, preventing an LP from exiting and effectively locking large amounts of principal + fees in the pool until state changes.

— RECOMMENDATION

Ensure that fee-growth rounding remainders cannot cause a bucket-split that violates `withdrawX <= reserveX` and `feesX <= feeBalanceX`. The most robust fix is to carry/distribute fee-growth remainders so that the sum of per-position claimable fees matches `feeBalance` over time. As a defensive mitigation at the AMM layer, reclassify dust between `withdraw` and `fees` before decrementing buckets when the total is covered but one bucket would underflow (for example, shift `withdraw0 - reserve0` from `withdraw0` into `fees0` only when `fees0 + delta <= feeBalance0`), so exits cannot be DoS'd by 1-wei precision loss.

PROOF OF CONCEPT <https://github.com/GuardianOrg/lbamm-core-fuzz-1763932869200/commit/315aff75882ad3f33e04e34993f4d7e2270f3a1e>

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-11**

ACKNOWLEDGED

Fixed pool reserve desync from split rounding

CATEGORY Unexpected Behavior
 LOCATION FixedHelper.sol; FixedPoolType.sol
 REVIEW Main Review

DESCRIPTION

The fixed pool design maintains two separate but interdependent accounting layers that must remain synchronized after every swap: (1) the core AMM `PoolState` reserves (`reserve0` / `reserve1`, plus fee buckets) and (2) the pool-type-fixed internal state (`position0Share0f0` / `position1Share0f1` and the height trackers, including `consumedLiquidity0` / `consumedLiquidity1`). The core AMM updates `PoolState` purely from the (`amountIn`, `amountOut`) values returned by the pool type, while the fixed pool simultaneously mutates its own internal state to reflect the same swap. If the fixed pool mutates its internal "consumed"/"principal" buckets in a way that is not algebraically consistent with the amounts the AMM credited/debited, the pool can enter a state where `PoolState.reserve*` no longer matches what the fixed pool's own state implies should be in reserves.

In the fixed pool, `_updateFixedPoolHeights` attempts to "split" each swap into virtual portions attributed to each side (`height0` vs `height1`) based on the pool's current composition. In the `zeroForOne` path, the output-side split uses floor rounding, but the input-side split uses ceiling rounding:

```
if (swapCache.zeroForOne) {
  uint256 position0Share0f1 = swapCache.pairedShare0fExpectedReserve;
  uint256 virtualReserve1 = swapCache.expectedReserve;
  uint256 amount1FilledByHeight0 = FullMath.mulDiv(amount1, position0Share0f1,
  virtualReserve1);
  amount1FilledByHeight1 = amount1 - amount1FilledByHeight0;
  amount0FilledByHeight0 = FullMath.mulDivRoundingUp(amount0, position0Share0f1,
  virtualReserve1);
  uint256 lpFeeAmountFor1 = FullMath.mulDiv(lpFeeAmount, amount1FilledByHeight1, amount1 > 0 ?
  amount1 : 1);
  uint256 lpFeeAmountFor0 = lpFeeAmount - lpFeeAmountFor1;
  _decreaseHeight(ptrPoolState.height0, ptrPoolState.heightInfo0, ptrPoolState.heightMap0,
  amount0FilledByHeight0, lpFeeAmountFor0, true);
  _increaseHeight(ptrPoolState.height1, ptrPoolState.heightInfo1, ptrPoolState.heightMap1,
```

M-11 Fixed pool reserve desync from split rounding

CONTINUED

```

amount1FilledByHeight1, lpFeeAmountFor1, true);
ptrPoolState.position0ShareOf0 += uint128(amount0FilledByHeight0);
ptrPoolState.position1ShareOf1 -= uint128(amount1FilledByHeight1);
}`

```

This asymmetry is not “precision loss noise”; it can deterministically create a large accounting gap when the pool price is extreme and one of the swap amounts is very small. The reason is that fixed pools convert between token0 and token1 using two-step Q96 math, and the inverse direction uses rounding up:

```

function calculateFixedOutput(
uint256 amountOut,
uint160 sqrtPriceX96,
bool zeroForOne
) internal pure returns (uint256 amountIn) {
if (zeroForOne) {
amountIn = FullMath.mulDivRoundingUp(amountOut, Q96, sqrtPriceX96);
amountIn = FullMath.mulDivRoundingUp(amountIn, Q96, sqrtPriceX96);
} else {
amountIn = FullMath.mulDivRoundingUp(amountOut, sqrtPriceX96, Q96);
amountIn = FullMath.mulDivRoundingUp(amountIn, sqrtPriceX96, Q96);
}
}
}

```

In the POC’s fixed price (`sqrtPriceX96 = 340307949074909932165501021441157658272`), a 1-unit change in “token0 owed/consumed” can correspond to an $\sim 1.8449517804189919302e19$ unit shift in required token1 backing (`calculateFixedOutput(1, sqrtPriceX96, false)`), i.e. ~ 18.45 token1 (18 decimals) per single token0 base unit. When `_updateFixedPoolHeights` uses `mulDivRoundingUp` to compute `amount0FilledByHeight0`, it can round a small fractional allocation up to `1`, which decrements `height0.consumedLiquidity` by `1` via `_decreaseHeight`. That single-unit decrement then changes the fixed pool’s “implied reserve” on the token1 side by ~ 18.45 token1 even if the output-side split (`amount1FilledByHeight0`) only attributed a much smaller token1 amount to `height0` due to floor rounding. The core AMM, however, updates reserves based on the gross swap amounts, independent of this split logic:

```

if (swapCache.zeroForOne) {
ptrPoolState.reserve0 = _safeIncrementUint128(ptrPoolState.reserve0, reserveIn);
ptrPoolState.reserve1 = _safeDecrementUint128(ptrPoolState.reserve1, swapCache.amountOut);
`ptrPoolState.feeBalance0 = _safeIncrementUint128(ptrPoolState.feeBalance0,

```

M-11 Fixed pool reserve desync from split rounding

CONTINUED

```
poolFeeOfAmountIn);
} else {
ptrPoolState.reserve0 = _safeDecrementUint128(ptrPoolState.reserve0, swapCache.amountOut);
ptrPoolState.reserve1 = _safeIncrementUint128(ptrPoolState.reserve1, reserveIn);
ptrPoolState.feeBalance1 = _safeIncrementUint128(ptrPoolState.feeBalance1, poolFeeOfAmountIn);
}`
```

Once the fixed pool internal state has drifted, the relationship “core reserves $\approx$ internal principal + fixed-price backing for consumed liquidity” stops holding. In the provided POC(`test_replay5` call trace), the post-swap snapshot shows:

`position1ShareOf1 = 2330196404425643948138` and `consumedLiquidity0 = 0` (fixed-pool internal), while `PoolState.reserve1 = 2366810033180055101233` (core AMM). This creates a gap of `36613628754411153095` units of token1 (≈ 36.61 token1) between what the fixed pool state implies should be in `reserve1` and what the AMM is tracking as `reserve1`, and the fuzz harness fails with `FIX_SWAP_MODEL_02: implied reserve1 drift > 100 vs PoolState.reserve1`.

Impact: This reserve desynchronization can strand large, non-dust balances in the AMM reserve bucket that are not represented by LP position accounting, and can lead to incorrect swap/exit behavior (including unexpected reverts or permanently unclaimable funds) for fixed pools at extreme prices.

— RECOMMENDATION

We would not recommend to perform any code changes in regards to this logic at this point, given the high risk of introducing a new bug and given that this issue only becomes relevant at extreme prices. However, a possible solution would be to refactor the fixed-pool swap attribution in `_updateFixedPoolHeights` so the `height0 / height1` split is computed in a way that is provably consistent with fixed-price conversions and with the exact amounts the AMM credits/debits to `PoolState.reserve0/reserve1`.

PROOF OF CONCEPT <https://github.com/GuardianOrg/lbamm-core-fuzz-1763932869200/blob/94b1517e62568df00d0bb33cff224dbbd3901dad/test/fuzzing/FoundryPlayground.sol#L2341-L2460>

— RESOLUTION

Status: Acknowledged.

MEDIUM M-12

ACKNOWLEDGED

Partial-consumed height underpays pairValue

CATEGORY	Rounding
LOCATION	FixedPoolType.sol; FixedHelper.sol
REVIEW	Main Review

DESCRIPTION

The function `_collectPositionSide()` attempts to compute the principal owed to a position by splitting its liquidity into an unconsumed "side token" portion (`sideValue`) and a consumed portion converted into the paired token (`pairValue`). When `startHeight <= currentHeight < endHeight`, it computes the in-range amounts as follows:

```
uint256 liquidity = endHeight - startHeight;
...
sideValue = endHeight - currentHeight;
pairValue = calculateFixedInput(liquidity - sideValue, sqrtPriceX96, sideZero);
if (height.liquidity != height.remainingAtHeight) {
  --sideValue;
}
height.consumedLiquidity -= (liquidity - sideValue);
```

The condition `height.liquidity != height.remainingAtHeight` indicates that the current execution height has partial consumption, and the code deterministically assigns the "ambiguous" unit at `currentHeight` as already consumed by the withdrawing position by decrementing `sideValue` by 1. However, `pairValue` is computed before this decrement, while the pool state update (`height.consumedLiquidity -= (liquidity - sideValue)`) uses the post-decrement value of `sideValue`.

As a result, in the partial-consumption case the function treats the position as having one additional unit of consumed liquidity (because `liquidity - sideValue` increases by 1 after `--sideValue`), but it does not include the paired-token principal corresponding to that additional consumed unit in `pairValue`. This creates a mismatch between (a) the principal amounts returned to the AMM for transfer and (b) the internal accounting of consumed liquidity being removed from the pool state.

Impact: withdrawing LPs can be underpaid on in-range withdrawals when the active height is partially consumed, and the pool's accounting can drift, potentially leaving unaccounted reserves and reducing swap capacity or causing unexpected reverts over time.

M-12 Partial-consumed height underpays pairValue

CONTINUED

— RECOMMENDATION

Ensure `pairValue` is computed from the final (post-adjustment) `sideValue` so that returned principal and `height.consumedLiquidity` updates remain consistent. The simplest approach is to apply the partial-consumption adjustment before computing `pairValue`:

```
sideValue = endHeight - currentHeight;  
if (height.liquidity != height.remainingAtHeight) --sideValue;  
pairValue = calculateFixedInput(liquidity - sideValue, sqrtPriceX96, sideZero);
```

This keeps the consumed amount (`liquidity - sideValue`) consistent across both the returned values and the state mutation.

PROOF OF CONCEPT <https://github.com/GuardianOrg/lbamm-core-fuzz-1763932869200/commit/b829bddfa0256fbb23367fc0cac5ecabfc1530e1>

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-13**

ACKNOWLEDGED

Missing Token Flashloan Validation

CATEGORY Validation
LOCATION LBAMM.sol
REVIEW Main Review

— DESCRIPTION

The code has a defined per token configuration bit

```
/// @dev Token setting flag enabling flash loan operations for the token  
uint16 constant TOKEN_SETTINGS_FLASHLOANS_FLAG = 1 << 7;
```

But the actual flashloan execution path never requires this flag to be set

In `_flashLoan()`, the only gating check is global

```
if (Storage.appStorage().flashLoanBPS > MAX_BPS) revert LBAMM__FlashloansDisabled();
```

Then it calls

```
(address feeToken, uint256 tokenFeeAmount) = _executeTokenFlashloanHooks(flashloanRequest,  
tokenSettings);
```

And `_executeTokenFlashloanHooks()` only uses `TOKEN_SETTINGS_FLASHLOANS_FLAG` to decide whether to call the token hook

If the flag is not set, the hook does not run, but the flashloan continues anyway

This could lead to policy violation issues for some tokens on Limit Break, because the current implementation forces all tokens to be allowed to be flashloaned

If some tokens disallow this and try to enforce their tokens to be used in specific places, all their policies could be bypassed.

— RECOMMENDATION

Add this error and enforce it for that case:

```
error LBAMM__FlashloansDisabledForToken();
```

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-14**

ACKNOWLEDGED

Missing Collect Fees Hook

CATEGORY Logical Error
LOCATION LBAMM.sol
REVIEW Main Review

DESCRIPTION

In the amm `_positionAddLiquidity()` we collect fees returned by the pool type by

```
(
  context.positionId,
  deposit0,
  deposit1,
  fees0,
  fees1
) = ILimitBreakAMMPoolType(
  PoolDecoder.getPoolType(liquidityParams.poolId)
).addLiquidity();
if (fees0 > 0) {
  poolState.feeBalance0 = _safeDecrementUint128(poolState.feeBalance0, fees0);
}
if (fees1 > 0) {
  poolState.feeBalance1 = _safeDecrementUint128(poolState.feeBalance1, fees1);
}
// net flow includes fee payout
_distributeAndCollectLiquidityTokens(
  context.provider,
  context.token0,
  context.token1,
  deposit0.toInt256() - fees0.toInt256() + hookFee0.toInt256(),
  deposit1.toInt256() - fees1.toInt256() + hookFee1.toInt256()
);
```

But the hooks we execute are only the add liquidity hooks:

```
(hookFee0, hookFee1) = _executeAddLiquidityHooks(); // validateAddLiquidity hooks
```

M-14 Missing Collect Fees Hook

CONTINUED

And the system allow collect fees hooks to execute when calling the collect fees. But here in both `AddLiquidity` and `RemoveLiquidity` the collect fees hook path is never invoked, even though fees are being paid out.

Those are not being called

- `TOKEN_SETTINGS_COLLECT_FEES_HOOK_FLAG`
- `ILimitBreakAMMTokenHook.validateCollectFees`
- `ILimitBreakAMMLiquidityHook.validatePositionCollectFees`
- `ILimitBreakAMMPoolHook.validatePoolCollectFees`

If any token/pool/position relies on collect-fees hooks to enforce restrictions or fees on fee withdrawal, an LP can bypass that entire policy by withdrawing fees via `addLiquidity()` instead of calling `collectFees()`.

— RECOMMENDATION

When `fees0 > 0 || fees1 > 0` inside `_positionAddLiquidity`, also run the collect-fees hooks.

Or document it for hook devs that any policies related with fees they enforce in Collectfees only and not in `addLiquidity` and `removeLiquidity` too can be bypassed, so they make sure to enforce them there as well.

— RESOLUTION

Status: Acknowledged.

LOW **L-01**

ACKNOWLEDGED

Executor-set fee-on-top not bound to permit

CATEGORY Unexpected Behavior
LOCATION PermitTransferHandler.sol
REVIEW Main Review

DESCRIPTION

`PermitTransferHandler.ammHandleTransfer` accepts `feeOnTop` from the AMM but the signed `additionalDataHash` /cosignature omit it. The swap hash only covers recipient, amountSpecified/limitAmount, tokenOut, exchangeFee, cosigner, and hook:

```
bytes32 additionalDataHash = EfficientHash.efficientHashTenStep2(
    EfficientHash.efficientHashTenStep1(
        SWAP_TYPEHASH,
        bytes32(uint256(FALSE)),
        bytes32(uint160(swapOrder.recipient)),
        bytes32(uint256(swapOrder.amountSpecified)),
        bytes32(swapOrder.limitAmount),
        bytes32(uint160(swapOrder.tokenOut)),
        bytes32(uint160(exchangeFee.recipient)),
        bytes32(uint256(exchangeFee.BPS))
    ),
    bytes32(uint160(permitData.cosigner)),
    bytes32(uint160(permitData.hook))
);
```

`feeOnTop` is ignored and passed in from the executor/AMM. In the AMM (`_finalizeSwapCollectFundsAndDisburse`), `feeOnTop.amount` is collected and sent to `feeOnTop.recipient` before paying outputs. A relayer with a user's signed permit can replay it and set an arbitrary `feeOnTop` paying themselves; validation still succeeds because `feeOnTop` is not bound to the signature, so the user loses extra tokens if their slippage/limit tolerates the reduced output.

L-01 Executor-set fee-on-top not bound to permit

CONTINUED

Anyone holding the signed permit can submit it; it is not bound to a specific executor. There is no on-chain cap on `fee0nTop.amount` beyond the swap's limit/min-output checks, so a malicious executor can set `fee0nTop` as high as those constraints allow (even most/all of `tokenIn`) and siphon that amount to themselves without the signer's consent. The swap only reverts if post-fee amounts violate the order's limits; otherwise it succeeds with the unauthorized fee.

— RECOMMENDATION

Consider including `fee0nTop.amountMax` in the hashed swap data used for `PermitC` validation (and cosignature).

— RESOLUTION

Status: Acknowledged.

LOW **L-02**

ACKNOWLEDGED

Role server outage bricks paused proxy

CATEGORY Warning
LOCATION SecureProxy.sol
REVIEW Main Review

DESCRIPTION

When the proxy is paused, every call goes through `_checkPauseState`, which first looks up the admin via the external `RoleSetServer` before checking the allowlist:

```
function _checkPauseState() internal {  
    SecurityStorage storage ptrSecurityStorage = _securityStorage();  
    uint256 pauseExpiration = ptrSecurityStorage.pauseExpiration;  
    if (pauseExpiration != UNPAUSED_EXPIRATION) {  
        if (pauseExpiration < block.timestamp) {  
            ptrSecurityStorage.pauseExpiration = UNPAUSED_EXPIRATION;  
            ptrSecurityStorage.currentEscalationTier = TIER_NOT_PAUSED;  
            emit SecurityUnpause();  
        } else {  
            if (  
                msg.sender != _getRoleHolderView(SECURE_PROXY_ADMIN_ROLE) &&  
                !ptrSecurityStorage.allowedCallerDuringPause[msg.sender]  
            ) {  
                revert SecureProxy__Paused();  
            }  
        }  
    }  
}
```

Role resolution uses `TTL=0` (`_setupRole(..., 0)`), so `_getRoleHolderView` performs an external call to `RoleSetServer.getRoleHolder` on every paused call. If `RoleSetServer` reverts or is unreachable (outage, misconfig, upgrade failure), `_getRoleHolderView` reverts before the allowlist check runs. Because `FULL_PAUSE_EXPIRATION` is `type(uint256).max`, there is no time-based escape; admin cannot clear the pause, and allowlisted callers cannot bypass it. The contract stays permanently paused until the external role server is restored out-of-band.

L-02 Role server outage bricks paused proxyCONTINUED

— RECOMMENDATION

Remove the hard dependency on live role lookups during pause by checking the allowlist before external role resolution.

— RESOLUTION

Status: Acknowledged.

LOW **L-03**

ACKNOWLEDGED

SecureProxy doesn't enforce accountability

CATEGORY	Trust Assumptions
LOCATION	SecureProxy.sol
REVIEW	Main Review

— DESCRIPTION

In SecureProxy, pause codes are supposed to be generated by a trusted party (a manager), are pre-distributed to holders, who can apply them. The admin plays an overseer role. I.e. we have the following roles:

- Admin assumption: able to overwrite actions in case of misbehavior
- Manager assumption: pause codes are not known to outside parties
- Holders assumption: act honestly, and don't pause maliciously

Citing from the README: "This enables ... a predictable and auditable pause lifecycle"

The problem is that the contract doesn't enforce the above requirement.

Admin can't revoke manager access to pausing

Suppose the current manager loses admin's trust, and admin wants to revoke their access. What can admin do?

1. Revoke `SECURE_PROXY_CODE_MANAGER_ROLE` role from the manager via the role server, and give it to a new one:
 - This prevents the old manager from issuing new pause codes
 - The new manager can change the current `codeSetId`, but notice that this doesn't prevent the usage of the old pause codes, see `secureAddPauseCodes`: the old codes will expire only after 1 hour.
2. Expire code sets via `secureExpireCodeSets`. This allows the admin to expire only old code sets, not the current one.
3. Unpause the system via `secureAdminPause`: doesn't prevent the old manager from pausing it again, while the codes are valid.

Effectively, if an admin wants to revoke access immediately, they have to go via a multi-step process, which is far from ideal:

L-03 SecureProxy doesn't enforce accountability

CONTINUED

1. Change `SECURE_PROXY_CODE_MANAGER_ROLE` to a new manager (or themselves)
2. Increment the current `codeSetId` using the manager role via `secureAddPauseCodes` with `incrementCodeSet == true`
3. Expire the now non-current, old code set via `secureExpireCodeSets`.

**** SecureProxy doesn't enforce accountability or non-repudiation ****

If we consider pause codes flows:

- Off-chain, they have to be securely transmitted between the manager and the (multiple) holders
- This creates the opportunity for attackers to get access to the codes "in-flight"
- On-chain, anyone can submit a pause code, and thus pause the system, disrupting its operations.

There is no accountability enforced as to where the submitted pause code comes from. Combined with the possibly insecure transmissions, it allows attackers gain illegitimate access to the codes, or any of the parties involved (manager, holders, any of their personnel) to employ the code anonymously.

— RECOMMENDATION

Wrt. the inability to revoke manager's access, we recommend to refine admin-functions such that they are able to revoke pausing access for old pause codes in one go. The simplest as it seems is to remove unnecessary restrictions from `secureExpireCodeSets` and allow admin to expire any code sets.

Wrt. accountability and non-repudiation, we recommend to extend the roles to include the `SECURE_PROXY_CODE_PAUSER_ROLE` (such that multiple participants may have it), grant it to trusted pause code holders, and add `callerHasRole(SECURE_PROXY_CODE_PAUSER_ROLE)` modifier to the `securePause` function, restricting it only to authorized holders.

— RESOLUTION

Status: Acknowledged.

LOW **L-04**

ACKNOWLEDGED

Partial withdraw wipes position on prec. loss

CATEGORY	Unexpected Behavior
LOCATION	FixedHelper.sol, FixedPoolType.sol
REVIEW	Main Review

DESCRIPTION

`withdrawLiquidity` redeposits leftover value after harvesting a position, but it snaps the redeposit to the pool's height precision and never checks that any liquidity remains. If the leftover amount is below the precision step, snapping truncates it to zero and the function proceeds to return the full position and clear all height metadata, even when the requested withdrawal was tiny or zero. Relevant logic:

```
// after collecting full value
uint256 redeposit0 = value0 - liquidityParams.amount0;
uint256 redeposit1 = value1 - liquidityParams.amount1;
_calculateLiquidityStartAndEndHeights(..., redeposit0, redeposit1, ...);
uint256 redeposited0 = liquidityCache.amountAdded0f0To0 + liquidityCache.amountAdded0f0To1;
uint256 redeposited1 = liquidityCache.amountAdded0f1To0 + liquidityCache.amountAdded0f1To1;
withdraw0 = value0 - redeposited0;
withdraw1 = value1 - redeposited1;
...
if (liquidityCache.startHeight0 == liquidityCache.endHeight0) { position.startHeight0 = 0; ...
}
```

When `redeposited0` or `redeposited1` round down to zero, the entire principal is paid out and the position is wiped, causing unintentional full exits and loss of "dust" liquidity. This same issue is prevented on the deposit path by `LiquidityAddInsufficientForPrecision`, which reverts when rounding would remove all added liquidity. The withdrawal path lacks an equivalent guard.

Impact: LPs can lose their remaining liquidity and fee checkpoints even if they intended only a small withdrawal or a fee-only action.

L-04 Partial withdraw wipes position on prec. loss

CONTINUED

Concrete example: spacing = 1,000 units. A position holds 1,001 DAI. The user withdraws 1.1 DAI, intending to leave 999.9 DAI. After snapping to the 1,000 grid, the remainder rounds down to 0, so the redeposit amounts become zero. The function then pays out the full 1,001 DAI and clears start/end heights and fee checkpoints. Therefore, this is a full exit triggered by a “leave dust” withdrawal.

Note: this edge case can occur any time the post-withdraw remainder is smaller than the height spacing, including “fee-only” style calls when the entire position value has fallen below the spacing granularity. Any remainder < spacing will snap to zero and clear the position.

— RECOMMENDATION

After snapping the redeposit amounts, require that at least one side remains non-zero; otherwise revert instead of clearing the position. Alternatively, mirror the deposit guard (`LiquidityAddInsufficientForPrecision`) in the withdrawal path so a partial withdraw cannot silently force a full exit when the leftover value is below the precision grid.

— RESOLUTION

Status: Acknowledged.

LOW **L-05**

ACKNOWLEDGED

Pool rules bypass before hook config

CATEGORY	Unexpected Behavior
LOCATION	AMMStandardHook.sol, AMMModule.sol
REVIEW	Main Review

— DESCRIPTION

Pool creation in the AMM core only invokes token hooks when the token's pool-creation hook flag is set. The core first loads the token's current settings and then conditionally calls `validatePoolCreation`:

```
if (_isFlagSet(tokenSettings.packedSettings, TOKEN_SETTINGS_POOL_CREATION_HOOK_FLAG)) {  
  ILimitBreakAMMTokenHook(tokenSettings.tokenHook).validatePoolCreation(  
    poolId, msg.sender, hookForToken0, details, tokenHookData  
  );  
}
```

New tokens start with default zeroed settings: no hook address and no flags set. While the token is in this unconfigured state, anyone can call `createPool` for that token and choose arbitrary pair token, pool type and fee. Because the pool-creation flag is not yet set, the AMM does not call the token hook and no pool-creation constraints are enforced. In particular, `AMMStandardHook.validatePoolCreation` is never executed for these early pools, so checks such as pool-type whitelist, pair-token whitelist, min/max fee bounds, initial price bounds and initial LP whitelist are completely skipped.

Later, when the token creator configures a hook and enables the pool-creation flag, existing pools created during the unconfigured window are already registered and continue to operate. Pool-creation validation is not re-run on swaps or liquidity operations, so these pools remain grandfathered even if they would violate the current token rules. The creator can pause trading or enforce pricing bounds globally after the fact, but cannot retroactively prevent those pools from existing or having been initialized under disallowed parameters.

Realistic scenario: Circle deploys a USDC variant but hasn't yet set a token hook/flags. An attacker immediately calls `createPool` for USDC/SHIBINU using a dynamic pool type and 200 bps fee. Because the pool-creation flag is unset, token validation is skipped and the pool initializes. Later, Circle configures `AMMStandardHook` to allow only stablecoin pairs, fixed pool type, and 5–30 bps fees with whitelisted LPs. The USDC/SHIBINU pool remains registered and usable; swaps/liquidity on that pool never re-run pool-creation checks, so

L-05 Pool rules bypass before hook config

CONTINUED

USDC trades against a disallowed token, with a disallowed pool type/fee and unapproved LPs, permanently violating the creator's rules.

Impact: an attacker can frontrun token governance, create unauthorized pools such as a low-fee SHIBINU/USDC pool before the hook is configured, and those pools permanently bypass pool-creation-time constraints such as allowed pairs, pool types, fees and initial LP restrictions.

— RECOMMENDATION

Consider adding a `disablePool(poolId)` function callable by token owner (per token's ownership/role checks) or protocol admin. Store a `poolDisabled[poolId]` flag and enforce it on all swap and liquidity entry points. When disabled, allow only withdrawals/fee collection by LPs, no new swaps or adds. This preserves LP exit while shutting down unauthorized pools created before governance was set.

— RESOLUTION

Status: Acknowledged.

LOW **L-06**

ACKNOWLEDGED

Dynamic-Fee Hook May Silently Return 0 poolFee

CATEGORY Validation

LOCATION <https://github.com/GuardianOrg/lbamm-core-team1-1763932865865/blob/remediations/src/modules/AMMModule.sol#L1791>

REVIEW Main Review

DESCRIPTION

In the dynamic-fee hook execution path (`_executePoolFeeHook`), the pool fee is updated only when the hook returns exactly 32 bytes:

```
let success := call(gas(), hook, 0x00, add(hookMemoryPointer, 0x1C), add(0x244,
hookDataCopyLength), 0x00, 0x20)
if iszero(success) {
    returndatacopy(0x00, 0x00, returndatasize())
    revert(0x00, returndatasize())
}
if eq(returndatasize(), 0x20) { poolFeeBPS := mload(0x00) }
```

If the hook returns an unexpected size—empty, short, or malformed—the code simply skips the assignment, leaving `poolFeeBPS` unchanged at its default of 0. As a result, a dynamic-fee pool paired with an EOA or a mis-implemented hook unintentionally becomes a zero-fee pool, eliminating LP and protocol fees.

This differs from stricter patterns (for example, in Uniswap V4), where unexpected return data triggers a revert rather than silently defaulting.

RECOMMENDATION

Require the hook to return exactly 32 bytes and revert otherwise with an error indicating an invalid fee hook response.

RESOLUTION

Status: Acknowledged.

LOW **L-07**

ACKNOWLEDGED

checkAMMExecutionState Return Can Be Wrong

CATEGORY	Unexpected Behavior
LOCATION	https://github.com/GuardianOrg/lbamm-core-team1-1763932865865/blob/d5435e12c4ceb6975468d41694eaf3d7e68525e/src/modules/AMMModule.sol#L3196
REVIEW	Main Review

— DESCRIPTION

The `checkAMMExecutionState` function returns the current state of the AMM by looking at the reentrancy flag.

However during the `executeQueuedHookFeesByHookTransfers` flow the flag is set to `NO_FLAGS`. This could be important to prevent other issues but leads to a wrong return of the `checkAMMExecutionState` if called during that flow (for example by a ERC777 hook).

— RECOMMENDATION

Consider to document this behavior.

— RESOLUTION

Status: Acknowledged.

LOW **L-08**

ACKNOWLEDGED

Misleading Error

CATEGORY	Error
LOCATION	https://github.com/GuardianOrg/lbamm-core-team1-1763932865865/blob/d5435e12c4ceb6975468d41694eaf3d7e68525e/src/modules/AMMModule.sol#L2727-L2729
REVIEW	Main Review

— DESCRIPTION

The `LBAMM__InsufficientInputForFees` error is used three times in the core system. Two times for the input amount which makes sense but also once for the output amount. Here the error is misleading as it talks about a insufficient input amount not a insufficient output amount.

— RECOMMENDATION

Consider to change the name of the error in that case.

— RESOLUTION

Status: Acknowledged.

LOW **L-09**

ACKNOWLEDGED

Fixed pool user loss via silent rounding to 1

CATEGORY Rounding
 LOCATION FixedHelper.sol#L1020-L1032
 REVIEW Main Review

DESCRIPTION

Function `FixedHelper::calculateFixedOutput` performs double `mulDivRoundingUp` to calculate required `amountIn` for the given `amountOut`:

```
function calculateFixedOutput(
  uint256 amountOut,
  uint160 sqrtPriceX96,
  bool zeroForOne
) internal pure returns (uint256 amountIn) {
  if (zeroForOne) {
    amountIn = FullMath.mulDivRoundingUp(amountOut, Q96, sqrtPriceX96);
    amountIn = FullMath.mulDivRoundingUp(amountIn, Q96, sqrtPriceX96);
  } else {
    amountIn = FullMath.mulDivRoundingUp(amountOut, sqrtPriceX96, Q96);
    amountIn = FullMath.mulDivRoundingUp(amountIn, sqrtPriceX96, Q96);
  }
}
```

The problem is that calculations may silently round down to 0, while `mulDivRoundingUp` masks that by adding 1 to it. As a result, the taker may obtain output tokens, while paying 1 unit of the input token, which is silently rounded up from 0 to 1. Example:

```
→ calculateFixedOutput(10**20 * 10**18, 4_295_128_739, false)
└ Decimal: 1
→ calculateFixedOutput(10 * 10**18, 10**18, false)
└ Decimal: 1
```

As examples above demonstrate, this rounding up may happen in a wide range of prices: with the minimal price (`MIN_SQRT_RATIO = 4_295_128_739`) a very large amount (`10**20 * 10**18`) requested from the pool will require 1 unit of input token, while it should be substantially less.

L-09 Fixed pool user loss via silent rounding to 1

CONTINUED

— RECOMMENDATION

As it is not possible to have swaps with < 1 units of input, consider adding a check for the minimal amount of the input token required to be provided, e.g. require that `amountIn >= 100`; that way the rounding errors can be kept to be at most 1%.

— RESOLUTION

Status: Acknowledged.

LOW **L-10**

ACKNOWLEDGED

FixedPool hook may cause a silent overflow

CATEGORY Math
LOCATION FixedHelper.sol#L999
REVIEW Main Review

DESCRIPTION

In `FixedHelper::_calculateOutputLPAndProtocolFee` we have this fragment:

```
lpFeeAmount = FullMath.mulDivRoundingUp(reserveAmountIn, poolFeeBPS, MAX_BPS - poolFeeBPS);
unchecked {
    amountInAfterFees = reserveAmountIn + lpFeeAmount;
}
```

When `poolFeeBPS` is large and close to `MAX_BPS`, `lpFeeAmount` can become much larger than `reserveAmountIn`, multiplying it by up to `10000`. As a result, the `unchecked` block can overflow, returning `amountInAfterFees` much less than `reserveAmountIn`.

As `poolFeeBPS` can be obtained dynamically from the pool hook via `AMMModule::_getPoolFee`, a malicious hook can supply such high value specifically at the right moment, to coordinate with the pool draining transaction.

This is not possible with the current codebase due to the unrelated reverts in other parts of the code.

We reproduce the computations in the following fragment of `FixedHelper::swapByOutput` (see `FixedHelper.sol#L954-L960`):

```
uint256 reserveAmountIn = calculateFixedOutput(amountOut, swapCache.sqrtPriceCurrentX96,
swapCache.zeroForOne);
(
    uint256 swapAmountIn,
    uint256 lpFeeAmount,
    uint256 protocolFeeAmount
) = _calculateOutputLPAndProtocolFee(reserveAmountIn, poolFeeBPS, swapCache.protocolFeeBPS);
```

We take:

L-10 FixedPool hook may cause a silent overflow

CONTINUED

- `amountOut = 307127924032257297266219606769966241547`
- `sqrtPriceCurrentX96 = 408037230205`
- `poolFeeBPS = 9999`
- `protocolFeeBPS = 0`

Calculations:

- `reserveAmountIn = calculateFixedOutput(amountOut, sqrtPriceCurrentX96, true) = 11579208923731619542357098500868790785355651669654478608575570533233926915`
- `swapAmountIn = _calculateOutputLPAndProtocolFee(reserveAmountIn, poolFeeBPS, 0) = 286532030904222046298121324426139510064`

As can be seen:

- The amount to be paid (`swapAmountIn`) is many orders of magnitude smaller than the actual amount (`reserveAmountIn`) that is due for the received `amountOut`
- All of `amountOut` , `swapAmountIn` are less than `type(uint128).max` .

The only conditions that cause reverts are:

- in `FixedHelper.sol#L1080-L1084` , a revert would happen in `_applySwapToLiquidity` when attempting to convert `reserveAmountIn` to `uint128`
- in `AMMModule.sol#L1663-L1665` a revert would happen in `_validateProtocolFees` because `totalFee > amountIn`

— RECOMMENDATION

Despite the fact that the transaction with the overflow would revert, the overflow itself is possible, ignored, and corrupted values propagate to other parts of the codebase.

To exclude the possibility of bugs arising from this ignored overflow we recommend to check for overflows in `FixedHelper::_calculateOutputLPAndProtocolFee` .

— RESOLUTION

Status: Acknowledged.

LOW **L-11**

ACKNOWLEDGED

Wrong slippage parameter specs for direct swaps

CATEGORY Unexpected Behavior
LOCATION DataTypes.sol:L451-L452
REVIEW Main Review

DESCRIPTION

Direct swaps slippage parameters naming and description in the natspec are confusing and don't reflect what happens in the code. The natspec for two parameters describe exactly the same, only in reversed order, see `DataTypes.sol#L451-L452` :

```
/**  
 * @dev **limitSwapAmountOutIn** Minimum output for an input swap or maximum input for an  
 output swap (slippage limit).  
 * @dev **limitSwapAmountInOut** Maximum input for an output swap or minimum output for an  
 input swap (slippage limit).  
 **/
```

In the code `limitSwapAmountOutIn` is treated as the max `amountOut` (what's paid to the order creator), see `AMMModule.sol#L1855-L1857`

```
if (swapCache.amountOut > directSwapParams.limitSwapAmountOutIn) {  
    revert LBAMM__LimitAmountExceeded();  
}
```

while `limitSwapAmountInOut` is treated as the min `amountIn` (`tokenInToExecutor`), see `AMMModule.sol#L1920-L1922` :

```
if (tokenInToExecutor < directSwapParams.limitSwapAmountInOut) {  
    revert LBAMM__LimitAmountExceeded();  
}
```

This wrong naming and descriptions may cause the users of direct swaps to supply wrong slippage parameters, and thus receive unexpected results from swaps (e.g. pay more than expected).

L-11 Wrong slippage parameter specs for direct swapsCONTINUED

— RECOMMENDATION

We recommend to rename the parameters in order to clearly reflect their intended usage (e.g.

`limitSwapAmountOutIn` into `maxAmountOut`, and `limitSwapAmountInOut` into `minAmountIn`), as well as provide the appropriate descriptions.

— RESOLUTION

Status: Acknowledged.

LOW **L-12**

ACKNOWLEDGED

feeGrowthOutside Not Initialized for New Heights

CATEGORY Warning
LOCATION FixedHelper.sol
REVIEW Main Review

DESCRIPTION

The Fixed Pool's fee growth accounting mechanism deviates from Uniswap V3's established pattern for initializing `feeGrowthOutsideX128` when creating new height boundaries. In Uniswap V3, when a tick is initialized at or below the current tick, the protocol sets the tick's `feeGrowth0outside` values equal to the current global fee growth. This ensures that newly created boundaries "absorb" all historical fees, so future `feeGrowthInside` calculations start from the correct baseline.

The Fixed Pool maintains a running `feeGrowthGlobalX128` counter for each token side that increases monotonically as fees accrue. For any height boundary `b`, the pool stores `feeGrowth0outsideX128[b]` to track fees accumulated "outside" that boundary.

In `_addLiquidityToHeight`, when a new height boundary is created (indicated by `flipped = liquidityGrossAfter == 1`), the code does not initialize `feeGrowth0outsideX128` to the current global value:

```
function _addLiquidityToHeight(
    uint256 toHeight,
    mapping (uint256 => FixedHeightInfo) storage heightInfo,
    mapping (uint256 => FixedHeightMap) storage heightMap,
    uint256 informationHeight,
    bool start
) internal {
    FixedHeightInfo storage toHeightInfo = heightInfo[toHeight];
    uint128 liquidityGrossAfter = toHeightInfo.liquidityGross + 1;
    bool flipped = liquidityGrossAfter == 1;
    if (flipped) {
        // Height map linking logic only - no feeGrowth0outside initialization
        FixedHeightMap storage mapToHeight = heightMap[toHeight];
        // ...
    }
```

L-12 feeGrowthOutside Not Initialized for New Heights

CONTINUED

```

toHeightInfo.liquidityGross = liquidityGrossAfter;
toHeightInfo.liquidityNet += start ? int8(1) : int8(-1);
}

```

This produces unexpected intermediate values when querying fee growth. Consider this scenario:

1. Pool has accumulated fees: `feeGrowthGlobal = 100`
2. `currentHeight = 55`
3. User adds liquidity creating new boundaries at `startHeight = 50` and `endHeight = 70`
4. Since `startHeight (50) < currentHeight (55) < endHeight (70)`, `_getFeeGrowthInside` computes:

```

inside = global - outside(start) - outside(end)
inside = 100 - 0 - 0 = 100

```

This reports `feeGrowthInside = 100` (equal to global) when logically, for a newly created position, the "inside" fee growth should be 0 since no fees have been earned yet. With Uniswap's approach, `startHeight = 50` being below `currentHeight = 55` would trigger initialization of `outside(start) = 100`, yielding `inside = 100 - 100 - 0 = 0`.

Currently, this does not result in exploitable behavior because the error cancels out in the fee collection delta calculation. When fees are collected:

```

unchecked {
fee0 = (feeGrowthInside0X128 - position.feeGrowthInside0LastX128) / Q128 +
(feeGrowthInside00f1X128 - position.feeGrowthInside00f1LastX128) / Q128;
fee1 = (feeGrowthInside10f0X128 - position.feeGrowthInside10f0LastX128) / Q128 +
(feeGrowthInside10f1X128 - position.feeGrowthInside10f1LastX128) / Q128;
}

```

At deposit time, the position stores `feeGrowthInsideLast = 100` (incorrectly high). At collection time, the current `feeGrowthInside = 100 + newFees`. The delta becomes `(100 + newFees) - 100 = newFees`, which is correct. Additionally, the AMM's `_safeDecrementUint128` check prevents any underflow-exploited large values from draining funds.

However, this represents a latent risk. The deviation from established patterns creates unexpected intermediate values that could become exploitable with code changes such as: any path that resets `feeGrowthInsideLast` without correcting height `feeGrowth0outside` values; functionality allowing partial fee collection with user-specified amounts; migration or upgrade logic that preserves heights but resets position snapshots; or external protocol integrations that read `feeGrowthInside` directly expecting Uniswap-compatible semantics.

L-12 feeGrowthOutside Not Initialized for New Heights

CONTINUED

— RECOMMENDATION

Consider redesigning `_addLiquidityToHeight` to initialize `feeGrowthOutsideX128` when a height boundary is created at or below the current height, consistent with Uniswap V3's approach.

Otherwise, given the current error-cancellation behavior prevents immediate exploitation, we recommend being cautious with any future code changes that touch fee accounting, position snapshots, or height boundary management. Any modification to these areas should be carefully analyzed to ensure the error-cancellation property is preserved.

— RESOLUTION

Status: Acknowledged.

LOW **L-13**

ACKNOWLEDGED

Rounding Effect Causes Swap Reverts

CATEGORY Rounding
LOCATION FixedHelper.sol
REVIEW Main Review

DESCRIPTION

The `calculateFixedOutput` function uses `FullMath.mulDivRoundingUp` twice to compute the required input for a desired output. While double ceiling is intended to ensure sufficient input is collected from the swapper, it can produce a `reserveAmountIn` that exceeds the pool's `consumedLiquidity`, causing `_decreaseHeight` to revert with `FixedPool__UnderflowCurrentHeight`.

When liquidity is accumulated via `_increaseHeight` (from input swaps), amounts are based on floor-rounded calculations. However, `swapByOutput` computes `reserveAmountIn` using `calculateFixedOutput` with double ceiling, which can return a value 1-2 wei higher than what was actually accumulated. This inflated `reserveAmountIn` is passed to `_applySwapToLiquidity` → `_updateFixedPoolHeights` → `_decreaseHeight`, where:

```
if (consumedLiquidity < amount) {  
    revert FixedPool__UnderflowCurrentHeight();  
}
```

In the POC `test_break1`:

1. Iteration 27 - Input swap adds `consumedLiquidity = 39` to `height0`
2. Iteration 29 - Output swap calls `calculateFixedOutput(39, ...)` which returns `40` due to double ceiling
3. `_decreaseHeight` tries to subtract `40` from `39` → reverts

Output swaps can unexpectedly revert with `FixedPool__UnderflowCurrentHeight` even when the pool has sufficient reserves. This occurs at rounding boundaries with small amounts and certain price ratios.

RECOMMENDATION

L-13 Rounding Effect Causes Swap Reverts

CONTINUED

This is an edge case that causes undesired swap reverts. However, any fix to the rounding logic could introduce other side effects in fee calculations or reserve accounting. The current behavior is fail-safe (reverts rather than corrupting state), but users should be aware that certain output swaps near rounding boundaries may fail unexpectedly.

PROOF OF CONCEPT <https://github.com/GuardianOrg/lbamm-core-fuzz-1763932869200/commit/e66d7e122df57c5342d5a4853acf4198a3491f65#diff-55add19251802f0e16856a973337384ba227d3e51d53e629fc456ed4a6992a95>

— RESOLUTION

Status: Acknowledged.

LOW **L-14**

ACKNOWLEDGED

Tokens Can't Disallow Pairs After Pool Creation

CATEGORY Validation

LOCATION <https://github.com/GuardianOrg/lbamm-hooks-and-handlers-team1-1763932904310/blob/39b3b0e5122f5c75ad082aca52f97eca4212450a/src/hooks/AMMStandardHook.sol#L537-L558>

REVIEW Main Review

— DESCRIPTION

The `pairedTokenWhitelistId` check happens in the `_validateTokenTradingRules` function only for direct swaps. For pools this rule is only enforced during the creation of it.

Therefore it is not possible for tokens to prevent the trading of this pair after a pool was already created.

— RECOMMENDATION

Consider to document this behavior if intended or consider to enforce this rule after pool creation.

— RESOLUTION

Status: Acknowledged.

LOW **L-15**

ACKNOWLEDGED

Fee growth calc can overflow on output swaps

CATEGORY Unexpected Behavior
LOCATION UnsafeMath.sol
REVIEW Main Review

DESCRIPTION

The fee growth accounting in the height walkers (`_increaseHeight` and `_decreaseHeight`) computes a per-height fee growth increment using `UnsafeMath.simpleMulDiv`:

```
uint256 feeGrowthGlobalIncrement = UnsafeMath.simpleMulDiv(
    feeDistributedToHeight,
    Q128,
    heightCache.liquidity
);
```

This performs a “multiply then divide” using a non-512-bit safe path. Because `Q128` is `2**128`, the intermediate multiplication `feeDistributedToHeight * Q128` can exceed `2**256 - 1` if `feeDistributedToHeight` is larger than `type(uint128).max`. In that case, `simpleMulDiv` will wrap the multiplication (or otherwise lose the high bits, depending on implementation), and the computed `feeGrowthGlobalIncrement` becomes incorrect rather than reverting.

While input-based swaps naturally keep LP fee amounts bounded by the input (which is later constrained to `uint128` in `_applySwapToLiquidity`), output-based swaps can produce much larger LP fee values because `_calculateOutputLPAndProtocolFee` computes:

```
lpFeeAmount = FullMath.mulDivRoundingUp(
    reserveAmountIn,
    poolFeeBPS,
    MAX_BPS - poolFeeBPS
);
```

When `poolFeeBPS` is large (close to `MAX_BPS`), `lpFeeAmount` can become significantly larger than `reserveAmountIn` and can exceed `type(uint128).max` even though `reserveAmountIn` itself is constrained to `uint128` by later casts. Once `lpFeeAmount` (and therefore `feeDistributedToHeight`) exceeds `type(uint128).max`, the multiplication by `Q128` inside `simpleMulDiv` can overflow 256-bit arithmetic and silently corrupt fee growth accounting.

L-15 Fee growth calc can overflow on output swaps

CONTINUED

Impact: under certain fee configurations (particularly extreme/unlikely output-swap fee regimes), fee growth can be miscomputed, causing LP fee accrual to be incorrect and potentially enabling over- or under-collection of fees relative to what was actually paid into the pool.

— RECOMMENDATION

Compute fee growth increments with a 512-bit safe `mulDiv` implementation (e.g., `FullMath.mulDiv`) instead of `UnsafeMath.simpleMulDiv`, or enforce bounds that prevent `feeDistributedToHeight * Q128` from overflowing.

A minimal safe change is:

```
uint256 feeGrowthGlobalIncrement =  
FullMath.mulDiv(feeDistributedToHeight, Q128, uint256(heightCache.liquidity));
```

Additionally, enforce fee parameter constraints such that `poolFeeBPS < MAX_BPS` and ensure LP fee amounts cannot exceed a safe bound (e.g., `feeDistributedToHeight <= type(uint128).max`) before applying fee growth updates.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-01**

ACKNOWLEDGED

Brute-forceable pause codes

CATEGORY Warning
LOCATION SecureProxy.sol
REVIEW Main Review

DESCRIPTION

Pause code hashes are emitted publicly when codes are added, yet the contract relies entirely on the secrecy of the underlying strings and never enforces entropy or minimum length. In `secureAddPauseCodes`, `_addCodesToTier` assigns the hash to a tier and emits an event, making the hash public:

```
event PauseCodeAdded(uint256 indexed codeSetId, bytes32 indexed codeHash, uint256 indexed tier);  
...  
function _addCodesToTier(  
    uint256 codeSetId,  
    mapping (bytes32 => uint256) storage ptrCodeTier,  
    uint256 tier,  
    bytes32[] calldata tierCodeHashes  
) internal {  
    bytes32 codeHash;  
    for (uint256 i = 0; i < tierCodeHashes.length; ++i) {  
        codeHash = tierCodeHashes[i];  
        ptrCodeTier[codeHash] = tier;  
        emit PauseCodeAdded(codeSetId, codeHash, tier);  
    }  
}
```

Any account can later call `securePause` with an arbitrary string; the contract hashes it and compares it to the stored value without checking complexity or length:

```
bytes32 pauseCodeHash = keccak256(bytes(pauseCode));  
...  
uint256 codeTier = ptrCodeTier[pauseCodeHash];  
if (codeTier == TIER_INVALID) {  
    revert SecureProxy__CodeInvalid();  
}
```

I-01 Brute-forceable pause codes

CONTINUED

```
...  
ptrCodeTier[pauseCodeHash] = TIER_INVALID;
```

The included tests use single-character codes like "a", "b", "c" for Tier 1/2/3, demonstrating that low-entropy codes are accepted. Because the hash is public in logs and state, an attacker can brute-force weak codes offline, then call `securePause` to trigger or escalate Tier 1→3 pauses, causing a multi-hour/day DoS across all deployments where the same code set is active. Impact: attacker-induced pauses without authorization.

— RECOMMENDATION

Consider enforcing high-entropy codes on-chain (e.g., reject codes below a strong length/entropy threshold).

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-02

ACKNOWLEDGED

Pause codes can be reused in SecureProxy

CATEGORY	Trust Assumptions
LOCATION	SecureProxy.sol
REVIEW	Main Review

DESCRIPTION

In the current design of the `SecureProxy` contract, pause codes are supposed to be generated by a trusted party (a manager), are pre-distributed to pause code holders, and are expected to be single-use. The key assumption from the managers is that they generate the pause codes which are not known to outside parties.

The problem though is that the `SecureProxy` contract itself doesn't enforce this assumption, even when it's reasonably easy to do so. One example is the finding "Brute-forceable pause codes", which describes the situation when pause codes with low entropy are employed. In this finding we focus on another aspect, namely that there is no protection against (accidental) pause code reuse.

Currently, pause codes are registered by their `keccak256` hashes, and the only protection employed is the fact that the pauser (can be anyone) knows the pause code (the preimage of its `keccak256` hash):

```
bytes32 pauseCodeHash = keccak256(bytes(pauseCode));  
mapping (bytes32 => uint256) storage ptrCodeTier = ptrCodeStorage.codeTier;  
uint256 codeTier = ptrCodeTier[pauseCodeHash];  
if (codeTier == TIER_INVALID) {  
    revert SecureProxy__CodeInvalid();  
}
```

I.e. the `pauseCode` can be an absolutely arbitrary string. Even when strong entropy codes are used, this doesn't protect against (accidental) reuse of (strong but known) pause codes. E.g. a manager may employ a code that has already been used somewhere else; or may recycle the code after some time. As pause codes are only hashed, this doesn't provide any guarantees that they are protocol-specific. An attacker, knowing the hashes of all pause codes, may compare them against a database of known preimages with hashes (employed in any protocol), and any database hit will mean for them the ability to pause the protocol with devastating consequences.

Moreover, when a code is used, it is set in the storage to a "clean state":

I-02 Pause codes can be reused in SecureProxy

CONTINUED

```
// Consume code when used  
ptrCodeTier[pauseCodeHash] = TIER_INVALID;
```

This allows a manager to submit the same code at part of this or another code set, as `_addCodesToTier` doesn't enforce any requirements on the submitted codes or the storage state.

— RECOMMENDATION

While not all known preimages can be excluded, at least making the pause codes protocol-specific and non-reusable can be enforced.

To make pause codes protocol-specific we recommend to apply domain separation in the style of EIP-712 (<https://eips.ethereum.org/EIPS/eip-712>), i.e. require the pause code to be a structured data including at least the following:

- protocol name: e.g. `LBAMM`
- version: e.g. `1.0`
- `codeSetId`
- the pause code itself

To enforce pause codes non-reusability, we recommend to store the already employed pause codes (the last component above) in a mapping, and reject attempts to pause with one of the already employed pause codes (irrespective of the `codeSetId`).

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-03

ACKNOWLEDGED

Fee-on-top not prorated on partial fills

CATEGORY Warning
LOCATION AMModule.sol; FeeHelper.sol
REVIEW Main Review

DESCRIPTION

Fee-on-top is defined as a flat, per-swap charge on input swaps. It is computed once up front and stored in `swapCache.feeOnTopAmount`, and if the pool partially fills the swap, the fee-on-top amount is not reduced—the AMM scales exchange/protocol fees and `adjustedAmountSpecified` but leaves the flat fee unchanged. Finalization still transfers the full fee-on-top to the configured recipient. Integrators should be aware that fee-on-top is not prorated on partial fills; consider `minAmountSpecified` and UX accordingly if proportional behavior is desired.

RECOMMENDATION

No protocol change required; document the behavior.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-04**

ACKNOWLEDGED

Wrong error used for sqrtPrice bounds

CATEGORY	Best Practices
LOCATION	FixedPoolType.sol
REVIEW	Main Review

DESCRIPTION

In `FixedPoolType.createPool`, the sqrt price guard uses the wrong revert error. The code checks that `sqrtPriceRatioX96` is within `[MIN_SQRT_RATIO, MAX_SQRT_RATIO]`, but on failure it reverts with `FixedPool__InvalidHeightSpacing`, which is unrelated to price bounds. This misleads integrators and test harnesses: a price-range failure surfaces as a spacing error, obscuring the actual cause and complicating debugging and monitoring.

RECOMMENDATION

Use a price-specific error (e.g., introduce `FixedPool__InvalidPrice()` or reuse an existing price-bound error) when `sqrtPriceRatioX96` falls outside `[MIN_SQRT_RATIO, MAX_SQRT_RATIO]`, so revert reasons reflect the actual guard being enforced.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-05**

ACKNOWLEDGED

Missing Zero Address Checks In Constructors

CATEGORY Best Practices
LOCATION GLOBAL
REVIEW Main Review

— DESCRIPTION

There `LimitBreakAMM` constructor saves the received parameters without validation.

— RECOMMENDATION

Consider to add zero address checks for params passed in to constructors to follow best practices.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-06**

ACKNOWLEDGED

Missing Zero Address Checks In Constructors

CATEGORY Best Practices
LOCATION GLOBAL
REVIEW Main Review

— DESCRIPTION

There `LimitBreakAMM` constructor saves the received parameters without validation.

— RECOMMENDATION

Consider to add zero address checks for params passed in to constructors to follow best practices.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-07**

ACKNOWLEDGED

Unused Code

CATEGORY	Superfluous Code
LOCATION	GLOBAL
REVIEW	Main Review

DESCRIPTION

There is unused code in multiple parts of the codebase.

Unused Imports:

- Core:
 - LimitBreakAMM.sol - "@limitbreak/tm-core-lib/src/licenses/LicenseRef-PolyForm-Strict-1.0.0.sol"
 - AMMModule.sol - "@limitbreak/tm-core-lib/src/licenses/LicenseRef-PolyForm-Strict-1.0.0.sol"
 - ModuleAdmin.sol - "@limitbreak/tm-core-lib/src/licenses/LicenseRef-PolyForm-Strict-1.0.0.sol"
 - ModuleFeeCollection.sol - "@limitbreak/tm-core-lib/src/licenses/LicenseRef-PolyForm-Strict-1.0.0.sol"
 - ModuleLiquidity.sol - "@limitbreak/tm-core-lib/src/licenses/LicenseRef-PolyForm-Strict-1.0.0.sol"
- Fixed Pool:
 - FixedPoolQuoter. - "./Constants.sol"
 - FixedPoolQuoter - "@limitbreak/lb-amm-core/src/DataTypes.sol"
 - FixedPoolQuoter - "@limitbreak/tm-core-lib/src/utis/cryptography/EfficientHash.sol"
 - FixedPoolQuoter - "@limitbreak/tm-core-lib/src/licenses/LicenseRef-PolyForm-Strict-1.0.0.sol"
 - FixedPoolType - "@limitbreak/tm-core-lib/src/licenses/LicenseRef-PolyForm-Strict-1.0.0.sol"
- Hooks and Handlers:
 - PermitTransferHandler - "@limitbreak/tm-core-lib/src/token/erc20/IERC20.sol"
 - PermitTransferHandler - "@limitbreak/tm-core-lib/src/licenses/LicenseRef-PolyForm-Strict-1.0.0.sol"
 - AMMStandardHook - "@limitbreak/tm-core-lib/src/licenses/LicenseRef-PolyForm-Strict-1.0.0.sol"
 - CreatorHookSettingsRegistry - "@limitbreak/tm-core-lib/src/licenses/LicenseRef-PolyForm-Strict-1.0.0.sol"

I-07 Unused Code

CONTINUED

RECOMMENDATION

Consider to remove unused code.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-08**

ACKNOWLEDGED

Redundant temporary vars in `_poolSwapByInput`

CATEGORY Superfluous Code
LOCATION `AMMModule.sol:L1380-L1382`
REVIEW Main Review

DESCRIPTION

In `AMMModule::_poolSwapByInput` (see `AMMModule.sol#L1380-L1382`) temporary variables are created:

```
InternalSwapCache memory tmpSwapCache = swapCache;  
SwapHooksExtraData calldata tmpSwapHooksExtraData = swapHooksExtraData;
```

Then many operations are performed on them. This operation seems to be meaningless, because both of the created temporary vars act as a reference to their counterparts. Thus the temporary vars can be removed, and their usage substituted with their counterpart variables.

RECOMMENDATION

Remove these two lines, and change everywhere `tmpSwapCache` into `swapCache`, and `tmpSwapHooksExtraData` into `swapHooksExtraData`.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-09

ACKNOWLEDGED

Price Bounds isSet Can Not Be Reset

CATEGORY	Validation
LOCATION	https://github.com/GuardianOrg/lbamm-hooks-and-handlers-team1-1763932904310/blob/39b3b0e5122f5c75ad082aca52f97eca4212450a/src/hooks/CreatorHookSettingsRegistry.sol#L430-L446
REVIEW	Main Review

— DESCRIPTION

The `setPricingBounds` function allows to set price bounds to zero later on to deactivate them, however this will not update the `isSet` variable back to `False`. Therefore the hooks will continue to perform unnecessary computations in the `_validatePricingBounds` function.

The same behavior occurs in the `registryUpdatePricingBounds` function.

— RECOMMENDATION

Consider to update the `isSet` variable to `false` if both of the given price bounds equal zero.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-10**

ACKNOWLEDGED

Unused Return Declaration In ammHandleTransfer

CATEGORY	Best Practices
LOCATION	https://github.com/GuardianOrg/lbamm-hooks-and-handlers-team1-1763932904310/blob/39b3b0e5122f5c75ad082aca52f97eca4212450a/src/handlers/permit/PermitTransferHandler.sol#L114
REVIEW	Main Review

DESCRIPTION

The `ammHandleTransfer` declares a return value as `bytes memory` but never returns anything.

RECOMMENDATION

Consider to remove this declaration or return something.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-11

ACKNOWLEDGED

Exact Out Slippage Failure

CATEGORY Logical Error
LOCATION LBAMM.sol
REVIEW Main Review

DESCRIPTION

The slippage mechanism that prevents a user from receiving too few tokens fails during exact output swaps. The code has a bug when checking if the slippage condition is met during a partial fill. It compares the Gross Amount (user + fees) against the Net Minimum (user limit)

Because the gross amount includes fees, it is artificially higher, causing the check to pass even when the user is receiving less than their minimum. This happens because in `_poolSwapByOutput`. Before the swap, fees are added to `swapCache.amountOut`. This turns the user amount (net) into the pool request (gross).

```
// swapCache.amountOut becomes (UserAmount + HookFees)
_applySwapByOutputOutputFees(swapCache, tokenInSettings, tokenOutSettings);
```

The pool returns `actualAmountOut`. This is the gross amount: `(actualAmountOut, ) = ILimitBreakAMMPoolType().swapByOutput();`

And here

```
if (actualAmountOut != originalAmountOut) {
// @Audit, comparing Gross output vs Net minimum
if (actualAmountOut < swapCache.minAmountSpecified) {
revert LBAMM__PartialFillLessThanMinimumSpecified();
}
}
```

`actualAmountOut` includes the fees, while `minAmountSpecified` does not.

RECOMMENDATION

We should verify the minimum amount against what the user actually receives, not what the pool sends out.

I-11 Exact Out Slippage Failure

CONTINUED

— **RESOLUTION**

Status: Acknowledged.

CRITICAL C-01

ACKNOWLEDGED

Zero-Amount Cross Can Underflow Liquidity

CATEGORY DoS
LOCATION FixedPoolType.sol, FixedHelper.sol
REVIEW Remediation Review 1

DESCRIPTION

The fixed pool height update flow can call the upward height walker with an amount of zero, and the implementation still performs a terminal cross. In the swap path, the height splitter computes two amounts and always calls the increase and decrease functions, only reverting if both sides are zero. This makes a zero-amount call to the increase side reachable during normal swaps when rounding pushes all of the output to the other side.

```
function _updateFixedPoolHeights(
    FixedPoolState storage ptrPoolState,
    FixedSwapCache memory swapCache,
    uint256 amount0,
    uint256 amount1
) internal {
    ...
    if (swapCache.zeroForOne) {
        (
            amount0FilledByHeight0,
            amount1FilledByHeight1,
            lpFeeAmountFor0,
            lpFeeAmountFor1
        ) = _splitAmountsAndFeesByHeight(...);
        _decreaseHeight(..., amount0FilledByHeight0, ...);
        _increaseHeight(..., amount1FilledByHeight1, ...);
        ...
    } else {
        (
            amount1FilledByHeight1,
            amount0FilledByHeight0,
            lpFeeAmountFor1,
            lpFeeAmountFor0
        ) = _splitAmountsAndFeesByHeight(...);
```

C-01 Zero-Amount Cross Can Underflow Liquidity

CONTINUED

```

_increaseHeight(..., amount0FilledByHeight0, ...);
_decreaseHeight(..., amount1FilledByHeight1, ...);
...
}
if (amount0FilledByHeight0 == 0 && amount1FilledByHeight1 == 0) {
    revert FixedPool__BothTokenAmountsZeroOnSwap();
}
}

```

The splitter uses rounding to apportion the swap across sides. It is explicitly allowed for one side to be zero as long as the total filled amount is nonzero, so zero on one side is not considered an error. This means `amountOutFilledByOutputHeight` (and therefore the amount passed into the increase side) can be zero while the swap still proceeds.

```

uint256 expectedAmountOutByInputHeight = FullMath.mulDivRoundingUp(
    amountOut,
    swapCache.pairedShare0fExpectedReserve,
    swapCache.expectedReserve
);
amountInFilledByInputHeight = calculateFixedSwapRoundingDown(
    expectedAmountOutByInputHeight,
    swapCache.sqrtPriceCurrentX96,
    !zeroForOne
);
...
if (amountInFilledByInputHeight > 0) {
    uint256 actualAmountOutByInputHeight = ...;
    amountOutFilledByOutputHeight = amountOut - actualAmountOutByInputHeight;
} else {
    amountOutFilledByOutputHeight = amountOut;
}
...
if (totalAmountInFilled == 0) {
    revert FixedPool__ZeroInputSwap();
}

```

Inside the increase path, the main loop only executes when remaining is nonzero, but a final cross is executed unconditionally when the terminal condition is met. If amount is zero, the loop never runs and the final cross still executes.

C-01 Zero-Amount Cross Can Underflow Liquidity

CONTINUED

```
uint256 remaining = amount;
while (remaining != 0) {
  if (heightCache.currentHeight == heightCache.nextHeightAbove && heightCache.remainingAtHeight
  == 0) {
    _crossHeight(heightCache, heightInfo, heightMap, true);
    crossedHeights = true;
  }
  ...
}
if (heightCache.currentHeight == heightCache.nextHeightAbove && heightCache.remainingAtHeight
== 0) {
  _crossHeight(heightCache, heightInfo, heightMap, true);
  crossedHeights = true;
}
```

The tail of the height linked list is self-referential, so a terminal height naturally has `nextHeightAbove` equal to itself. This is established when a new tail is created, and is preserved when the list has only a tail element.

```
} else if (toHeight > informationHeight && informationHeight == informationNextHeightAbove) {
  // Height is new tail height.
  mapInformationHeight.nextHeightAbove = toHeight;
  mapToHeight.nextHeightBelow = informationHeight;
  mapToHeight.nextHeightAbove = toHeight;
  break;
}
```

End heights carry a negative `liquidityNet` because they are the end of a position range. That is encoded directly in the `liquidityNet` update, so it is normal for the net change at the end boundary to be negative.

```
toHeightInfo.liquidityNet += start ? int8(1) : int8(-1);
```

The cross function applies this net change with unchecked signed arithmetic and then casts to `uint128`. When liquidity is zero at the tail and `liquidityNet` is negative, the signed sum becomes negative and the cast wraps to a huge value. Because `crossedHeights` is set, the corrupted values are written back into storage.

```
if (increasing) {
  heightCache.liquidity = uint128(
  int128(heightCache.liquidity) +
```

C-01 Zero-Amount Cross Can Underflow Liquidity

CONTINUED

```
heightInfo[currentHeight].liquidityNet
);
heightCache.nextHeightBelow = currentHeight;
heightCache.nextHeightAbove = heightMap[currentHeight].nextHeightAbove;
heightCache.remainingAtHeight = heightCache.liquidity;
}
```

A realistic execution path is a swap that consumes the last unit of liquidity on one side, leaving the height state at the tail with `remainingAtHeight` equal to zero and liquidity equal to zero, followed by another swap where rounding allocates zero output to that side. In that case, the increase path is called with amount zero, the final cross fires and liquidity underflows to $2^{128} - 1$. This permanently corrupts the height state, fee growth and linked list traversal on that side.

— RECOMMENDATION

Guard the final cross so it only occurs when a nonzero amount actually advanced the height walker, for example by checking amount is nonzero or by tracking whether the loop executed and a boundary was reached. Additionally, harden the signed liquidity update to revert if the signed result is negative before casting to `uint128`. A minimal fix is shown below.

```
if (amount != 0 && heightCache.currentHeight == heightCache.nextHeightAbove &&
heightCache.remainingAtHeight == 0) {
    _crossHeight(heightCache, heightInfo, heightMap, true);
    crossedHeights = true;
}

int128 newLiquidity = int128(heightCache.liquidity) + heightInfo[currentHeight].liquidityNet;
if (newLiquidity < 0) {
    revert FixedPool__UnderflowCurrentHeight();
}
heightCache.liquidity = uint128(newLiquidity);
```

PROOF OF CONCEPT <https://gist.github.com/GuardianAudits/f5a22707c18b93c44f57d2d08b1f6304>

— RESOLUTION

Status: Acknowledged.

HIGH **H-01**

ACKNOWLEDGED

Missing Hook In CLOB

CATEGORY	Validation
LOCATION	https://github.com/GuardianOrg/lbamm-hooks-and-handlers-team1-1763932904310/blob/bf7419d9b80151ca608dd824e7942d17d6e667a8/src/handlers/clob/CLOBTransferHandler.sol#L420-L439
REVIEW	Remediation Review 1

DESCRIPTION

The `CLOBTransferHandler` implements the `validateAddLiquidity` hook in the `openOrder` flow but does not implement the `validateRemoveLiquidity` hook in the `closeOrder` flow.

Therefore transaction could pass that a token does not want to allow to pass and fees could be lost.

RECOMMENDATION

Consider to implement the `validateRemoveLiquidity` hook in the `closeOrder` flow.

RESOLUTION

Status: Acknowledged.

HIGH **H-02**

ACKNOWLEDGED

increaseHeight Leaves Zero Remaining Mid-Range

CATEGORY	Logical Error
LOCATION	FixedPoolType.sol, FixedHelper.sol, FixedPoolQuoter.sol
REVIEW	Remediation Review 1

DESCRIPTION

The height walker can finish a step with `remainingAtHeight` set to zero while `currentHeight` is still below `nextHeightAbove`. This happens in the exact-division branch of `_increaseHeight` when the swap output consumes more than the remaining liquidity at the current height and the post-division remainder is zero. In that path, the code sets `remainingAtHeight` to zero and only advances `currentHeight` by `heightToMove`, leaving a non-normalized state that is neither fully inside a height nor at a boundary.

```
if (remaining > heightRemainingLiquidity) {
    remaining -= heightRemainingLiquidity;
    uint256 heightToMove = remaining / heightCache.liquidity;
    remaining -= heightToMove * heightCache.liquidity;
    if (remaining == 0) {
        heightCache.remainingAtHeight = 0;
        heightCache.currentHeight += heightToMove;
    } else {
        heightCache.remainingAtHeight = heightCache.liquidity - uint128(remaining);
        heightCache.currentHeight += (heightToMove + 1);
    }
}
```

Other branches normalize a zero `remainingAtHeight` by advancing the height and refilling `remainingAtHeight` unless the cursor is exactly at a boundary, which implies the canonical invariant is that zero `remainingAtHeight` only occurs at a boundary height. Impact is protocol-side accounting drift in fixed pools when a swap lands exactly on a height boundary. The state ends up with `liquidity > 0` and `remainingAtHeight == 0` while `currentHeight < nextHeightAbove`, which effectively means this height is fully consumed but the cursor did not advance. Subsequent swaps treat the pool as one height step further along, so traders can receive slightly too much output or pay slightly too little input on the next swap at that boundary. LP value and fee accounting can be off by one unit for positions

H-02 increaseHeight Leaves Zero Remaining Mid-Range

CONTINUED

spanning `currentHeight`, because position valuation and quoting subtract one when `liquidity != remainingAtHeight`, underpaying by one unit at that height.

```
if (height.liquidity != height.remainingAtHeight) {  
  --sideValue;  
}
```

These off-by-one effects can accumulate over repeated boundary hits and manifest as small but real reserve or fee mismatches or pricing drift.

— RECOMMENDATION

Normalize the `remaining == 0` branch in `_increaseHeight` so that finishing a height advances `currentHeight` by one more step and sets `remainingAtHeight` to full `liquidity` unless `currentHeight` equals `nextHeightAbove`, or refactor to reuse the same normalization used in the other branch when `remainingAtHeight` reaches zero.

PROOF OF CONCEPT <https://github.com/GuardianOrg/lbamm-core-fuzz-1763932869200/blob/5fd395cf415c96729f8a7cc942b56daef46d926a/test/fuzzing/FoundryPlayground.sol#L426>

— RESOLUTION

Status: Acknowledged.

HIGH **H-03**

ACKNOWLEDGED

Split Rounding Shifts Excess Output Causing DoS

CATEGORY	DoS
LOCATION	FixedHelper.sol
REVIEW	Remediation Review 1

DESCRIPTION

The `splitAmountsAndFeesByHeight` function in `FixedHelper.sol` contains a rounding error vulnerability that can assign more output amount to `increaseHeight` than the output height's liquidity structure can accommodate. This causes the height traversal to exhaust all available liquidity while still having remaining amount to consume, resulting in an infinite loop at the tail height that triggers an underflow and reverts the transaction.

The `splitAmountsAndFeesByHeight` function splits swap output between two height structures:

1. Input height (via `decreaseHeight`) - handles the "paired share" portion
2. Output height (via `_increaseHeight`) - handles the remainder

```
// Step 1: Calculate expected output from input height (proportional)
expectedAmountOutByInputHeight = mulDivRoundingUp(amountOut, pairedShare, expectedReserve);
// Step 2: Convert to liquidity units using price (ROUNDS DOWN)
amountInFilledByInputHeight = calculateFixedSwapRoundingDown(expectedAmountOutByInputHeight,
sqrtPrice, !zeroForOne);
// Step 3: Calculate actual output filled by input height (ROUNDS DOWN again)
actualAmountOutByInputHeight = calculateFixedSwapRoundingDown(consumed, sqrtPrice,
zeroForOne)
- calculateFixedSwapRoundingDown(consumed - amountInFilled, sqrtPrice, zeroForOne);
// Step 4: Remainder goes to output height
amountOutFilledByOutputHeight = amountOut - actualAmountOutByInputHeight; // ← PROBLEM
```

Steps 2 and 3 use `RoundingDown`, which means `actualAmountOutByInputHeight` is less than `expectedAmountOutByInputHeight`. The difference gets added to `amountOutFilledByOutputHeight`, potentially exceeding the output height's actual capacity.

POC steps:

H-03 Split Rounding Shifts Excess Output Causing DoS

CONTINUED

1. `_splitAmountsAndFeesByHeight` returns `amountOutFilledByOutputHeight = 8.245e21`
2. `_increaseHeight` is called with this amount
3. The output height structure only has capacity for `8.226e21` units
4. After traversing all positions, liquidity = 0 but remaining = `1.84e19`
5. The loop gets stuck at the tail height (self-referencing `nextHeightAbove`)
6. Each iteration attempts to cross the same height: $0 + (-1) = -1 \rightarrow$ Underflow/Panic

This creates a DoS vector where swaps near full reserve utilization will revert, blocking legitimate large trades even when reserves technically exist.

RECOMMENDATION

1. Simple revert when encountering this issue:

```
if (amountOutFilledByOutputHeight > outputHeightCapacity) {  
    revert FixedPool__InsufficientOutputLiquidity();  
}
```

Prevents the infinite loop/underflow issue, but large swaps near capacity will revert.

1. Alternatively, cap output height by capacity (complex):

Step 1: Detect and Cap Output Height Portion

```
uint256 outputHeightCapacity = zeroForOne  
? ptrPoolState.position1ShareOf1  
: ptrPoolState.position0ShareOf0;  
uint256 shortfall = 0;  
if (amountOutFilledByOutputHeight > outputHeightCapacity) {  
    shortfall = amountOutFilledByOutputHeight - outputHeightCapacity;  
    amountOutFilledByOutputHeight = outputHeightCapacity;  
}
```

Step 2: Reallocate Shortfall to Input Height (if capacity available)

```
if (shortfall > 0) {  
    uint256 inputHeightRemaining = consumedLiquidityInputHeight - amountInFilledByInputHeight;  
    uint256 additionalFromInput = min(shortfall, inputHeightRemaining);  
    if (additionalFromInput > 0) {  
        amountInFilledByInputHeight += additionalFromInput;  
        shortfall -= additionalFromInput;  
    }  
}
```

H-03 Split Rounding Shifts Excess Output Causing DoS

CONTINUED

Step 3: Reduce Total Output if Shortfall Remains

```
uint256 actualAmountOut = amountOut;
if (shortfall > 0) {
    actualAmountOut = amountOut - shortfall;
    swapCache.amountOut = actualAmountOut; // Propagate to caller!
}
```

Step 4: Handle Fee Implications

```
// For swapByInput: excess input becomes fee
if (actualAmountOut < amountOut && swapCache.swapByInput) {
    uint256 requiredInput = calculateFixedSwap(actualAmountOut, sqrtPriceX96, !zeroForOne);
    uint256 excessInput = amountInAfterFees - requiredInput;
    (swapCache.lpFeeAmount, swapCache.protocolFee) = _calculateExcessLPAndProtocolFee(
        excessInput, swapCache.protocolFeeBPS, swapCache.lpFeeAmount, swapCache.protocolFee
    );
}

// For swapByOutput: recalculate amountIn and fees
if (actualAmountOut < amountOut && !swapCache.swapByInput) {
    uint256 newReserveIn = calculateFixedSwap(actualAmountOut, sqrtPriceX96, !zeroForOne);
    (swapCache.amountIn, swapCache.lpFeeAmount, swapCache.protocolFee) =
        _calculateOutputLPAndProtocolFee(newReserveIn, poolFeeBPS, protocolFeeBPS);
}

// Recalculate fee split ratio with updated values
lpFeeAmountForOutputHeight = FullMath.mulDiv(swapCache.lpFeeAmount,
    amountOutFilledByOutputHeight, actualAmountOut);
lpFeeAmountForInputHeight = swapCache.lpFeeAmount - lpFeeAmountForOutputHeight;
```

PROOF OF CONCEPT <https://github.com/GuardianOrg/lbamm-core-fuzz-1763932869200/commit/91a2e9ef620104ce58fb3fc0a95fc857e51dbbaa#diff-55add19251802f0e16856a973337384ba227d3e51d53e629fc456ed4a6992a95>

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-01**

ACKNOWLEDGED

Zero-Amount Orders Can DoS Fills

CATEGORY DoS
LOCATION CLOBTransferHandler.sol, CLOBHelper.sol
REVIEW Remediation Review 1

DESCRIPTION

The `openOrder` path only enforces that `orderAmount` is not below the group minimum. When the minimum order base is zero, the group minimum evaluates to zero and `orderAmount` can be zero, so a zero-amount order is accepted and inserted into the FIFO list without any rejection in the helper. The `closeOrder` path treats `inputAmount` equal to zero as already filled or closed and reverts, which makes a zero-amount order unclosable by its maker.

A group key is a packed configuration of hook address, `minimumOrderBase` and `minimumOrderScale` that defines a distinct order book for a given `tokenIn` and `tokenOut` pair. The minimum order is computed as `minimumOrderBase` multiplied by 10 to the power of `minimumOrderScale`, so `minimumOrderBase` equal to zero implies a zero minimum and permits zero-sized orders. For example, a USDC and WETH group with no hook and a zero minimum can be created as follows.

```
bytes32 groupKey = generateGroupKey(address(0), 0, 6);  
clob.openOrder(USDC, WETH, priceP1, 0, groupKey, 0, HooksExtraData("", "", ""));
```

During `fillOrder`, the logic traverses to the next order and checks whether `orderInputRemaining` is zero while the fill still needs input. If so, it reverts with `InsufficientInputToFill`. A zero-amount order at an active price level can trigger this condition and block any fill that needs to move past that order into higher prices, even when there is available liquidity.

```
(ptrOrderBucket, ptrOrder, orderInputRemaining, currentPrice) = traverseCLOB(...);  
if (orderInputRemaining == 0) {  
    if (fillInputRemaining != 0) {  
        revert CLOBTransferHandler__InsufficientInputToFill();  
    }  
}
```

This allows any user to poison an order book group with a zero minimum by inserting a zero-amount order, causing persistent swap-level denial of service for fills that must cross that boundary and leaving higher-priced liquidity unreachable. As a realistic scenario,

M-01 Zero-Amount Orders Can DoS Fills

CONTINUED

assume USDC and WETH orders at price P1 total 5,000 USDC and there is more liquidity at higher prices. An attacker appends a zero-amount order at price P1. A taker attempting to fill 9,000 USDC will consume the real liquidity at P1, then hit the zero-amount order, triggering `InsufficientInputToFill` and preventing traversal into higher prices. Because the zero-amount order cannot be closed by its maker, the denial of service persists for that order book.

— RECOMMENDATION

Require `orderAmount` to be greater than zero in the `openOrder` entrypoint and defensively enforce the same check in the helper. If group keys are intended to enforce a minimum, disallow `minimumOrderBase` of zero. As an additional hardening measure, consider skipping zero-amount orders during traversal instead of reverting, but the primary fix is to forbid zero-size orders.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-02**

ACKNOWLEDGED

Missing tokenIn != tokenOut Validation

CATEGORY Unexpected Behavior

LOCATION CLOBHelper.sol: L80, LimitBreakAMM.sol: L351

REVIEW Remediation Review 1

DESCRIPTION

The `CLOBTransferHandler.openOrder` function allows creating order books where `tokenIn` and `tokenOut` are the same token. While pool-based swaps (`singleSwap` / `multiSwap`) are protected by pool creation validation (`token0 != token1`), the `directSwap` function in `LimitBreakAMM` has no such check and can be used with the CLOB transfer handler to interact with these malformed order books.

Impact:

1. Attackers can abuse same-token swaps to artificially inflate trading volume metrics. This could be exploited to:

- Game airdrop eligibility systems that reward users based on trading volume
- Inflate protocol TVL/volume statistics for misleading marketing or valuation purposes

1. Self-referential swaps have undefined economic semantics - the price calculation converts `sqrtPriceX96` to determine how much output to give for a given input, but swapping a token for itself at any price ratio is economically meaningless.

2. Token hooks and fee calculations that assume distinct tokens could behave unexpectedly

RECOMMENDATION

Add validation in both contracts:

CLOBTransferHandler.sol:

```
function openOrder(...) external nonReentrant returns (uint256 orderNonce) {
    if (tokenIn == tokenOut) {
        revert CLOBTransferHandler__CannotPairIdenticalTokens();
    }
    // ... rest of function
}
```

LimitBreakAMM.sol (directSwap):

M-02 Missing tokenIn != tokenOut Validation

CONTINUED

```
function directSwap(...) external payable ... {  
  _validateDeadline(swapOrder.deadline);  
  _validateRecipient(swapOrder.recipient);  
  _validateExchangeFee(exchangeFee);  
  _validateFeeOnTop(feeOnTop);  
  if (swapOrder.tokenIn == swapOrder.tokenOut) {  
    revert LBAMM__CannotSwapIdenticalTokens();  
  }  
  // ... rest of function  
}
```

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-03**

ACKNOWLEDGED

CLOB openOrder Reverts With AMM Hook

CATEGORY Unexpected Behavior
LOCATION CLOBTransferHandler.sol, AMMStandardHook.sol
REVIEW Remediation Review 1

DESCRIPTION

CLOB `openOrder` executes token add liquidity hooks when the token has the add liquidity hook flag enabled. The default token hook in this setup is `AMMStandardHook`, and its `validateAddLiquidity` function enforces that the caller is the AMM. When `openOrder` is called, the caller of `validateAddLiquidity` is the CLOB transfer handler, not the AMM, so `AMMStandardHook` reverts with `AMMStandardHook__CallerIsNotAMM`. This causes any CLOB `openOrder` that targets a token configured with `AMMStandardHook` and the add liquidity hook flag to revert even though the order inputs are otherwise valid. As a result, the CLOB cannot be used for those tokens and the order book is unavailable for them.

RECOMMENDATION

If CLOB orders are expected to be compatible with `AMMStandardHook` tokens, allow the CLOB transfer handler to pass the hook authorization check or route the hook validation through the AMM so the caller is the AMM. Alternatively, ensure that tokens intended for CLOB orders use a hook implementation that permits CLOB callers or disable the add liquidity hook flag for those tokens.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-04**

ACKNOWLEDGED

Remove hintSqrtPriceX96 Griefing Attack

CATEGORY	Gas Griefing
LOCATION	https://github.com/GuardianOrg/lbamm-hooks-and-handlers-team1-1763932904310/blob/39b3b0e5122f5c75ad082aca52f97eca4212450a/src/handlers/clob/libraries/CLOBHelper.sol#L90-L142
REVIEW	Remediation Review 1

DESCRIPTION

The `openOrder` flow does not check if the given `hintSqrtPriceX96` exists.

This allows the following griefing attack:

- Bob wants to open a order at a new price and passes in the correct `hintSqrtPriceX96` to do so
- Eve is the only one with liquidity on the given `hintSqrtPriceX96` and front runs the call to remove it
- Now Bob's passed in `hintSqrtPriceX96` does not exist and therefore it's next price above and below equals zero
- Now the loop will probably be a lot longer as it starts to loop from the bottom of the CLOB instead of from a smart hint that should be near the correct position

This could drastically increase Bob's gas costs and even lead to DoS.

RECOMMENDATION

Revert if the given `hintSqrtPriceX96` does not exist.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-05**

ACKNOWLEDGED

Price Validation Fails If beforeSwap Disabled

CATEGORY DoS

LOCATION <https://github.com/GuardianOrg/lbamm-hooks-and-handlers-team1-1763932904310/blob/master/remediations/src/hooks/AMMStandardHook.sol#L749>

REVIEW Remediation Review 1

DESCRIPTION

In `_validatePricingBounds`, direct swap price calculation depends on the amount stored via `_setTstorish` during `beforeSwap`, which is then retrieved in `afterSwap`:

```
if (isBeforeSwap) {
    _setTstorish(DIRECT_SWAP_BEFORE_SWAP_AMOUNT_SLOT, params.amount);
    return;
} else {
    (uint256 amount0, uint256 amount1) = params.inputSwap == zeroForOne ?
    (_getTstorish(DIRECT_SWAP_BEFORE_SWAP_AMOUNT_SLOT), params.amount) :
    (params.amount, _getTstorish(DIRECT_SWAP_BEFORE_SWAP_AMOUNT_SLOT));
    sqrtPriceX96 = SqrtPriceCalculator.computeRatioX96(amount1, amount0);
}
```

Since `_requiredHookFlags = 0`, a token owner can register with only `TOKEN_SETTINGS_AFTER_SWAP_HOOK_FLAG` enabled. If neither token triggers `beforeSwap` on the hook, and if pricing bounds are configured, the direct swap will fail because:

1. `beforeSwap` is never called → `_setTstorish` never executes
2. In `afterSwap`, `_getTstorish` returns 0
3. `computeRatioX96` returns `MAX_SQRT_RATIO` or `MIN_SQRT_RATIO` when one amount is zero
4. The extreme price value fails the bounds check with `InvalidPrice`

This results in complete DOS of direct swap functionality for affected tokens with no clear error indicating the root cause.

RECOMMENDATION

1. Make `beforeSwap` required when `afterSwap` is used:

```
uint32 private constant _requiredHookFlags = TOKEN_SETTINGS_BEFORE_SWAP_HOOK_FLAG;
```

1. Or check if pricing bounds exist at registration time and require both hooks.

M-05 Price Validation Fails If beforeSwap Disabled

CONTINUED

— **RESOLUTION**

Status: Acknowledged.

MEDIUM **M-06**

ACKNOWLEDGED

Token Liquidity Hook Fees Ignored

CATEGORY	Rewards
LOCATION	https://github.com/GuardianOrg/lbamm-hooks-and-handlers-team1-1763932904310/blob/39b3b0e5122f5c75ad082aca52f97eca4212450a/src/handlers/clob/CL0BTransferHandler.sol#L574-L601
REVIEW	Remediation Review 1

DESCRIPTION

The `_enforceTokenLiquidityHooks` function calls the `validateAddLiquidity` hooks of the given tokens but does not capture the returned fee values.

The `AMMStandardHook` implementation may return zero fees when calling `validateAddLiquidity`. However a different implementation may return fees and in that case they are just ignored.

RECOMMENDATION

Consider to enforce the LPs to pay for the returned fees.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-07**

ACKNOWLEDGED

Price Bounds Bypass Via snapPrice

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/lbamm-hooks-and-handlers-team1-1763932904310/blob/master/remediations/src/hooks/AMMStandardHook.sol#L198
REVIEW	Remediation Review 1

— DESCRIPTION

The Dynamic Pool `snapPrice` function allows a malicious LP to set pool prices outside of token creator-configured price bounds. The root cause is that `validateAddLiquidity` in `AMMStandardHook.sol` does not check price bounds - it only enforces LP whitelist restrictions.

Price bounds are validated during pool creation (`validatePoolCreation`) and swaps (`_validatePricingBounds`), but not during liquidity operations. This oversight allows `snapPrice` to set any price without validation.

Attack Flow:

1. Token creator sets price bounds (`minSqrtPriceX96` , `maxSqrtPriceX96`)
2. Pool is created within valid bounds
3. All liquidity is removed (pool has 0 liquidity)
4. Malicious LP calls `addLiquidity` with `snapSqrtPriceX96` outside configured bounds
5. Price is set to attacker's value, bypassing bounds entirely

Impact: Token creators relying on price bounds for protection are undermined. Attackers can manipulate prices beyond intended limits, enabling arbitrage opportunities and breaking trust assumptions for tokens using this security feature.

— RECOMMENDATION

Add price bounds validation in `AMMStandardHook.validateAddLiquidity`.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-08**

ACKNOWLEDGED

Current Height Manipulation Bypasses Protection

CATEGORY Validation
LOCATION FixedHelper.sol
REVIEW Remediation Review 1

DESCRIPTION

After adding the remediation implemented in

```
FixedLiquidityModificationParams.maxStartHeight0/maxStartHeight1
```

and the checks in `depositLiquidity()` / `withdrawLiquidity()`

```
if (liquidityCache.startHeight0 > liquidityParams.maxStartHeight0) revert  
FixedPool__StartHeightExceedsMaximum();  
if (liquidityCache.startHeight1 > liquidityParams.maxStartHeight1) revert  
FixedPool__StartHeightExceedsMaximum();
```

There is still an attack that is triggerable

because we add only an upper bound `maxStartHeight`, That stops the push height upward so LP starts too high of H-05

But it does not stop the symmetric attack

"push the relevant `currentHeightX` down, so the victim liquidity is placed at a very low height band, then backrun to restore heights. the victim liquidity ends up far below the restored height (inactive), earning no fees."

Because the protection is one sided, we check `startHeightX <= maxStartHeightX`,

but do not check any of

```
startHeightX >= minStartHeightX
```

```
currentHeightX within [min, max]
```

So any attacker who can decrease `heightX.currentHeight` or make the computed `startHeightX` smaller will never trip the bound.

In swaps, one side height goes down while the other goes up, so an attacker can pick the swap direction that decreases the side they want to sabotage.

M-08 Current Height Manipulation Bypasses Protection

CONTINUED

If the victim is adding only token0 side liquidity, the attacker uses `zeroForOne=true` to push `currentHeight0` down.

If the victim is adding only token1 side liquidity, the attacker uses `zeroForOne=false` to push `currentHeight1` down.

Example targeting a token0 side add :

1 - Victim sends a liquidity add that will create a `[startHeight0, endHeight0]` range, they set `maxStartHeight0` around the normal region.

2 - Attacker frontruns with a swap that decreases `ptrPoolState.height0.currentHeight` significantly.

3 - Victim executes `_calculateLiquidityStartAndEndHeights()`, which uses the now-lowered `currentHeight0`, so `startHeight0` becomes very low and liquidity is placed into a low band.

4 - Attacker backruns a reverse swap to restore heights near where they were.

5 - The victim `startHeight0, endHeight0` range can end up entirely below the restored `currentHeight0`.

It's now out of the active zone and earns negligible or no fees under normal flow, recreating the monopolize fees by pushing others inactive impact of

H-05 addLiquidity Frontrun via Height Manipulation.

— RECOMMENDATION

Extend params to include

`minStartHeight0, maxStartHeight0`

`minStartHeight1, maxStartHeight1`

and enforce both

```
if (startHeight0 < minStartHeight0 || startHeight0 > maxStartHeight0) revert;
```

```
if (startHeight1 < minStartHeight1 || startHeight1 > maxStartHeight1) revert;
```

Or bound the `currentHeight` rather than derived `startHeight`

User supplies `(expectedCurrentHeight0, maxDeviation0)`

And we enforce `abs(currentHeight0 - expected) <= maxDeviation`.

Same for side1.

M-08 Current Height Manipulation Bypasses Protection

CONTINUED

— **RESOLUTION**

Status: Acknowledged.

MEDIUM **M-09**

ACKNOWLEDGED

Input Swap Split Can Exceed Input

CATEGORY DoS
LOCATION FixedHelper.sol
REVIEW Remediation Review 1

DESCRIPTION

The swap splitting logic in `splitAmountsAndFeesByHeight` can reconstruct a total input amount that is larger than the input passed into the swap. The function first assigns a proportional output share to the input height using rounding up, then converts that output share back into input using rounding down and finally computes the output height remainder and converts that remainder back into input using rounding down. The mix of rounding directions makes the split non additive, so the sum of the reconstructed input portions can exceed the actual swap input even when the swap is otherwise valid.

```
uint256 expectedAmountOutByInputHeight =
FullMath.mulDivRoundingUp(amountOut, pairedShare, expectedReserve);
amountInFilledByInputHeight =
calculateFixedSwapRoundingDown(expectedAmountOutByInputHeight, sqrtPrice, !zeroForOne);
uint256 actualAmountOutByInputHeight =
calculateFixedSwapRoundingDown(consumed, sqrtPrice, zeroForOne) -
calculateFixedSwapRoundingDown(consumed - amountInFilledByInputHeight, sqrtPrice,
zeroForOne);
amountOutFilledByOutputHeight = amountOut - actualAmountOutByInputHeight;
uint256 amountInFilledByOutputHeight =
calculateFixedSwapRoundingDown(consumedOut + amountOutFilledByOutputHeight, sqrtPrice,
!zeroForOne) -
calculateFixedSwapRoundingDown(consumedOut, sqrtPrice, !zeroForOne);
if (amountInFilledByInputHeight + amountInFilledByOutputHeight > amountIn) {
revert FixedPool__InputValidationFailed();
}
```

This issue is a rounding edge that depends on pool state and price and can occur during normal swaps, including swaps invoked by hooks or in composite flows like add liquidity with hook callbacks. The result is an unexpected revert for trades that should succeed, which can deny execution of otherwise valid swaps and cause higher level operations to revert.

M-09 Input Swap Split Can Exceed Input

CONTINUED

RECOMMENDATION

Make the split rounding consistent so the reconstructed input never exceeds the original input. A practical approach is to treat the input height portion as authoritative, compute the remaining input as `amountIn - amountInFilledByInputHeight`, and then derive the output height portion from that remainder, updating `swapCache.amountOut` and fee splits to match the adjusted output. Alternatively, avoid rounding up the proportional output share and use rounding down consistently across the split so the sum of converted inputs cannot exceed `amountIn`. If any adjustment reduces output, propagate the new output and fee amounts so downstream accounting and event emissions remain coherent.

PROOF OF CONCEPT <https://github.com/GuardianOrg/lbamm-core-fuzz-1763932869200/blob/aab005f7355ad6a392f28f211f83883ceeaffc73/test/fuzzing/FoundryPlayground.sol#L118>

RESOLUTION

Status: Acknowledged.

MEDIUM **M-10**

ACKNOWLEDGED

Stale Escalation Tier Blocks New Emergency Pause

CATEGORY DoS
LOCATION SecureProxy.sol
REVIEW Remediation Review 1

DESCRIPTION

The `securePause` function does not reset the escalation tier when a previous pause has expired. The tier is only cleared in `_checkPauseState` via the fallback, not during pause code execution.

After a Tier 1 pause expires naturally, `currentEscalationTier` remains at 1. When a new security incident occurs and a code holder attempts to use a fresh Tier 1 code, it reverts because $1 \neq 0$.

Example scenario:

- t=0: Tier 1 pause issued for incident A (expires t+30min)
- t=30min: Pause expires, incident resolved, no further action needed
- t=45min: New incident B detected
- t=45min: Code holder uses fresh Tier 1 code → reverts

The only workarounds are:

1. Wait for a normal protocol transaction to trigger `_checkPauseState` via fallback
2. Admin multisig calls `secureAdminPause(true)`

Both options introduce delays to what is designed as a rapid cross-chain emergency response system, potentially leaving the protocol vulnerable during a critical incident.

Additionally, after Tier 1 expires with `currentEscalationTier == 1`, a malicious actor with a Tier 2 code can execute it since $1 == 2 - 1$ passes. This creates a ~6 hour pause extension without a legitimate active pause.

RECOMMENDATION

Reset stale escalation state at the start of `securePause`.

M-10 Stale Escalation Tier Blocks New Emergency Pause

CONTINUED

— **RESOLUTION**

Status: Acknowledged.

LOW **L-01**

ACKNOWLEDGED

Unbounded Fill Loop Enables Gas Griefing

CATEGORY	DoS
LOCATION	CL0BHelper.sol, CL0BTransferHandler.sol
REVIEW	Remediation Review 1

DESCRIPTION

The `fillOrder` logic iterates through orders until the requested input is fully consumed and does not impose any hard cap on the number of iterations. The minimum order size is entirely defined by the group key, which packs `minimumOrderBase` and `minimumOrderScale`, and there is no global enforcement of a nontrivial minimum. If a group key is configured with a very small minimum, a malicious user can create a large number of tiny orders across many price buckets in that order book, forcing fills to traverse a long linked list. This can make large swaps against that specific order book run out of gas and revert even when liquidity exists, because the loop must visit each small order to progress.

```
while (fillInputRemaining != 0) {  
  ...  
  (ptrOrderBucket, ptrOrder, orderInputRemaining, currentPrice) = traverseCL0B(...);  
  ...  
}
```

The impact is a swap-level denial of service for users who select that token pair and group key, where fills can become non-executable due to gas limits.

RECOMMENDATION

Enforce a meaningful minimum order size at the protocol level for all group keys or restrict which group keys can be used in production. Consider adding caps on the number of orders or price levels processed per fill, or support partial fills with continuation to avoid unbounded gas growth.

RESOLUTION

Status: Acknowledged.

LOW **L-02**

ACKNOWLEDGED

OrderBucket Prev Pointers Go Stale

CATEGORY	Warning
LOCATION	https://github.com/GuardianOrg/lbamm-hooks-and-handlers-team1-1763932904310/blob/remediations/src/handlers/clob/libraries/CLOBHelper.sol#L46
REVIEW	Remediation Review 1

DESCRIPTION

The order bucket queue keeps `nextOrder` and `previousOrder` mappings plus a tail sentinel stored at `previousOrder[0]`. When the head order is removed by `closeOrder` or by a full fill, `traverseCL0B` advances `currentOrderId` to the next order but does not reset `previousOrder` for the new head to 0. When a bucket becomes empty, `traverseCL0B` clears price list pointers and `inputAmountRemaining` but leaves `previousOrder[0]` pointing at the closed head. Later opens append using `previousOrder[0]`, so the new head can have `previousOrder` set to a closed order and the closed order gains a `nextOrder` link into the live list. The forward `nextOrder` chain still yields FIFO traversal today, but the list is no longer a valid doubly linked list with a clean 0 sentinel and can mislead future logic or tooling that relies on backward traversal or head and tail invariants. A minimal sequence is shown below.

```
openOrder at price P -> order A
closeOrder(A) or fillOrder(A) to completion
openOrder at price P -> order B
previousOrder[B] == A, previousOrder[0] == A, nextOrder[A] == B

openOrder at price P -> order A
openOrder at price P -> order B
closeOrder(A) or fillOrder(A) to completion
currentOrderId == B, previousOrder[B] ==
```

Impact today is low because only `currentOrderId` and `nextOrder` are used for traversal, but the invariant break is a correctness hazard if any new/future logic depends on `previousOrder` or tail sentinel integrity.

RECOMMENDATION

When advancing the head, set `previousOrder[newHead]` to `bytes32(0)` so the new head is anchored to the sentinel. When a bucket becomes empty, clear `previousOrder[bytes32(0)]` and consider zeroing `nextOrder` for the removed head to avoid stray links into future buckets. Keep the forward `nextOrder` chain unchanged to preserve FIFO traversal.

L-02 OrderBucket Prev Pointers Go Stale

CONTINUED

— **RESOLUTION**

Status: Acknowledged.

LOW **L-03**

ACKNOWLEDGED

Zero Amount Deposits And Withdrawals Permitted

CATEGORY Validation
LOCATION CL0BTransferHander.sol
REVIEW Remediation Review 1

DESCRIPTION

The `depositToken` and `withdrawToken` functions in `CL0BTransferHandler` do not validate that the amount parameter is greater than zero. This allows any user to execute meaningless zero-value operations that emit events without changing any state.

RECOMMENDATION

Add zero-amount validation to both functions.

RESOLUTION

Status: Acknowledged.

LOW **L-04**

ACKNOWLEDGED

Unsafe Pattern: Missing Tstorish Reset

CATEGORY Warning

LOCATION <https://github.com/GuardianOrg/lbamm-hooks-and-handlers-team1-1763932904310/blob/remediations/src/hooks/AMMStandardHook.sol#L745>

REVIEW Remediation Review 1

DESCRIPTION

`Tstorish` falls back to `sstore` on chains without EIP-1153. Any value written via `_setTstorish` is then persistent unless explicitly cleared. In `AMMStandardHook._validatePricingBounds`, a direct-swap amount is written in `beforeSwap`, read in `afterSwap`, and never cleared:

```
if (isBeforeSwap) {
  _setTstorish(DIRECT_SWAP_BEFORE_SWAP_AMOUNT_SLOT, params.amount);
  return;
} else {
  (uint256 amount0, uint256 amount1) = params.inputSwap == zeroForOne ?
    (_getTstorish(DIRECT_SWAP_BEFORE_SWAP_AMOUNT_SLOT), params.amount) :
    (params.amount, _getTstorish(DIRECT_SWAP_BEFORE_SWAP_AMOUNT_SLOT));
  sqrtPriceX96 = SqrtPriceCalculator.computeRatioX96(amount1, amount0);
  // Storage slot is never cleared here
}
```

This is an unsafe pattern because transient semantics are assumed, but the fallback storage persists across transactions. A prior direct swap can poison `DIRECT_SWAP_BEFORE_SWAP_AMOUNT_SLOT` with a stale value. Subsequent swaps that only hit `afterSwap` (without `beforeSwap`) read the stale value.

Other parts of the codebase explicitly reset transient values after use, even with a `Tstorish` fallback:

- Queueing hook fee transfers uses `tstorish` for a per-tx queue; the queue length is reset to 0 before execution, so stale entries aren't read when length is zero.
- Authorized operator "transient" storage is explicitly cleared by `afterAuthorizedTransfer` (it writes zero). If an authorizer fails to call the "after" hook, the `sstore` fallback would persist across txs, but that's already called out in the docstring.
- Reentrancy guards set and then reset the `tstorish` slot each call, so they don't depend on auto-clearing.

L-04 Unsafe Pattern: Missing Tstorish Reset

CONTINUED

RECOMMENDATION

Clear the direct-swap slot after reading it in `afterSwap`, matching the explicit reset pattern used elsewhere:

```
} else {  
    uint256 storedAmount = _getTstorish(DIRECT_SWAP_BEFORE_SWAP_AMOUNT_SLOT);  
    _clearTstorish(DIRECT_SWAP_BEFORE_SWAP_AMOUNT_SLOT);  
    ...  
}
```

RESOLUTION

Status: Acknowledged.

LOW **L-05**

ACKNOWLEDGED

Zero-Liquidity Price Manipulation

CATEGORY Warning
LOCATION DynamicHelper.sol
REVIEW Remediation Review 1

DESCRIPTION

`swapByInput` and `swapByOutput` in `DynamicPoolType` execute on pools with zero liquidity, advancing the stored price without exchanging tokens. This matches Uniswap V3 behavior but can result in misleading stored prices.

Impact:

- LPs not using `snapPrice` may deposit at incorrect price ratios
- External systems trusting spot price without checking liquidity may get stale/manipulated values

RECOMMENDATION

Document this behavior explicitly - integrators should check liquidity depth and LPs should use `snapPrice` for inactive pools.

RESOLUTION

Status: Acknowledged.

LOW **L-06**

ACKNOWLEDGED

Token0 Not Restored After Precision Rounding

CATEGORY Rounding
LOCATION FixedHelper.sol
REVIEW Remediation Review 1

DESCRIPTION

When adding liquidity to the fixed pool, in the logic related with enforcing only allowing liquidity to be placed on the grid, the precision

In the token1 side in range branch when `currentHeight1 % precision1 != 0` and `addInRange1 == true`, we do

```
uint256 depth1 = currentHeight1 - liquidityCache.startHeight1;
uint256 consumedLiquidity1 = ptrPoolState.height1.consumedLiquidity;
uint256 depth1ValueOf0 =
    calculateFixedSwapRoundingDown(consumedLiquidity1 + depth1, ptrPoolState.sqrtPriceX96, false)
-
    calculateFixedSwapRoundingDown(consumedLiquidity1, ptrPoolState.sqrtPriceX96, false);
if (originalAdd0 < depth1ValueOf0) revert;
add1 += depth1;
add0 -= depth1ValueOf0; // @audit, token0 is consumed here
liquidityCache.amountAdded0f0To1 = depth1ValueOf0;
```

Later, we round `add1` down to precision

```
uint256 precisionAddLoss1 = add1 % precision1;
if (precisionAddLoss1 != 0) {
    add1 -= precisionAddLoss1; // @audit, can make add1 become 0
}
liquidityCache.endHeight1 = liquidityCache.startHeight1 + add1;
```

Then, if `add1` rounded down to 0 so `startHeight1 == endHeight1`, we do

L-06 Token0 Not Restored After Precision Rounding

CONTINUED

```

if (addInRange1) {
  if (liquidityCache.startHeight1 != liquidityCache.endHeight1) {
    liquidityCache.amountAddedOf1To1 = liquidityCache.endHeight1 - currentHeight1;
  } else {
    liquidityCache.amountAddedOf0To1 = 0; // @audit, we cancel the in range accounting
  }
}

```

And we have the correct undo logic on the token0 side `addInRange0` path

```

} else { // startHeight0 == endHeight0
  add1 += liquidityCache.amountAddedOf1To0; // @audit, restore the token1 we had subtracted
  earlier
  liquidityCache.amountAddedOf1To0 = 0;
}

```

But for token1 side, we do not restore

`add0 += depth1ValueOf0` or `add0 += liquidityCache.amountAddedOf0To1`

we only zero the cache field, so the algorithm forgets that `add0` was reduced

This could lead to losing a whole precision0 chunk on the token0 side

— RECOMMENDATION

We can't just directly undo the token0 subtraction when the token1 side collapses to zero, because in the current layout we compute `endHeight0` before we finish the token1 side rounding / cancellation, so if we restore `add0` after that, it won't get included in the token0 side interval. we could

Add snapshots right before we round `add0`

Then keep our existing token0 rounding block as is

Then in the token1 collapse branch, do a full rollback of the token1 side conversion and recompute token0 side rounding / amounts

This could be the best remediation, because we rollback + recompute only in this collapse case

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-01**

ACKNOWLEDGED

OrderBookFill Event May Report Wrong Nonce

CATEGORY Events
LOCATION CLOBTransferHandler.sol, CLOBHelper.sol
REVIEW Remediation Review 1

— DESCRIPTION

The fill logic advances the current order pointer whenever an order is fully consumed. When the last order in a bucket (or the last price in the book) is consumed, traversal moves to the next bucket or sentinel before the final nonce is recorded. As a result, the reported final order nonce can reflect the next order or zero rather than the nonce of the last filled order. This affects the `OrderBookFill` event emitted after the fill, which can confuse indexers and offchain consumers that rely on the nonce to identify the last filled order. Impact: event data can be inaccurate even though onchain state updates remain correct.

— RECOMMENDATION

Cache the nonce of the order being filled before traversal and return or emit that cached nonce instead of reading it after the pointer moves. Alternatively, track the last filled nonce inside the fill loop and emit that value.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-02

ACKNOWLEDGED

Non-Canonical Token Ordering In Hook Context

CATEGORY	Logical Error
LOCATION	CLOBTransferHandler.sol: L557
REVIEW	Remediation Review 1

DESCRIPTION

In `CLOBTransferHandler.sol`, the `_enforceTokenLiquidityHooks` function passes `tokenIn` as `token0` and `tokenOut` as `token1` when constructing the `LiquidityContext` for token hook calls. This differs from the AMM's canonical ordering convention where `token0 < token1` by address.

```
function _enforceTokenLiquidityHooks(
    bytes32 orderBookKey,
    address tokenIn,
    address tokenOut,
    uint160 sqrtPriceX96,
    uint256 orderAmount,
    bytes calldata tokenInHookData,
    bytes calldata tokenOutHookData
) internal {
    LiquidityContext memory context = LiquidityContext({
        provider: msg.sender,
        token0: tokenIn,
        token1: tokenOut,
        positionId: bytes32(0)
    });
    // ... hook calls with hookForToken0 = true for tokenIn, false for tokenOut
}
```

Consider tokens where USDC (0x1234...) and WETH (0x5678...):

- Canonical ordering: `token0 = USDC`, `token1 = WETH` (since `USDC < WETH`)

If a CLOB order is created with `tokenIn = WETH`, `tokenOut = USDC`:

- CLOB sends: `hookForToken0 = true` for WETH's hook, `context.token0 = WETH`
- Canonical expectation: `hookForToken0 = false` for WETH (it's token1), `context.token0 = USDC`

I-02 Non-Canonical Token Ordering In Hook Context**CONTINUED**

Token hooks that rely on canonical ordering assumptions may behave unexpectedly when called from the CLOB.

— RECOMMENDATION

Document clearly that hooks should not assume `context.token0` == "canonically smaller address".` The token0/token1 fields in LiquidityContext represent positional roles within the calling context, not canonical ordering. Hooks designed for use with multiple handlers (AMM pools, CLOB, etc.) should avoid logic that depends on canonical address ordering.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-03

ACKNOWLEDGED

Unused excessAmountIn Field In FixedSwapCache

CATEGORY	Superfluous Code
LOCATION	https://github.com/GuardianOrg/lbamm-pool-type-fixed-team1-1763932948844/blob/mediations/src/FixedPoolType.sol#L335
REVIEW	Remediation Review 1

DESCRIPTION

The `excessAmountIn` field in the `FixedSwapCache` struct is defined and initialized to 0 on every swap, but `swapCache.excessAmountIn` is never read or written anywhere in the codebase.

In `_splitAmountsAndFeesByHeight`, a local variable with the same name handles the excess amount logic instead, immediately folding excess input into fees without storing it in the struct.

RECOMMENDATION

Remove the `excessAmountIn` field from the `FixedSwapCache` struct and its initialization in both swap functions.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-04

ACKNOWLEDGED

swapByOutput Can Undercharge Input

CATEGORY Rounding
LOCATION FixedHelper.sol
REVIEW Remediation Review 1

DESCRIPTION

In the `swapByOutput`, the code re prices the swap using a value computed with `calculateFixedSwapRoundingDown` which can end up charging less input than is required to pay reserves at the fixed price

`swapByOutput()` computes a conservative reserve input (rounding up)

```
uint256 reserveAmountIn = calculateFixedSwap(amountOut, swapCache.sqrtPriceCurrentX96,  
!swapCache.zeroForOne);
```

`calculateFixedSwap()` uses `mulDivRoundingUp` twice, so it rounds up (for exact output user should pay at least this much to get `amountOut`)

Then `_splitAmountsAndFeesByHeight()` recomputes the required input using rounding down deltas, the filled input is computed via differences of `calculateFixedSwapRoundingDown`

```
uint256 amountInFilledByOutputHeight =  
calculateFixedSwapRoundingDown(consumedLiquidityOutputHeight + amountOutFilledByOutputHeight,  
sqrtPrice, !zeroForOne) -  
calculateFixedSwapRoundingDown(consumedLiquidityOutputHeight, sqrtPrice, !zeroForOne);
```

and the input height portion is also derived using `calculateFixedSwapRoundingDown()`

Therefore `reserveAmountIn` (what reserves need) is computed with rounding up

`totalAmountInFilled` (what the algorithm thinks is actually needed) is computed with rounding down and as a delta of a non linear, double rounded function

The issue here is that for exact output swaps, the code reduces the user required input to that rounding down result

```
// _splitAmountsAndFeesByHeight()  
if (totalAmountInFilled < amountIn) {  
    if (swapCache.swapByInput) {  
    } else {  
        (swapCache.amountIn, swapCache.lpFeeAmount, swapCache.protocolFee) =
```

I-04 swapByOutput Can Undercharge Input

CONTINUED

```
_calculateOutputLPAndProtocolFee(totalAmountInFilled, swapCache.poolFeeBPS,  
swapCache.protocolFeeBPS);  
}  
}
```

In exact output and `totalAmountInFilled < amountIn`

it replaces `amountIn` with `totalAmountInFilled` derived from rounding down

— RECOMMENDATION

If we do not reduce the `amountIn`, then we go back to the issue of a de-sync between Fixed Pool state and AMM core reserves. https://www.notion.so/guardianaudits/M-11-2cc8bda5828c812a9e42daf70aa2511d?v=2b48bda5828c81f9822c000ca982a8c4&source=copy_link

The rounding impact is very limited, sending this as information, and in case you would like to acknowledge this for the contest

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-05

ACKNOWLEDGED

Misleading Error On Invalid Tick

CATEGORY	Error
LOCATION	TickMath.sol
REVIEW	Remediation Review 1

DESCRIPTION

`TickMath.getSqrtPriceAtTick` checks that the absolute tick does not exceed `MAX_TICK`, but the revert uses `DynamicPool__InvalidTickSpacing`. That error name is intended for invalid tick spacing, so the revert reason is misleading when the tick is simply out of range. This does not change execution, but it makes debugging and monitoring harder because an out-of-range tick looks like a spacing configuration issue. The impact is limited to developer experience and error reporting accuracy.

```
if (absTick > uint256(int256(MAX_TICK))) {  
    revert DynamicPool__InvalidTickSpacing();  
}
```

RECOMMENDATION

Revert with `DynamicPool__InvalidTick` or introduce a dedicated out-of-range tick error for this check and update any tests that assert the revert reason.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-06

ACKNOWLEDGED

Tstore Activation EOA-Only Not Enforced

CATEGORY Documentation
LOCATION TStorish.sol
REVIEW Remediation Review 1

DESCRIPTION

The `__activateTstore` function is documented to require a direct externally owned account call, but there is no enforcement in the implementation and the `OnlyDirectCalls` error is never used. The documentation is the function comment that states:

```
/**  
 * @dev External function to activate TSTORE usage. Does not need to be  
 * called if TSTORE is supported from deployment, and only needs to be  
 * called once. Reverts if TSTORE has already been activated or if the  
 * opcode is not available. Note that this must be called directly from  
 * an externally-owned account to avoid potential reentrancy issues.  
 */
```

This allows any contract to activate `tstore` support, which can violate assumptions made by integrators or downstream overrides that expect EOA-only activation. Impact is primarily a documentation and behavior mismatch.

RECOMMENDATION

Decide on the intended behavior and make it explicit. If EOA-only activation is required, add an explicit check and revert with `OnlyDirectCalls`. If contract activation is acceptable, update the documentation and remove the unused error to avoid misleading developers.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-07

ACKNOWLEDGED

Fee Shortage Incorrectly Amplified In Outputs

CATEGORY Math
LOCATION AMModule.sol
REVIEW Remediation Review 1

DESCRIPTION

In `_applySwapByOutputInputFees` which runs during exact output swaps

The issue is in how it tops up the protocol fee when the minimum isn't met

It uses a formula meant for exact input swaps where taking more protocol fee reduces the swap amount and dilutes other fees. But in exact output swaps the code doesn't reduce the swap amount, it does

```
swapAmountIn += protocolFeeFromInput;  
protocolFeeFromHookFees += protocolFeeFromInput;
```

any extra protocol fee you add is not going through the pool swap math like exact input swaps, it is collected on top of the already computed `amountIn`, so it does not reduce or dilute LP fees / protocol LP fees.

In the formula here

```
protocolFeeFromInput = FullMath.mulDivRoundingUp(  
    shortage,  
    DOUBLE_BPS,  
    (DOUBLE_BPS - uint256(poolFeeBPS) * uint256(swapCache.protocolFeeStructure.lpFeeBPS))  
);
```

That denominator `DOUBLE_BPS - poolFeeBPS * lpFeeBPS` only makes sense when the protocol fee is taken out of the amount being swapped, because then increasing protocol fee reduces the swap base, which reduces LP fees, which reduces protocol LP share

That's correct in `_applySwapByInputInputFees` because it does `swapAmountIn -= protocolFeeFromInput` and then recompute expected LP / protocol LP fees, but In the exact output case,

There is no dilution to compensate for, so scaling shortage by that factor is not correct

I-07 Fee Shortage Incorrectly Amplified In Outputs

CONTINUED

The overcharge here grows as $\text{poolFeeBPS} * \text{lpFeeBPS}$ gets large, so a small shortage can turn into a much larger extra charge

— RECOMMENDATION

For exact output swaps, since the extra protocol fee is collected on top and does not affect pool fees already computed, the extra amount needed to satisfy the hop fee minimum should be the shortage

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-01**

RESOLVED

Zero Ratio Component Bricks Swaps At Min

CATEGORY DoS
LOCATION FixedHelper.sol
REVIEW Remediation Review 2

DESCRIPTION

The fixed pool normalizes a sqrt price into a packed ratio by performing two consecutive floor divisions. When the sqrt price is extremely small, the lower half of the ratio can round down to zero. The current minimum sqrt bound allows exactly this zero value, which means a pool can be created with a packed ratio that has a zero component.

```
uint160 constant MIN_SQRT_RATIO = 7_922_816_251;

function normalizePriceToRatio(uint160 sqrtPriceX96) internal pure returns (uint256
packedRatio) {
    uint256 ratio0;
    uint256 ratio1;
    if (sqrtPriceX96 > Q96) {
        ratio1 = RATIO_BASE;
        ratio0 = FullMath.mulDiv(ratio1, Q96, sqrtPriceX96);
        ratio0 = FullMath.mulDiv(ratio0, Q96, sqrtPriceX96);
    } else {
        ratio0 = RATIO_BASE;
        ratio1 = FullMath.mulDiv(ratio0, sqrtPriceX96, Q96);
        ratio1 = FullMath.mulDiv(ratio1, sqrtPriceX96, Q96);
    }
    packedRatio = ratio0 << 128 | ratio1;
}

if (fixedPoolDetails.sqrtPriceRatioX96 < MIN_SQRT_RATIO || fixedPoolDetails.sqrtPriceRatioX96
>= MAX_SQRT_RATIO) {
    revert FixedPool__InvalidSqrtPriceX96();
}
```

When a pool is created at the minimum sqrt price, the packed ratio contains a zero half. The swap math unpacks the ratio and uses one component as the denominator for division. A zero denominator causes a division by zero in the swap math, which reverts and permanently bricks one swap direction for that pool. This also affects any reserve or quote logic that

M-01 Zero Ratio Component Bricks Swaps At Min

CONTINUED

relies on the same ratio. The impact is a pool that can be created in a permanently broken configuration and can be used to trigger consistent reverts for one swap direction.

— RECOMMENDATION

After computing the packed ratio, enforce that both components are non-zero, or tighten the allowed sqrt price range so the normalized ratio never underflows to zero. For example:

```
uint256 packedRatio = FixedHelper.normalizePriceToRatio(fixedPoolDetails.sqrtPriceRatioX96);
uint128 r0 = uint128(packedRatio >> 128);
uint128 r1 = uint128(packedRatio);
require(r0 != 0 && r1 != 0, "FixedPool__InvalidSqrtPriceX96");
```

Alternatively, raise the minimum sqrt bound to the smallest value that yields a non-zero ratio component.

PROOF OF CONCEPT <https://gist.github.com/r0bert-ethack/6b7de01b9ea8a631ab63faeba66d98f0>

— RESOLUTION

Status: Resolved.

LOW **L-01**

ACKNOWLEDGED

Split Rounding Can Overpay Output

CATEGORY Rounding
LOCATION FixedHelper.sol
REVIEW Remediation Review 2

— DESCRIPTION

The swap-by-input path computes a fixed-ratio output up front, then re-computes the output by splitting the swap across the input-height return path and the output-height consume path. Each path has its own rounding, and the total is summed. The implementation then overwrites the original fixed-ratio output with this summed value. That means the actual output can be greater than the floor of the fixed ratio for the same input.

```
if (swapCache.swapByInput) {  
    swapCache.amountOut = amountOut = totalAmountOutFilled;  
}
```

This overpayment is not hypothetical. A minimal reproduction (see POC) using the current helper math shows the following concrete numbers:

```
amountIn = 18  
sqrtPriceX96 = 1.1 * Q96  
packedRatio = normalizePriceToRatio(sqrtPriceX96)  
consumedLiquidityInputHeight = 84  
consumedLiquidityOutputHeight = 40  
position1ShareOf1 = 59  
amountOutInitial = floor(amountIn * ratio) = 21  
amountOutFinal (after split) = 22
```

The split logic produces 22 units of output even though the fixed-ratio floor for the same input is 21. That extra unit comes from independent rounding in the two internal paths and is applied directly to the trader. The impact is a correctness issue where swaps can systematically overpay output relative to the fixed-price formula, resulting in a small value leak from pool reserves and violating the intended fixed-price semantics.

— RECOMMENDATION

L-01 Split Rounding Can Overpay Output

CONTINUED

Clamp the output in swap-by-input to the original fixed-ratio floor amount, or redesign the split math to ensure the sum of path outputs never exceeds the single-ratio floor. If the protocol intends to allow this rounding gain, document this behavior.

PROOF OF CONCEPT <https://gist.github.com/r0bert-ethack/5984d3c07335f37698aacdcc39bffe13>

— RESOLUTION

Status: Acknowledged.

LOW **L-02**

RESOLVED

Exact Output Splits Can Underfill

CATEGORY	Unexpected Behavior
LOCATION	FixedHelper.sol
REVIEW	Remediation Review 2

— DESCRIPTION

The exact-output swap path can revert even when the pool's expected reserve indicates the output should be available. The pool first caps the requested output to the expected reserve, then

`_splitAmountsAndFeesByHeight` reconstructs the output by combining the input-height return path and the output-height consume path. Because the two paths each use their own rounding and the results are summed, the reconstructed output can be slightly less than the requested amount. When that happens, the exact-output path reverts with `FixedPool__OutputValidationFailed`.

```
if (totalAmountOutFilled < amountOut) {  
  revert FixedPool__OutputValidationFailed();  
}
```

The expected reserve check happens earlier and can still pass, so callers see an intermittent revert on boundary states even though reserves appear sufficient. This is a liveness and integrator reliability issue for exact-output swaps.

Concrete reproduction with fixed numbers:

```
sqrtPriceX96 = 2 * Q96  
token0-only liquidity = 100  
token1-only liquidity = 1  
swap token1 -> token0 exact output = 1 (creates paired reserve)  
expectedReserve1 >= 5  
swap token0 -> token1 exact output = 5 (reverts)
```

In the reproduction, the pool state reports enough expected reserve for 5 units, but the exact-output swap still reverts at the output validation check because the split reconstruction underfills the requested output.

— RECOMMENDATION

Consider making the exact-output split conservative and consistent so that the reconstructed output never underfills the requested amount.

L-02 Exact Output Splits Can Underfill

CONTINUED

PROOF OF CONCEPT <https://gist.github.com/r0bert-ethack/4c9224b47b38a0fe16df778e8662a087>

— RESOLUTION

Status: Resolved.

LOW **L-03**

RESOLVED

_crossHeight Guard Checks Wrong Value

CATEGORY	Logical Error
LOCATION	FixedHelper.sol
REVIEW	Remediation Review 2

— DESCRIPTION

`FixedHelper._crossHeight` contains a dead underflow guard that checks `heightCache.liquidity < 0`, which is always false because `liquidity` is an unsigned integer. The intended guard should check the signed `newLiquidity` result before casting it to `uint128`. This is a correctness issue that removes a critical safety check and could allow silent state corruption if a negative liquidity delta is ever reached.

— RECOMMENDATION

Replace the dead check with a guard on `newLiquidity` before casting:

```
int128 newLiquidity = int128(heightCache.liquidity) + heightInfo[currentHeight].liquidityNet;
if (newLiquidity < 0) {
    revert FixedPool__UnderflowLiquidity();
}
heightCache.liquidity = uint128(newLiquidity);
```

— RESOLUTION

Status: Resolved.

LOW **L-04**

RESOLVED

Missing HANDLER_ORDER_VALIDATE Flag In Mask

CATEGORY Configuration

LOCATION lbamm-core-team1-1763932865865/src/Constants.sol

REVIEW Remediation Review 2

DESCRIPTION

`ModuleAdmin.setTokenSettings` validates hook flags by masking `packedSettings` with `TOKEN_SETTINGS_HOOK_FLAGS_MASK` and comparing only those bits to the hook's `requiredFlags` / `supportedFlags`.

`TOKEN_SETTINGS_HANDLER_ORDER_VALIDATE_FLAG` is not included in `TOKEN_SETTINGS_HOOK_FLAGS_MASK`, so a token admin can set that bit even if the hook does not advertise support for `validateHandlerOrder`. The validation step silently ignores the flag, and the settings update succeeds.

Later, the CLOB transfer handler checks `packedSettings` directly and will call `validateHandlerOrder` when this flag is set. If the hook doesn't implement it, the call can revert, effectively breaking order creation for that token.

```
uint16 constant TOKEN_SETTINGS_HOOK_FLAGS_MASK =  
    TOKEN_SETTINGS_BEFORE_SWAP_HOOK_FLAG |  
    TOKEN_SETTINGS_AFTER_SWAP_HOOK_FLAG |  
    TOKEN_SETTINGS_ADD_LIQUIDITY_HOOK_FLAG |  
    TOKEN_SETTINGS_REMOVE_LIQUIDITY_HOOK_FLAG |  
    TOKEN_SETTINGS_COLLECT_FEES_HOOK_FLAG |  
    TOKEN_SETTINGS_POOL_CREATION_HOOK_FLAG |  
    TOKEN_SETTINGS_HOOK_MANAGES_FEES_FLAG |  
    TOKEN_SETTINGS_FLASHLOANS_FLAG |  
    TOKEN_SETTINGS_FLASHLOANS_VALIDATE_FEE_FLAG |  
    // TOKEN_SETTINGS_HANDLER_ORDER_VALIDATE_FLAG; <-- missing
```

RECOMMENDATION

Include `TOKEN_SETTINGS_HANDLER_ORDER_VALIDATE_FLAG` in `TOKEN_SETTINGS_HOOK_FLAGS_MASK`

RESOLUTION

Status: Resolved.

LOW **L-05**

RESOLVED

hopFeeBPS = MAX_BPS Blocks Output Swaps

CATEGORY Unexpected Behavior

LOCATION <https://github.com/GuardianOrg/lbamm-core-team1-1763932865865/blob/remediations/src/modules/AMMModule.sol#L2821>

REVIEW Remediation Review 2

DESCRIPTION

`ModuleAdmin.setTokenFees` allows `hopFeeBPS` to be set up to `MAX_BPS` (10000). In output-swap fee calculations (`_applySwapByOutputInputFees`), the code divides by `(MAX_BPS - inputTokenHopFeeBPS)` when topping up protocol fees:

```
protocolFeeFromInput = FullMath.mulDivRoundingUp(
    shortage,
    MAX_BPS,
    (MAX_BPS - inputTokenHopFeeBPS)
);
```

If `inputTokenHopFeeBPS == MAX_BPS`, the denominator becomes zero and the swap reverts.

RECOMMENDATION

Consider restricting `hopFeeBPS` to `< MAX_BPS`.

RESOLUTION

Status: Resolved.

LOW **L-06**

RESOLVED

Direct Swap Max Out Ignores Hook Fees

CATEGORY Unexpected Behavior
LOCATION AMModule.sol
REVIEW Remediation Review 2

DESCRIPTION

In direct swaps with input-specified orders, the executor pays a gross amount of `tokenOut` equal to `swapAmount`. After token hooks run, output-token hook fees are deducted from the recipient amount, reducing the net `amountOut`. The limit check compares `maxAmountOut` against the net `amountOut`, not the gross amount collected from the executor. When output-token hook fees are non-zero, the executor can therefore pay more than their intended maximum while the transaction still passes the cap check. This is a correctness and UX issue because `maxAmountOut` is documented as the maximum amount of output token to be supplied by the executor, but the current check only caps the post-fee net amount delivered to the recipient. The impact is that a taker can overpay relative to their configured cap whenever output-token hook fees apply.

```
if (swapCache.inputSwap) {  
    swapCache.amountOut = directSwapExecutorInput = directSwapParams.swapAmount;  
    ...  
    _applySwapByInputOutputFees(...);  
}  
if (swapCache.amountOut > directSwapParams.maxAmountOut) {  
    revert LBAMM__LimitAmountExceeded();  
}  
_collectToken(executor, swapOrder.tokenOut, directSwapExecutorInput);
```

RECOMMENDATION

If `maxAmountOut` is intended to cap the executor's gross outflow, compare `directSwapExecutorInput` (or `swapAmount`) against `maxAmountOut`. If the intended cap is on net recipient output, update the naming and documentation to make that explicit and avoid user confusion.

RESOLUTION

Status: Resolved.

LOW **L-07**

RESOLVED

No Direct Way To Disable Hooks

CATEGORY Validation

LOCATION <https://github.com/GuardianOrg/lbamm-core-team1-1763932865865/blob/c956d55c8e9833987716a1c4f11ee214ecebaaa3/src/modules/ModuleAdmin.sol#L274>

REVIEW Remediation Review 2

DESCRIPTION

The `setTokenSettings` function will always call `ILimitBreakAMMTokenHook(tokenHook).hookFlags()` even if the given `tokenHook` is the zero address. Therefore the config update will revert if the caller tries to deactivate the given hook.

If the given hook has required flags than the only solution for the caller would be to deploy a new hook contract without any required flags to be able to disable all of them.

This makes disabling a hook a complicated process which could lead to damage if the hook is malicious and fast action is needed.

RECOMMENDATION

Consider to allow a clear path to disabling hooks.

RESOLUTION

Status: Resolved.

LOW **L-08**

RESOLVED

CLOB Fills Let Executor Skim Maker-Funded Fees

CATEGORY	Unexpected Behavior
LOCATION	CLOBTransferHandler.sol
REVIEW	Remediation Review 2

— DESCRIPTION

The AMM swap entrypoints allow the swap caller to provide an exchange fee (BPS + recipient) and a flat fee-on-top (amount + recipient). In the CLOB fill flow, the transfer handler is responsible for sourcing the input token by consuming maker balances from the on-chain order book and transferring the required input token amount into the AMM during finalization.

However, the CLOB transfer handler does not validate, restrict, or account for the provided fee parameters when filling orders. The handler consumes the full input amount from makers and transfers that full input amount into the AMM, while the AMM still applies the configured fee logic and transfers the fee amounts out of the collected input token balance to the configured fee recipients.

Importantly, in this route the maker does not control the AMM swap's slippage parameter (`limitAmount`) because it is supplied by whoever calls the swap (the executor). The maker's protection is their posted CLOB limit price (`sqrtPriceX96`), which the handler enforces by reverting if the AMM output is insufficient to satisfy the maker-required output at that price for the filled input amount. This protection does not constrain executor-selected fee parameters, and any fee configuration that still leaves enough output to satisfy makers will succeed. As a result, this is not primarily a "maker set min output wrong" scenario, it is a missing on-chain fee policy in a flow where makers fund the input but the executor selects the fee recipients and amounts.

L-08 CLOB Fills Let Executor Skim Maker-Funded Fees

CONTINUED

If an untrusted executor can set fee recipients (for example, setting the fee recipient to themselves) while routing swaps through the CLOB transfer handler, they can extract additional value funded by makers' deposited input token. This can be combined with the existing "excess output refund" behavior of the CLOB handler: when the AMM output exceeds the amount required to satisfy makers at their posted prices, the remainder is refunded to the executor as output token. As a result, an executor can capture input-token fees and still receive any remaining output-token surplus, effectively "double-dipping" on the spread plus fees as long as the fee-reduced AMM output is still sufficient to satisfy the maker payouts.

This creates a hidden economic lever for executors that is not enforced or surfaced by the handler itself. In the worst case, it enables systematic value extraction from maker liquidity (and may lead to unexpected swap reverts when fee settings are too aggressive for the available output to satisfy maker prices).

— RECOMMENDATION

If arbitrary executor-controlled fees are not intended for CLOB fills, explicitly reject nonzero fees in the CLOB transfer handler by requiring the exchange fee BPS to be zero and the fee-on-top amount to be zero for handler-routed swaps.

If fees are intended, introduce an explicit fee policy that is enforceable on-chain for CLOB fills. At minimum, ensure that fee parameters are validated (including recipient allowlisting or authenticated fee commitments) and that the order fill logic accounts for fee-adjusted amounts so maker payouts and input collection remain consistent with the AMM's fee application. Additionally, pass fee parameters into any executor validation hook so the hook can enforce fee constraints for a given order book/group.

PROOF OF CONCEPT <https://gist.github.com/r0bert-ethack/e207ed5485c4f4c83a2803dda6dfe062>

— RESOLUTION

Status: Resolved.

INFORMATIONAL **I-01**

RESOLVED

Unused Error

CATEGORY	Superfluous Code
LOCATION	https://github.com/GuardianOrg/lbamm-pool-type-fixed-team1-1763932948844/blob/b6be985916c615187c584cdfed58dc23eabf7704/src/Errors.sol#L56
REVIEW	Remediation Review 2

DESCRIPTION

The `FixedPool__OutputExceedsCapacity` error is defined but never used.

RECOMMENDATION

Consider to remove it.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-02

ACKNOWLEDGED

Hook Validates Different Fee Than Charged

CATEGORY Validation
LOCATION AMModule
REVIEW Remediation Review 2

DESCRIPTION

```
// _executeTokenFlashloanHooks()  
bool feeAllowed = ILimitBreakAMMTokenHook(feeTokenSettings.tokenHook).validateFlashloanFee(  
msg.sender,  
flashloanRequest.loanToken,  
flashloanRequest.loanAmount,  
feeToken,  
tokenFeeAmount, // @audit, what gets validated  
flashloanRequest.executor,  
flashloanRequest.feeTokenHookData  
);
```

validateFlashloanFee() is not validating the actual fee that will be charged in feeToken

Right after that hook returns, the protocol computes the real fee it will enforce

```
// _flashLoan()  
if (tokenFeeAmount == 0) {  
feeAmount = ceil(loanAmount * flashLoanBPS / MAX_BPS);  
} else {  
feeAmount = tokenFeeAmount + ceil(tokenFeeAmount * flashLoanBPS / MAX_BPS);  
}
```

So the fee token hook is asked to approve `tokenFeeAmount`, but the executor is later required to pay a different `feeAmount`

That makes `feeTokenHookData` based fee token validation meaningfully ineffective for any fee token hook logic that depends on the fee amount to have a cap

The fee token hook doesn't get the correct used fee amount to validate, so any fee token protections based on fee size will validate a different amount

I-02 Hook Validates Different Fee Than Charged

CONTINUED

— RECOMMENDATION

Consider computing the exact fee that will be charged in feeToken before calling validateFlashloanFee

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-01**

ACKNOWLEDGED

Flashloan Cross-Token Fee Can Use Wrong Units

CATEGORY Unexpected Behavior
LOCATION AMModule.sol
REVIEW Remediation Review 3

DESCRIPTION

The flashloan fee calculation supports token hooks that can return a `feeToken` that differs from the `loanToken`. When the hook returns `feeToken != loanToken` but returns `tokenFeeAmount == 0`, the AMM computes `feeAmount` as a fraction of `loanAmount` and then treats that value as an amount denominated in `feeToken`.

```
uint256 feeAmount;  
if (tokenFeeAmount == 0) {  
    feeAmount = FullMath.mulDivRoundingUp(flashloanRequest.loanAmount,  
Storage.appStorage().flashLoanBPS, MAX_BPS);  
} else {  
    feeAmount = tokenFeeAmount + FullMath.mulDivRoundingUp(tokenFeeAmount,  
Storage.appStorage().flashLoanBPS, MAX_BPS);  
}
```

If `feeToken` is not the loan token, this mixes units: `loanAmount` is denominated in `loanToken`, but `feeAmount` is later enforced and stored as if it were denominated in `feeToken`. A hook can therefore create nonsensical or manipulable fee outcomes (for example, making flashloans of a valuable token effectively cheap in value terms by charging the BPS-based fee in a different, low-value token when `tokenFeeAmount == 0`).

RECOMMENDATION

Enforce a consistent invariant between `feeToken` and the fee amount source. For example, if `feeToken != loanToken` then require the hook to return an explicit `tokenFeeAmount > 0` (a fee denominated in `feeToken`) and compute fees exclusively from that value. If `tokenFeeAmount == 0`, force `feeToken = loanToken` and charge the default BPS fee in the loan token. Add regression tests that cover the `feeToken != loanToken` and `tokenFeeAmount == 0` combination.

M-01 Flashloan Cross-Token Fee Can Use Wrong Units

CONTINUED

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-02**

ACKNOWLEDGED

Hook Pricing Breaks On Partial Fills

CATEGORY	Logical Error
LOCATION	SingleProviderPoolType.sol:302–329, SingleProviderHelper.sol:42–51
REVIEW	Remediation Review 3

DESCRIPTION

The hook is queried once with the originally requested amount, and the returned price is used for the entire swap -- even when the fill is capped to available reserves:

```
// SingleProviderPoolType.swapByInput
ISingleProviderPoolHook.HookPoolPriceParams memory priceParams;
priceParams.inputSwap = true;
priceParams.poolId = poolId;
priceParams.amount = amountIn; // full requested amount sent to hook
pools[poolId].lastSqrtPriceX96 =
swapCache.sqrtPriceCurrentX96 =
ISingleProviderPoolHook(swapCache.poolHook).getPoolPriceForSwap(
context,
priceParams, // hook sees full amount
swapExtraData
);
```

Then in `SingleProviderHelper.swapByInput`, if the computed output exceeds reserves, the swap is capped to a partial fill without re-querying the hook:

```
uint256 amountOut = swapCache.amountOut = calculateFixedInput(amountInAfterFees, sqrtPriceX96,
zeroForOne);
if (amountOut > swapCache.reserveOut) {
swapCache.amountOut = swapCache.reserveOut;
uint256 initialAmountIn = swapCache.amountIn;
swapByOutput(swapCache, poolFeeBPS);
}
```

If the hook's pricing is amount-sensitive (e.g., a market-maker hook that offers better rates for larger orders, or an oracle hook with price-impact logic), a trader can exploit this:

M-02 Hook Pricing Breaks On Partial Fills

CONTINUED

1. Request a huge amountIn swap (e.g., 1,000,000 tokens)
2. Hook returns a favorable price intended for that large size
3. The helper discovers output exceeds reserves, caps to a partial fill
4. The partial fill executes at the large-order price, not the price appropriate for the actual fill size

This is unique to the single-provider pool type because fixed/dynamic pricing doesn't depend on an external amount-sensitive hook.

— RECOMMENDATION

Either:

1. Re-query the hook with the capped amount when a partial fill occurs
2. Forbid partial fills for this pool type (revert when amountOut > reserveOut)
3. Enforce amount-independent pricing in the hook interface and document this constraint clearly

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-03**

RESOLVED

Floor Math Arbitrage Extracts Reserves

CATEGORY Rounding
LOCATION FixedHelper.sol
REVIEW Remediation Review 3

DESCRIPTION

Adding in range liquidity can increase height.consumedLiquidity because we pre consume depth below currentHeight

```
//_addLiquidity()  
if (startHeight < currentHeight) {  
    height.consumedLiquidity += (currentHeight - startHeight);  
}
```

The paired token amount required to support that pre consumed depth is computed using floor math, difference of floors

```
// _calculateLiquidityStartAndEndHeights() for addInRange0  
depth0ValueOf1 =  
    calculateFixedSwapByRatioRoundingDown(consumedLiquidity0 + depth0, packedRatio, true) -  
    calculateFixedSwapByRatioRoundingDown(consumedLiquidity0, packedRatio, true);
```

Because the conversion uses floor rounding math, sometimes adding +1 unit of depth doesn't increase the output at all (it rounds down to the same integer), so the extra value is 0

When a position is later withdrawn, the paired value for the consumed part is again computed as a difference of floors, but the vulnerability here is that

It uses the current global height.consumedLiquidity and then subtracts from it

```
//_collectPositionSide()  
uint256 consumedLiquidity = height.consumedLiquidity;  
pairValue =  
    floor(consumedLiquidity) -  
    floor(consumedLiquidity - consumedDelta);  
height.consumedLiquidity -= (liquidity - sideValue);
```

M-03 Floor Math Arbitrage Extracts Reserves

CONTINUED

This makes global consumedLiquidity behave like a LIFO stack the withdrawing position claims the top most marginal segment of the global floor function, not necessarily the marginal segment that corresponded to its own historical pre consumption.

And because the conversion is $\text{floor} (x * \text{numerator} / \text{denominator})$, adding 1 more unit of x doesn't always increase the output by the same amount, sometimes it increases by 0, sometimes by 1 or more, depending on where x sits relative to the denominator (the remainder $x \% \text{denominator}$)

That lets an attacker

Add in range at a global consumedLiquidity where the marginal conversion for +depth is 0 ,they pay 0 paired token for that pre consumed depth

Then wait until swaps move global consumedLiquidity to a point where the marginal conversion for the top segment is 1

Withdraw their position and receive 1 paired token for the same consumedDelta, because collectPositionSide computes the delta using the current global consumedLiquidity and subtracts from the top

This repeatable rounding arbitrage that can keep extracting pool reserves

example, picking a ratio where rounding matters

ratio0 = 3, ratio1 = 2, so token0 -> token1 = $\text{floor} (x * 2/3)$

Let depth = 1 (happens whenever $\text{currentHeight} \% \text{precision} \neq 0$ and $\text{addInRange0} = \text{true}$)

Assume before attacker adds liquidity

$\text{height0.consumedLiquidity} = 3$

Then the in range depth cost in token1 is

```
floor ( (3 + 1) * 2 / 3 ) - floor ( 3 * 2 / 3 )
```

```
floor ( 8 / 3 ) - floor ( 6 / 3 )
```

```
2 - 2 = 0
```

So attacker can add an in range position that increases consumedLiquidity by 1, but pays 0 token1 for the consumed portion

Now let swaps push global height0.consumedLiquidity to 5

When attacker withdraws and the position consumedDelta includes that same 1 unit, collectPositionSide computes

M-03 Floor Math Arbitrage Extracts Reserves

CONTINUED

 $\text{floor} (5 * 2/3) - \text{floor} (4 * 2/3)$ $\text{floor} (10/3) - \text{floor} (8/3)$ $3 - 2 = 1$

Attacker receives 1 token1 for that consumedDelta = 1, even though they paid 0 token1 for that pre consumed unit

That 1 token1 comes out of pool value, it's not newly minted, it's misattributed due to floor boundaries plus the LIFO subtraction

This can be repeated to harvest rounding increments whenever

addInRange paths are used (startHeight < currentHeight occurs),

the pool global consumedLiquidity moves across favorable boundaries

This lets an exploiter extract from the pool reserves by timing add / withdraw around discrete rounding boundaries

— RECOMMENDATION

Not recorded.

PROOF OF CONCEPT <https://gist.github.com/GuardianAudits/42402a849c5244c304dd41f06efc5e9b>

— RESOLUTION

Status: Resolved.

MEDIUM **M-04**

RESOLVED

Pool Has Reserves And swapByOutput Reverts

CATEGORY DoS
LOCATION FixedHelper.sol
REVIEW Remediation Review 3

DESCRIPTION

`swapByOutput` can revert even when the pool has enough reserves

In `splitAmountsAndFeesByHeight()` inside the swap by output branch

```
if (totalAmountInFilled > amountIn) {  
    // Allow a maximum of 1 unit of input over the initial amountIn for split rounding  
    if (totalAmountInFilled > amountIn + 1) {  
        revert FixedPool__InputValidationFailed();  
    }  
}
```

This assumes that the internal splitting math can only ever require at most +1 extra input unit beyond the initial `amountIn` computed from the fixed ratio

But that assumption is not always true, there are valid pool states where

the pool has enough expected reserves to satisfy the requested `amountOut`, and

`reserveAmountIn = calculateFixedSwapByRatio(amountOut, packedRatio, !zeroForOne)` is correct for the fixed price math,

but `_splitAmountsAndFeesByHeight()` ends up with `totalAmountInFilled > amountIn + 1` and the swap everts with `FixedPool__InputValidationFailed()`

The `_splitAmountsAndFeesByHeight()` tries to split the swap across two virtual reserve sources

Input height return path and output height consumption path

for `swapByOutput`, it must ensure the requested `amountOut` is fully filled, The function

starts with a proportional split of `amountIn` between the two sources (based on `inputShareOfExpectedReserve / expectedReserve`)

M-04 Pool Has Reserves And swapByOutput Reverts

CONTINUED

then, if output is underfilled, it forces the remainder to be produced by the output height, and recomputes how much input that forced output height consumption actually requires , `actualAmountInFromOutputHeight`

The key nonlinearity here

The mapping `consumedLiquidityOutputHeight -> outputHeightInputShare`

uses floor math `calculateFixedSwapByRatioRoundingDown` , so it moves in jumps at share boundaries.

So when we force output height to fill the remaining output, `outputHeightInputShare` can jump by more than one unit, the required input can increase by 2+ units at once

The algorithm tries to offset that by reducing the input height allocation, but it can only reduce it by amounts that do not change the input height output (`returnableInput`) or by already known unfilled input (`unfilledInput`)

In many states, that slack is 0

when the jump is bigger than the available slack, we get

`totalAmountInFilled = amountInFilledByInputHeight + actualAmountInFromOutputHeight`

and it can exceed `amountIn + 1` by more than 1, triggering the revert

But crucially, a different split can exist that works, like pushing more through output height, less through input height, but the function doesn't search for it, it just asserts the jump can't be > 1 and revert

<https://gist.github.com/GuardianAudits/7ea11404a030fc15e0843ada317ef0fe>

— RECOMMENDATION

we need to make `_splitAmountsAndFeesByHeight()` search for a feasible split in swap by output

— RESOLUTION

Status: Resolved.

MEDIUM **M-05**

ACKNOWLEDGED

Unbacked Output Becomes Unfunded Dust

CATEGORY Logical Error
LOCATION FixedHelper.sol
REVIEW Remediation Review 3

DESCRIPTION

In the swap by output branch of `splitAmountsAndFeesByHeight()` where, if internal rounding causes `totalAmountOutFilled > amountOut`

The code does not clamp output to `amountOut`, Instead it

- 1 - Treats the surplus as dust
- 2 - Stores it in `ptrPoolState.dust0/dust1`
- 3 - Leaves the swap charged `amountIn` unchanged (unless `totalAmountInFilled > amountIn`, which is not required here)

That creates a path where extra output is removed from reserves without charging additional input, and later that dust is swept into a withdrawal via `_accumulateDustToWithdrawal()`

In `splitAmountsAndFeesByHeight` swap by output branch

```
if (totalAmountOutFilled > amountOut) {  
    uint256 dust = totalAmountOutFilled - amountOut;  
    uint256 potentialDustForOneInput = calculateFixedSwapByRatio(1, swapCache.packedRatio,  
zeroForOne);  
    if (dust > potentialDustForOneInput) revert FixedPool__InvalidOutputDust();  
    amountOut = totalAmountOutFilled; // @audit, accepts larger internal out  
    if (zeroForOne) ptrPoolState.dust1 += dust;  
    else ptrPoolState.dust0 += dust;  
}
```

`potentialDustForOneInput` is not a correctness bound. It does not guarantee the dust is funded by the exact price, or the rounding slack of `amountIn`

So we can get dust even when the user `amountIn` is exactly correct for the requested `amountOut`

M-05 Unbacked Output Becomes Unfunded Dust

CONTINUED

<https://gist.github.com/GuardianAudits/4756c94a3e3deb556edbe2630043fc67>

— RECOMMENDATION

we need to enforce `totalAmountOutFilled <= amountOut` in `In` the exact output branch and not treat that surplus as dust or add it to `dust0/dust1`

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-06**

RESOLVED

Forced Top Up Makes Possible Swaps Revert

CATEGORY DoS
LOCATION FixedHelper.sol
REVIEW Remediation Review 3

DESCRIPTION

Inside `_splitAmountsAndFeesByHeight()` there is a block that forces the output height leg to top up to meet `amountOut` when the initial proportional split underfills

```
if (expectedAmountOutFilledByInputHeight + amountOutFilledByOutputHeight < amountOut) {  
    // Swap by output must fill the entire amountOut  
    amountOutFilledByOutputHeight = amountOut - expectedAmountOutFilledByInputHeight;  
    uint256 newOutputHeightInputShare =  
        calculateFixedSwapByRatioRoundingDown(consumedLiquidityOutputHeight +  
        amountOutFilledByOutputHeight, packedRatio, !zeroForOne);  
    uint256 actualAmountInFromOutputHeight = newOutputHeightInputShare -  
        swapCache.outputHeightInputShare;  
    expectedAmountInFilledByOutputHeight = actualAmountInFromOutputHeight;  
}
```

two important thing here, that Swap by output must fill the entire `amountOut` block, runs even when `swapCache.swapByInput = true` so an `swapByInput` can be forced into a situation where it tries to fill a precomputed `amountOut` using a split that is not feasible under discrete rounding when that top up occurs, the required input share on the output height is computed as a difference of floors

```
// C = consumedLiquidityOutputHeight  
// Δout = amountOutFilledByOutputHeight  
floor(( C + Δout) * ( num/den)) - floor(C * (num/den) )
```

that delta can jump discontinuously because of floor boundaries (when $\text{num} > \text{den}$, input per output > 1) so the required input for the output height leg can increase by more than the slack the function is willing / able to rebalance

Then the function hits its hard invariant check and reverts

```
if (totalAmountInFilled > amountIn) revert FixedPool__InputValidationFailed();
```

M-06 Forced Top Up Makes Possible Swaps Revert

CONTINUED

This becomes a DoS because the swap reverts even though there exists a valid split of the same amountIn that produces the requested amountOut

<https://gist.github.com/GuardianAudits/25dfa3a1562c14f8eb6a8d3a69895c00>

— RECOMMENDATION

We need to enforce running the forced top up block for swapByOutput only

— RESOLUTION

Status: Resolved.

MEDIUM **M-07**

ACKNOWLEDGED

Floor Rounding Stalls Height Consumption

CATEGORY DoS
LOCATION FixedHelper.sol
REVIEW Remediation Review 3

DESCRIPTION

In `calculateShareDeltaForLiquidityConsumption()` when the function detects that the desired output height consumption `consumedLiquidityDelta` is more than `availableLiquidity`

It tries to cap the consumption to `availableLiquidity` but in the capped branch it recomputes

```
newShareLiquidity = currentConsumedLiquidity + availableLiquidity;  
newShare = FullMath.mulDiv(newShareLiquidity, denominator, numerator); // floor  
newShareLiquidity = FullMath.mulDivRoundingUp(newShare, numerator, denominator); // ceil
```

Because `newShare` is computed with floor rounding, small increases in liquidity may not increase `newShare` at all (multiple liquidity values can map to the same integer share),

If the capped `availableLiquidity` isn't enough to cross the next rounding boundary, then `newShare` $\leq$ `currentShare`, meaning this height cannot make measurable progress in share terms, and the function returns

```
return (0, shareDelta);
```

So it reports

`consumedLiquidityDelta` = 0 (no liquidity consumed)

`unconsumedShareDelta` = `shareDelta` (all input share remains unused)

This is harmless by itself, but in the swap splitter `_splitAmountsAndFeesByHeight`, it becomes problematic because the swap is then forced to top up output, and the accounting ends up with

non zero output filled while total input filled becomes 0, hitting

```
revert FixedPool__ZeroValueSwap();
```

So the pool can report `expectedReserve > 0` but still hard revert swaps

<https://gist.github.com/GuardianAudits/9a8922282a8b1d1b3954e61637b33e6e>

M-07 Floor Rounding Stalls Height Consumption

CONTINUED

— RECOMMENDATION

We need to not allow `_splitAmountsAndFeesByHeight` to top up output when that height made no executable progress (`consumedLiquidityDelta == 0` or `usedShare == 0`)

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-08**

ACKNOWLEDGED

SwapByInput Can DoS In Valid States

CATEGORY DoS
LOCATION FixedHelper.sol
REVIEW Remediation Review 3

DESCRIPTION

In `_splitAmountsAndFeesByHeight`

```
// If there is excess output, convert to dust
if (totalAmountOutFilled > amountOut) {
    uint256 dust = totalAmountOutFilled - amountOut;
    // Validate output dust does not exceed the output of one input unit
    uint256 potentialDustForOneInput = calculateFixedSwapByRatio(1, swapCache.packedRatio,
zeroForOne);
    if (dust > potentialDustForOneInput) {
        revert FixedPool__InvalidOutputDust();
    }
}
```

This check runs for swap by input, because it's outside the `swapCache.swapByInput` / else branch

In `swapByInput`, the expected output is computed up front as

```
amountOut = floor(amountInAfterFees * ratio)
```

But `_splitAmountsAndFeesByHeight()` does not guarantee that the sum of

output generated by returning liquidity to the input height (`calculateShareDeltaForLiquidityReturn`)

+

output generated by consuming liquidity from the output height

(`calculateShareDeltaForLiquidityConsumption`)

will be $\leq$ that precomputed floor output (or only exceed it by ≤ 1 input unit of output)

In some valid states, the two legs independent rounding can make

```
totalAmountOutFilled - amountOut >= 2
```

M-08 SwapByInput Can DoS In Valid States

CONTINUED

When the fixed price has `calculateFixedSwapByRatio(1, zeroForOne) = 1` (when `ratio1 < ratio0` ,
price < 1 in zeroForOne), that dust bound is 1,

So dust = 2 becomes an unconditional revert

That means exact input swaps revert even though

amountOut <= expectedReserve (so swapByInput would normally proceed)

and there is liquidity and both legs make progress

— RECOMMENDATION

We can keep swap outputs exactly as they are, and make the output dust validity bound tolerate the known worst case two leg rounding in swap by input

<https://gist.github.com/GuardianAudits/25bae2b1b156358ab71278b01c5a15f1>

— RESOLUTION

Status: Acknowledged.

LOW **L-01**

RESOLVED

ISingleProviderPoolType Missing Inheritance

CATEGORY Best Practices

LOCATION SingleProviderPoolType.sol, ISingleProviderPoolType.sol

REVIEW Remediation Review 3

DESCRIPTION

The `ILimitBreakAMMPoolType` interface declares `collectFees`, `addLiquidity`, and `removeLiquidity` as non-view (state-changing) functions. However, `SingleProviderPoolType` declares them as `external view`, and `ISingleProviderPoolType` does not extend `ILimitBreakAMMPoolType`, unlike `IFixedPoolType` which does:

```
// IFixedPoolType.sol
interface IFixedPoolType is ILimitBreakAMMPoolType { ... }
// ISingleProviderPoolType.sol
interface ISingleProviderPoolType { ... } // no inheritance
```

The AMM core calls these via `ILimitBreakAMMPoolType(poolType).collectFees(...)` which makes a regular `CALL`. While function selectors still match (the `view` modifier is ABI metadata, not part of the selector), the compiler does not enforce interface compliance.

This means any future change to the `ILimitBreakAMMPoolType` base interface won't cause compilation failures in `SingleProviderPoolType`, removing the compile-time safety net for interface conformance.

RECOMMENDATION

Have `ISingleProviderPoolType` inherit from `ILimitBreakAMMPoolType`:

```
interface ISingleProviderPoolType is ILimitBreakAMMPoolType { ... }
```

RESOLUTION

Status: Resolved.

LOW **L-02**

RESOLVED

No Validation On Hook-Returned SqrtPrice

CATEGORY Validation

LOCATION SingleProviderPoolType.sol:324-329, SingleProviderPoolType.sol:407-412

REVIEW Remediation Review 3

DESCRIPTION

In both `swapByInput` and `swapByOutput`, the price is fetched from the hook and used directly without any validation:

```
pools[poolId].lastSqrtPriceX96 =  
swapCache.sqrtPriceCurrentX96 =  
ISingleProviderPoolHook(swapCache.poolHook).getPoolPriceForSwap(  
context,  
priceParams,  
swapExtraData  
);
```

There is no validation that the returned `sqrtPriceX96`:

1. Is non-zero
2. Falls within valid bounds (`MIN_SQRT_RATIO` to `MAX_SQRT_RATIO`, both defined in `Constants.sol`)

If hook returns `sqrtPriceX96 = 0`:

- `calculateFixedInput` does `FullMath.mulDiv(amountIn, 0, Q96) = 0` output -- swapper pays tokens, gets nothing
- `calculateFixedOutput` does `FullMath.mulDivRoundingUp(amountOut, Q96, 0)` -- division by zero

RECOMMENDATION

Add price validation after the hook call:

L-02 No Validation On Hook-Returned SqrtPrice

CONTINUED

```
uint160 hookPrice = ISingleProviderPoolHook(swapCache.poolHook).getPoolPriceForSwap(...);  
if (hookPrice < MIN_SQRT_RATIO || hookPrice >= MAX_SQRT_RATIO) {  
    revert SingleProviderPool__InvalidPrice();  
}  
pools[poolId].lastSqrtPriceX96 = swapCache.sqrtPriceCurrentX96 = hookPrice;
```

— RESOLUTION

Status: Resolved.

LOW **L-03**

ACKNOWLEDGED

getCurrentPriceX96 Is Likely Wrong

CATEGORY	Oracle
LOCATION	https://github.com/GuardianOrg/lbamm-pool-type-single-provider-team1-1757356753644/blob/626be46054f5846ba5c6f6c865f3e32cafc144ac/src/SingleProviderPoolType.sol#L425-L439
REVIEW	Remediation Review 3

— DESCRIPTION

The `SingleProviderPoolType` contract has a `getCurrentPriceX96` function which returns the `lastSqrtPriceX96` (the last price fetched from the hook).

The documentation of this function states out: `Returns the current square root price for a specific pool.`

While the documentation of the `SingleProviderPoolType` itself states out: `This pool type allows for highly customizable pools that can change pricing in response to who the executor of the swap is or price feeds from an oracle.`

This is therefore very likely not the current price as an oracle price changes constantly and the caller or other data is not taken into account.

— RECOMMENDATION

Consider to rethink this function or document clearly that it likely returns a wrong value.

— RESOLUTION

Status: Acknowledged.

LOW **L-04**

RESOLVED

Fixed Pool amountOut Exceeds Ceil Bound

CATEGORY Rounding
LOCATION FixedHelper.sol
REVIEW Remediation Review 3

DESCRIPTION

The fuzzing suite includes a handler, `fuzz_clob_poolSwap`, which routes an AMM `singleSwap` through a fixed-price pool while delivering swap output to the CLOB transfer handler. The postconditions for this path compute a conservative upper bound for `amountOut` (`expectedAmountOut`) from the pool's `packedRatio` and the net input after fees, and then assert that the actual swap output does not exceed that bound. In some states, the swap itself can succeed but the postcondition fails because the pool returns an `amountOut` that is 1 wei greater than the computed ceiling bound.

The failing assertion is in `clobPoolSwapPostconditions`:

```
fl.gt(amountOut, 0, "CLOB_SWAP_POOL_08: amountOut should be positive");  
fl.lte(amountOut, params.expectedAmountOut, "CLOB_SWAP_POOL_09: amountOut exceeds expected");
```

The value of `expectedAmountOut` for pool swaps is derived by modeling the fixed pool output from the pool's ratio using `FixedHelper.calculateFixedSwapByRatio` (which rounds up) as a claimed safe upper bound:

```
uint256 expectedPoolOutNoCapUpper =  
FixedHelper.calculateFixedSwapByRatio(amountInAfterFees, packedRatio, zeroForOne);
```

Conceptually, this is trying to enforce the bound `amountOut <= ceil(amountInAfterFees * price)` for a fixed price.

The root cause is a "double rounding" discrepancy introduced by the fixed pool's height-splitting execution path. Internally, `FixedHelper.swapByInput` computes an initial output estimate using `calculateFixedSwapByRatioRoundingDown(amountInAfterFees, packedRatio, zeroForOne)` and then delegates to height update logic that can decompose a single swap into two internal legs (one associated with the input height and one associated with the output height). Each leg performs its own conversion between input and output units with its own rounding rules. In particular, the output-height leg can round up (it uses `FullMath.mulDivRoundingUp` in the `numerator > denominator` branch), and the input-height leg can also effectively round up when it computes a share boundary delta. When both legs round up, the

L-04 Fixed Pool amountOut Exceeds Ceil Bound

CONTINUED

sum of the two rounded outputs can exceed the single-shot ceiling computed over the total input.

This is not an exotic edge case; it is a known mathematical property of rounding that $\text{ceil}(a) + \text{ceil}(b)$ can be greater than $\text{ceil}(a + b)$ by 1 when both a and b have fractional parts. A realistic example in wei makes this concrete. Assume a fixed price ratio of $3/2$ (so $\text{price} = 1.5$), and assume the swap's net input after fees is exactly 2000000000000000000 wei. A single-shot ceiling quote gives $\text{ceil}(2000000000000000000 * 3 / 2) = 3000000000000000000$ wei of output. Now assume the height-splitting logic allocates the input across two legs as $A = 9999999999999999$ and $B = 1000000000000000001$ (both are plausible in practice because the split is driven by proportional share math and boundary crossings). If each leg converts input to output using rounding up, the outputs become $\text{ceil}(A * 3 / 2) = 14999999999999999$ and $\text{ceil}(B * 3 / 2) = 1500000000000000002$, which sum to 3000000000000000001. In other words, splitting plus per-leg rounding can overpay the swap recipient by 1 wei relative to the global ceiling bound for the same total input and price ratio.

If this behavior is reachable in production configurations, it represents a systematic accounting deviation where `swapByInput` can overpay `tokenOut` relative to the pool price, transferring value from the pool (and ultimately LPs) to swappers. Even if the deviation is typically 1 wei per swap, it breaks conservative quoting assumptions and can accumulate into measurable loss under repeated interactions.

— RECOMMENDATION

In the fixed pool swap path, ensure the final `amountOut` produced by `swapByInput` cannot exceed the ratio-based ceiling derived from the same post-fee input and `packedRatio`. One straightforward fix is to compute `maxAmountOut = FixedHelper.calculateFixedSwapByRatio(amountInAfterFees, packedRatio, zeroForOne)` and clamp `totalAmountOutFilled` to `maxAmountOut`, treating any excess as dust that remains in the pool (for example by recording it in `dust0` or `dust1` for later attribution to LP withdrawals) rather than paying it to the swap recipient. Alternatively, adjust the height-splitting math so that rounding never increases the aggregate output beyond the global ceiling bound. If the protocol intentionally allows this overpayment, then the fuzzing suite's bound should be updated to reflect the intended tolerance; however, doing so should be accompanied by a protocol-level justification that the extra output cannot be exploited to extract value.

PROOF OF CONCEPT <https://github.com/GuardianOrg/lbamm-core-fuzz-1763932869200/blob/5edd5222ae3603a5be6e5b79254294cc241b6274/test/fuzzing/FoundryPlayground.sol#L522>

L-04 Fixed Pool amountOut Exceeds Ceil Bound

CONTINUED

— RESOLUTION

Status: Resolved.

LOW **L-05**

RESOLVED

Zero Output Swap Becomes One Output

CATEGORY Rounding
LOCATION FixedHelper.sol
REVIEW Remediation Review 3

DESCRIPTION

When $\text{ratio} < 1$

`calculateShareDeltaForLiquidityReturn()` can return a `shareDelta` of 1 for returning only 1 unit of input, even when the true fixed price floor output should be 0

This means a trade with `amount0out = 0` can be upgraded to `amount0out = 1` inside

`_splitAmountsAndFeesByHeight()`

That function computes

```
newShare = floor(totalConsumedLiquidity * numerator / denominator)
shareDelta = currentShare - newShare
```

This is a difference of floors

$\text{share delta} = \text{the floor of (total consumed liquidity } \times \text{ ratio)} - \text{the floor of ((total consumed liquidity - returned liquidity) } \times \text{ ratio)}$

For a small liquidity delta, this can be 0 or 1, but the 1 happens at boundaries

That boundary jump produces a marginal exchange rate larger than the pool ratio

In `swapByInput`,

`_splitAmountsAndFeesByHeight` overwrites `swapCache.amount0out` with `totalAmountOutFilled`

and never clamps it to the precomputed floor output, so a 0 output swap can become 1 output

<https://gist.github.com/GuardianAudits/ca6c1c71edaa67cb1a0f33b54ea74cfe>

RECOMMENDATION

Consider treating the precomputed floor quote in `swapByInput` as a hard upper bound, and don't let `_splitAmountsAndFeesByHeight()` increase it.

L-05 Zero Output Swap Becomes One Output

CONTINUED

— **RESOLUTION**

Status: Resolved.

INFORMATIONAL **I-01**

RESOLVED

Fixed Quoter NatSpec Mismatch

CATEGORY Documentation
LOCATION FixedPoolQuoter.sol
REVIEW Remediation Review 3

DESCRIPTION

The fixed pool quoter's NatSpec for `quoteValueRequiredForInRangeAdd` describes return values in a way that does not match the apparent `(amount0, amount1)` naming and typical token0/token1 conventions. As written, the documentation can be read as implying the returned amounts correspond to the opposite token side.

This kind of documentation mismatch is a common source of integration bugs in routers and frontends, especially when quoting functions are used to construct calldata or UI prompts.

RECOMMENDATION

Update the NatSpec to clearly state which return value corresponds to token0 and token1 amounts (and under what conditions either value can be zero), and add a short example in documentation/tests to lock in intended semantics.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-02

RESOLVED

NatSpec Claims Non-Existent Event

CATEGORY Documentation
LOCATION SingleProviderPoolType.sol:217
REVIEW Remediation Review 3

DESCRIPTION

The NatSpec for `removeLiquidity` claims:

Postconditions

1. The withdraw amounts and fees are returned to the caller.
2. A `SingleProviderPoolLiquidityRemoved` event is emitted with the pool ID, withdraw amounts, and fees.

However:

1. The `SingleProviderPoolLiquidityRemoved` event is not defined anywhere -- not in `ISingleProviderPoolType.sol` nor in `SingleProviderPoolType.sol`
2. The `removeLiquidity` function body contains no `emit` statement -- no event is emitted at all

The only event defined in the interface is `SingleProviderPoolSwapDetails`, which is only emitted in the swap functions.

RECOMMENDATION

Either define and emit the `SingleProviderPoolLiquidityRemoved` event as documented, or remove the claim from the NatSpec if no event is intended.

RESOLUTION

Status: Resolved.

MEDIUM **M-01**

ACKNOWLEDGED

FixedPoolType LP Sybil Attack

CATEGORY	Rewards
LOCATION	FixedPoolType
REVIEW	Remediation Review 4

— DESCRIPTION

Swaps will consume the current height before moving on to the next one. Multiple users are able to open positions in the same height and only if all of them are consumed, will the system touch the liquidity of the next height.

This logic enables a sybil attack which allows a single LP to likely gain all the fees while passive LPs which deposited liquidity among multiple heights own a very capital inefficient position:

- Precision of the FixedPool is 100
- Alice deposits \$100k liquidity from height 0 to 100k
- Eve creates 1000 different accounts and deposits \$100 liquidity with every account from height 0 to 100
- Now Eve will earn fees on \$100k liquidity while Alice only earns fees on \$100 liquidity before the rest of Alice's liquidity is touched. And Eve is able to add more liquidity to her position before that happens, which means Alice will likely never earn more fees.

Therefore a sybil attack is way more profitable for LPs than using the system as intended.

— RECOMMENDATION

Be aware about this and consider to rethinking the FixedPool logic in general or to document the behavior.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-02**

ACKNOWLEDGED

minimumProtocolFee Not Symmetric Across Flows

CATEGORY Math
LOCATION AMModule.sol
REVIEW Remediation Review 4

DESCRIPTION

The `minimumProtocolFee` is calculated based on the `amountIn` in both the `SwapByInput` and `SwapByOutput` flow. However different fees and other factors adjust the `amountIn` before this calculation in different ways depending on the chose swap type:

- `SwapByInput`
- `amountIn` decreased by `exchangeFee` & `feeOnTop`
- `minimumProtocolFee` calculated based on `amountIn`
- `SwapByOutput`
- `amountOut` increased by `beforeSwapHookFees` (influencing needed `amountIn`)
- swap happens: `amountIn` may be decreased as only a partial fill was possible
- `minimumProtocolFee` calculated based on needed `amountIn`

As we can see the `minimumProtocolFee` charged may differ and therefore the users may pay more or less fees depending on if they use the `SwapByInput` or `SwapByOutput` function.

RECOMMENDATION

Consider to make adjust these flows so that the `minimumProtocolFee` is the same no matter if the user decides to use the `SwapByInput` or `SwapByOutput` function.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-03**

RESOLVED

Floor Math Arbitrage Extracts Reserves

CATEGORY Rounding
LOCATION FixedHelper.sol
REVIEW Remediation Review 4

DESCRIPTION

Adding in range liquidity can increase height.consumedLiquidity because we pre consume depth below currentHeight

```
//_addLiquidity()  
if (startHeight < currentHeight) {  
    height.consumedLiquidity += (currentHeight - startHeight);  
}
```

The paired token amount required to support that pre consumed depth is computed using floor math, difference of floors

```
// _calculateLiquidityStartAndEndHeights() for addInRange0  
depth0ValueOf1 =  
    calculateFixedSwapByRatioRoundingDown(consumedLiquidity0 + depth0, packedRatio, true) -  
    calculateFixedSwapByRatioRoundingDown(consumedLiquidity0, packedRatio, true);
```

Because the conversion uses floor rounding math, sometimes adding +1 unit of depth doesn't increase the output at all (it rounds down to the same integer), so the extra value is 0

When a position is later withdrawn, the paired value for the consumed part is again computed as a difference of floors, but the vulnerability here is that

It uses the current global height.consumedLiquidity and then subtracts from it

```
//_collectPositionSide()  
uint256 consumedLiquidity = height.consumedLiquidity;  
pairValue =  
    floor(consumedLiquidity) -  
    floor(consumedLiquidity - consumedDelta);  
height.consumedLiquidity -= (liquidity - sideValue);
```

M-03 Floor Math Arbitrage Extracts Reserves

CONTINUED

This makes global consumedLiquidity behave like a LIFO stack the withdrawing position claims the top most marginal segment of the global floor function, not necessarily the marginal segment that corresponded to its own historical pre consumption.

And because the conversion is $\text{floor} (x * \text{numerator} / \text{denominator})$, adding 1 more unit of x doesn't always increase the output by the same amount, sometimes it increases by 0, sometimes by 1 or more, depending on where x sits relative to the denominator (the remainder $x \% \text{denominator}$)

That lets an attacker

Add in range at a global consumedLiquidity where the marginal conversion for +depth is 0 ,they pay 0 paired token for that pre consumed depth

Then wait until swaps move global consumedLiquidity to a point where the marginal conversion for the top segment is 1

Withdraw their position and receive 1 paired token for the same consumedDelta, because collectPositionSide computes the delta using the current global consumedLiquidity and subtracts from the top

This repeatable rounding arbitrage that can keep extracting pool reserves

example, picking a ratio where rounding matters

$\text{ratio0} = 3, \text{ratio1} = 2$, so $\text{token0} \rightarrow \text{token1} = \text{floor} (x * 2/3)$

Let $\text{depth} = 1$ (happens whenever $\text{currentHeight} \% \text{precision} \neq 0$ and $\text{addInRange0} = \text{true}$)

Assume before attacker adds liquidity

$\text{height0.consumedLiquidity} = 3$

Then the in range depth cost in token1 is

```
floor ( (3 + 1) * 2 / 3 ) - floor ( 3 * 2 / 3 )
```

```
floor ( 8 / 3 ) - floor ( 6 / 3 )
```

```
2 - 2 = 0
```

So attacker can add an in range position that increases consumedLiquidity by 1, but pays 0 token1 for the consumed portion

Now let swaps push global height0.consumedLiquidity to 5

When attacker withdraws and the position consumedDelta includes that same 1 unit, collectPositionSide computes

M-03 Floor Math Arbitrage Extracts Reserves

CONTINUED

```
floor ( 5 * 2/3 ) - floor ( 4 * 2/3 )
```

```
floor ( 10/3 ) - floor ( 8/3 )
```

```
3 - 2 = 1
```

Attacker receives 1 token1 for that consumedDelta = 1, even though they paid 0 token1 for that pre consumed unit

That 1 token1 comes out of pool value, it's not newly minted, it's misattributed due to floor boundaries plus the LIFO subtraction

This can be repeated to harvest rounding increments whenever

addInRange paths are used (startHeight < currentHeight occurs),

the pool global consumedLiquidity moves across favorable boundaries

This lets an exploiter extract from the pool reserves by timing add / withdraw around discrete rounding boundaries

— RECOMMENDATION

Not recorded.

PROOF OF CONCEPT <https://gist.github.com/GuardianAudits/42402a849c5244c304dd41f06efc5e9b>

— RESOLUTION

Status: Resolved.

MEDIUM **M-04**

RESOLVED

Pool Has Reserves And swapByOutput Reverts

CATEGORY DoS
LOCATION FixedHelper.sol
REVIEW Remediation Review 4

DESCRIPTION

`swapByOutput` can revert even when the pool has enough reserves

In `splitAmountsAndFeesByHeight()` inside the swap by output branch

```
if (totalAmountInFilled > amountIn) {  
    // Allow a maximum of 1 unit of input over the initial amountIn for split rounding  
    if (totalAmountInFilled > amountIn + 1) {  
        revert FixedPool__InputValidationFailed();  
    }  
}
```

This assumes that the internal splitting math can only ever require at most +1 extra input unit beyond the initial `amountIn` computed from the fixed ratio

But that assumption is not always true, there are valid pool states where

the pool has enough expected reserves to satisfy the requested `amountOut`, and

`reserveAmountIn = calculateFixedSwapByRatio(amountOut, packedRatio, !zeroForOne)` is correct for the fixed price math,

but `_splitAmountsAndFeesByHeight()` ends up with `totalAmountInFilled > amountIn + 1` and the swap everts with `FixedPool__InputValidationFailed()`

The `_splitAmountsAndFeesByHeight()` tries to split the swap across two virtual reserve sources

Input height return path and output height consumption path

for `swapByOutput`, it must ensure the requested `amountOut` is fully filled, The function

starts with a proportional split of `amountIn` between the two sources (based on `inputShareOfExpectedReserve / expectedReserve`)

M-04 Pool Has Reserves And swapByOutput Reverts

CONTINUED

then, if output is underfilled, it forces the remainder to be produced by the output height, and recomputes how much input that forced output height consumption actually requires , `actualAmountInFromOutputHeight`

The key nonlinearity here

The mapping `consumedLiquidityOutputHeight -> outputHeightInputShare`

uses floor math `calculateFixedSwapByRatioRoundingDown` , so it moves in jumps at share boundaries.

So when we force output height to fill the remaining output, `outputHeightInputShare` can jump by more than one unit, the required input can increase by 2+ units at once

The algorithm tries to offset that by reducing the input height allocation, but it can only reduce it by amounts that do not change the input height output (`returnableInput`) or by already known unfilled input (`unfilledInput`)

In many states, that slack is 0

when the jump is bigger than the available slack, we get

`totalAmountInFilled = amountInFilledByInputHeight + actualAmountInFromOutputHeight`

and it can exceed `amountIn + 1` by more than 1, triggering the revert

But crucially, a different split can exist that works, like pushing more through output height, less through input height, but the function doesn't search for it, it just asserts the jump can't be > 1 and revert

<https://gist.github.com/GuardianAudits/7ea11404a030fc15e0843ada317ef0fe>

— RECOMMENDATION

we need to make `_splitAmountsAndFeesByHeight()` search for a feasible split in swap by output

— RESOLUTION

Status: Resolved.

MEDIUM **M-06**

RESOLVED

Forced Top Up Makes Possible Swaps Revert

CATEGORY DoS
LOCATION FixedHelper.sol
REVIEW Remediation Review 4

DESCRIPTION

Inside `_splitAmountsAndFeesByHeight()` there is a block that forces the output height leg to top up to meet `amountOut` when the initial proportional split underfills

```
if (expectedAmountOutFilledByInputHeight + amountOutFilledByOutputHeight < amountOut) {  
    // Swap by output must fill the entire amountOut  
    amountOutFilledByOutputHeight = amountOut - expectedAmountOutFilledByInputHeight;  
    uint256 newOutputHeightInputShare =  
        calculateFixedSwapByRatioRoundingDown(consumedLiquidityOutputHeight +  
        amountOutFilledByOutputHeight, packedRatio, !zeroForOne);  
    uint256 actualAmountInFromOutputHeight = newOutputHeightInputShare -  
        swapCache.outputHeightInputShare;  
    expectedAmountInFilledByOutputHeight = actualAmountInFromOutputHeight;  
}
```

two important thing here, that Swap by output must fill the entire `amountOut` block, runs even when `swapCache.swapByInput = true` so an `swapByInput` can be forced into a situation where it tries to fill a precomputed `amountOut` using a split that is not feasible under discrete rounding when that top up occurs, the required input share on the output height is computed as a difference of floors

```
// C = consumedLiquidityOutputHeight  
// Δout = amountOutFilledByOutputHeight  
floor(( C + Δout) * ( num/den)) - floor(C * (num/den) )
```

that delta can jump discontinuously because of floor boundaries (when $\text{num} > \text{den}$, input per output > 1) so the required input for the output height leg can increase by more than the slack the function is willing / able to rebalance

Then the function hits its hard invariant check and reverts

```
if (totalAmountInFilled > amountIn) revert FixedPool__InputValidationFailed();
```

M-06 Forced Top Up Makes Possible Swaps Revert

CONTINUED

This becomes a DoS because the swap reverts even though there exists a valid split of the same amountIn that produces the requested amountOut

<https://gist.github.com/GuardianAudits/25dfa3a1562c14f8eb6a8d3a69895c00>

— RECOMMENDATION

We need to enforce running the forced top up block for swapByOutput only

— RESOLUTION

Status: Resolved.

MEDIUM **M-07**

ACKNOWLEDGED

Floor Rounding Stalls Height Consumption

CATEGORY DoS
LOCATION FixedHelper.sol
REVIEW Remediation Review 4

DESCRIPTION

In `calculateShareDeltaForLiquidityConsumption()` when the function detects that the desired output height consumption `consumedLiquidityDelta` is more than `availableLiquidity`

It tries to cap the consumption to `availableLiquidity` but in the capped branch it recomputes

```
newShareLiquidity = currentConsumedLiquidity + availableLiquidity;  
newShare = FullMath.mulDiv(newShareLiquidity, denominator, numerator); // floor  
newShareLiquidity = FullMath.mulDivRoundingUp(newShare, numerator, denominator); // ceil
```

Because `newShare` is computed with floor rounding, small increases in liquidity may not increase `newShare` at all (multiple liquidity values can map to the same integer share),

If the capped `availableLiquidity` isn't enough to cross the next rounding boundary, then `newShare` $\leq$ `currentShare`, meaning this height cannot make measurable progress in share terms, and the function returns

```
return (0, shareDelta);
```

So it reports

`consumedLiquidityDelta` = 0 (no liquidity consumed)

`unconsumedShareDelta` = `shareDelta` (all input share remains unused)

This is harmless by itself, but in the swap splitter `_splitAmountsAndFeesByHeight`, it becomes problematic because the swap is then forced to top up output, and the accounting ends up with

non zero output filled while total input filled becomes 0, hitting

```
revert FixedPool__ZeroValueSwap();
```

So the pool can report `expectedReserve > 0` but still hard revert swaps

<https://gist.github.com/GuardianAudits/9a8922282a8b1d1b3954e61637b33e6e>

M-07 Floor Rounding Stalls Height Consumption

CONTINUED

— RECOMMENDATION

We need to not allow `_splitAmountsAndFeesByHeight` to top up output when that height made no executable progress (`consumedLiquidityDelta == 0` or `usedShare == 0`)

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-08**

ACKNOWLEDGED

SwapByInput Can DoS In Valid States

CATEGORY DoS
LOCATION FixedHelper.sol
REVIEW Remediation Review 4

DESCRIPTION

In `_splitAmountsAndFeesByHeight`

```
// If there is excess output, convert to dust
if (totalAmountOutFilled > amountOut) {
    uint256 dust = totalAmountOutFilled - amountOut;
    // Validate output dust does not exceed the output of one input unit
    uint256 potentialDustForOneInput = calculateFixedSwapByRatio(1, swapCache.packedRatio,
zeroForOne);
    if (dust > potentialDustForOneInput) {
        revert FixedPool__InvalidOutputDust();
    }
}
```

This check runs for swap by input, because it's outside the `swapCache.swapByInput` / else branch

In `swapByInput`, the expected output is computed up front as

```
amountOut = floor(amountInAfterFees * ratio)
```

But `_splitAmountsAndFeesByHeight()` does not guarantee that the sum of

output generated by returning liquidity to the input height (`calculateShareDeltaForLiquidityReturn`)

+

output generated by consuming liquidity from the output height

(`calculateShareDeltaForLiquidityConsumption`)

will be $\leq$ that precomputed floor output (or only exceed it by ≤ 1 input unit of output)

In some valid states, the two legs independent rounding can make

```
totalAmountOutFilled - amountOut >= 2
```

M-08 SwapByInput Can DoS In Valid States

CONTINUED

When the fixed price has `calculateFixedSwapByRatio(1, zeroForOne) = 1` (when `ratio1 < ratio0` ,
price < 1 in zeroForOne), that dust bound is 1,

So dust = 2 becomes an unconditional revert

That means exact input swaps revert even though

amountOut <= expectedReserve (so swapByInput would normally proceed)

and there is liquidity and both legs make progress

— RECOMMENDATION

We can keep swap outputs exactly as they are, and make the output dust validity bound tolerate the known worst case two leg rounding in swap by input

<https://gist.github.com/GuardianAudits/25bae2b1b156358ab71278b01c5a15f1>

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-09**

ACKNOWLEDGED

minimumProtocolFee Not Reduced On Partial Fill

CATEGORY Math
LOCATION AMModule.sol
REVIEW Remediation Review 4

— DESCRIPTION

In case only a partial fill was possible fees like for example the `exchangeFee` is adjusted to match the new swap amount.

However the `minimumProtocolFee` was already deducted from the `amountIn` before the swap happened in the `SwapByInput` flow and is not decremented in case of a partial fill. Therefore the user could pay way too much fees in that case.

— RECOMMENDATION

Consider to adjust this charged fee and give it back to the user.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-10**

ACKNOWLEDGED

Missing Token Flashloan Validation

CATEGORY Validation
LOCATION LBAMM.sol
REVIEW Remediation Review 4

— DESCRIPTION

The code has a defined per token configuration bit

```
/// @dev Token setting flag enabling flash loan operations for the token  
uint16 constant TOKEN_SETTINGS_FLASHLOANS_FLAG = 1 << 7;
```

But the actual flashloan execution path never requires this flag to be set

In `_flashLoan()`, the only gating check is global

```
if (Storage.appStorage().flashLoanBPS > MAX_BPS) revert LBAMM__FlashloansDisabled();
```

Then it calls

```
(address feeToken, uint256 tokenFeeAmount) = _executeTokenFlashloanHooks(flashloanRequest,  
tokenSettings);
```

And `_executeTokenFlashloanHooks()` only uses `TOKEN_SETTINGS_FLASHLOANS_FLAG` to decide whether to call the token hook

If the flag is not set, the hook does not run, but the flashloan continues anyway

This could lead to policy violation issues for some tokens on Limit Break, because the current implementation forces all tokens to be allowed to be flashloaned

If some tokens disallow this and try to enforce their tokens to be used in specific places, all their policies could be bypassed.

— RECOMMENDATION

Add this error and enforce it for that case:

```
error LBAMM__FlashloansDisabledForToken();
```

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-11**

ACKNOWLEDGED

Missing Collect Fees Hook

CATEGORY Logical Error
LOCATION LBAMM.sol
REVIEW Remediation Review 4

DESCRIPTION

In the amm `_positionAddLiquidity()` we collect fees returned by the pool type by

```
(
  context.positionId,
  deposit0,
  deposit1,
  fees0,
  fees1
) = ILimitBreakAMMPoolType(
  PoolDecoder.getPoolType(liquidityParams.poolId)
).addLiquidity();
if (fees0 > 0) {
  poolState.feeBalance0 = _safeDecrementUint128(poolState.feeBalance0, fees0);
}
if (fees1 > 0) {
  poolState.feeBalance1 = _safeDecrementUint128(poolState.feeBalance1, fees1);
}
// net flow includes fee payout
_distributeAndCollectLiquidityTokens(
  context.provider,
  context.token0,
  context.token1,
  deposit0.toInt256() - fees0.toInt256() + hookFee0.toInt256(),
  deposit1.toInt256() - fees1.toInt256() + hookFee1.toInt256()
);
```

But the hooks we execute are only the add liquidity hooks:

```
(hookFee0, hookFee1) = _executeAddLiquidityHooks(); // validateAddLiquidity hooks
```

M-11 Missing Collect Fees Hook

CONTINUED

And the system allow collect fees hooks to execute when calling the collect fees. But here in both `AddLiquidity` and `RemoveLiquidity` the collect fees hook path is never invoked, even though fees are being paid out.

Those are not being called

- `TOKEN_SETTINGS_COLLECT_FEES_HOOK_FLAG`
- `ILimitBreakAMMTokenHook.validateCollectFees`
- `ILimitBreakAMMLiquidityHook.validatePositionCollectFees`
- `ILimitBreakAMMPoolHook.validatePoolCollectFees`

If any token/pool/position relies on collect-fees hooks to enforce restrictions or fees on fee withdrawal, an LP can bypass that entire policy by withdrawing fees via `addLiquidity()` instead of calling `collectFees()`.

— RECOMMENDATION

When `fees0 > 0 || fees1 > 0` inside `_positionAddLiquidity`, also run the collect-fees hooks.

Or document it for hook devs that any policies related with fees they enforce in Collectfees only and not in `addLiquidity` and `removeLiquidity` too can be bypassed, so they make sure to enforce them there as well.

— RESOLUTION

Status: Acknowledged.

LOW **L-01**

ACKNOWLEDGED

Potential DoS on fillOrder

CATEGORY	DoS
LOCATION	CL0BHelper.sol
REVIEW	Remediation Review 4

DESCRIPTION

`openOrder` debits the maker then inserts the order into a per-price linked list. The only size checks are `orderAmount >= getGroupKeyMinimumOrder` (freely chosen in calldata via `groupKey`) and `< type(uint128).max`, so an attacker can open huge numbers of dust orders at any price bucket.

On execution, the AMM calls `ammHandleTransfer` → `CL0BHelper.fillOrder`, which starts at `currentPrice` and loops `while (fillInputRemaining != 0)` through every order in FIFO order, advancing with `traverseCL0B`. There is no iteration cap and callers cannot skip the head bucket.

Spam is stored on-chain, so every legitimate fill at that price must touch every attacker order; gas grows linearly until the swap runs out of gas and reverts. Clearing the spam costs the same gas, so the DoS persists across blocks.

Because the engine only moves off a price after the bucket is emptied, seeding dust buckets above/below the prevailing price (e.g., 15k orders at ~99 and 15k at ~101 when mid is ~100) effectively pins the pair: any trade that would cross those buckets must churn through the attacker's queues first, making fills at the intended level infeasible.

`fillOrder` walks every order at the best price in a while-loop with no iteration cap:

```
while (fillInputRemaining != 0) {  
    // ...  
    (ptrOrderBucket, ptrOrder, orderInputRemaining, currentPrice) = traverseCL0B(...);  
    // ...  
}
```

RECOMMENDATION

Enforce meaningful minimum order sizes and/or per-price order caps, and add a configurable max-iteration or gas guard in `fillOrder` so fillers can batch progress instead of reverting an entire swap on excessive order counts.

L-01 Potential DoS on fillOrder

CONTINUED

PROOF OF CONCEPT <https://gist.github.com/GuardianAudits/ee4db123e9c48dd7bfc6d7e1fec991f3>

— RESOLUTION

Status: Acknowledged.

LOW **L-02**

ACKNOWLEDGED

Lacking Zero Validation

CATEGORY Warning
 LOCATION CLOBHelper.sol
 REVIEW Remediation Review 4

DESCRIPTION

openOrder() allows `orderAmount == 0`, and only checks

```
if (orderAmount < getGroupKeyMinimumOrder(groupKey)) revert;
```

But getGroupKeyMinimumOrder(groupKey) is

```
minimumOrder = minimumOrderBase * 10^minimumOrderScale;
```

If the anyone sets `minimumOrderBase = 0` in groupKey, then `orderAmount == 0` passes. No deposit is required because the collect deposit branch only triggers when

```
depositBalance < orderAmount
```

openOrder() stores that order with `inputAmount = 0`

```
ptrOrder.inputAmount = orderAmount; // = 0
```

But the system uses `inputAmount == 0` as the filled / closed sentinel. `closeOrder()` rejects such an order forever

```
if (ptrOrder.inputAmount == 0) {
  revert CLOBTransferHandler__OrderInvalidFilledOrClosed();
}
```

fillOrder() reverts when the next order has `inputAmount == 0` and there is still input left to fill. After fully consuming an order it does

```
(ptrOrderBucket, ptrOrder, orderInputRemaining, currentPrice) = traverseCLOB();
if (orderInputRemaining == 0) {
  if (fillInputRemaining != 0) {
    revert CLOBTransferHandler__InsufficientInputToFill();
  }
}
```

This could be exploited by doing two zero amount orders, which permanently bricks the orderbook. The first `0` order makes `currentPrice = MIN_SQRT_RATIO`. The second `0` order becomes

L-02 Lacking Zero Validation

CONTINUED

`nextOrder` after the firstNow any AMM fill that needs `inputAmount > 0` will hitcurrent order has `inputAmountRemaining = 0` traversal moves to the next order, which is also `0orderInputRemaining == 0` && `fillInputRemaining != 0` keeps revertingAdding as low because most orderbook creators will enforce a minimum order correctly, but we can add a check for caution

— RECOMMENDATION

Enforce that the group minimum cannot be zero. Reject `minimumOrderBase == 0` when initializing a `groupKey`

— RESOLUTION

Status: Acknowledged.

LOW **L-04**

RESOLVED

Fixed Pool amountOut Exceeds Ceil Bound

CATEGORY	Rounding
LOCATION	FixedHelper.sol
REVIEW	Remediation Review 4

DESCRIPTION

The fuzzing suite includes a handler, `fuzz_clob_poolSwap`, which routes an AMM `singleSwap` through a fixed-price pool while delivering swap output to the CLOB transfer handler. The postconditions for this path compute a conservative upper bound for `amountOut` (`expectedAmountOut`) from the pool's `packedRatio` and the net input after fees, and then assert that the actual swap output does not exceed that bound. In some states, the swap itself can succeed but the postcondition fails because the pool returns an `amountOut` that is 1 wei greater than the computed ceiling bound.

The failing assertion is in `clobPoolSwapPostconditions`:

```
fl.gt(amountOut, 0, "CLOB_SWAP_POOL_08: amountOut should be positive");
fl.lte(amountOut, params.expectedAmountOut, "CLOB_SWAP_POOL_09: amountOut exceeds expected");
```

The value of `expectedAmountOut` for pool swaps is derived by modeling the fixed pool output from the pool's ratio using `FixedHelper.calculateFixedSwapByRatio` (which rounds up) as a claimed safe upper bound:

```
uint256 expectedPoolOutNoCapUpper =
FixedHelper.calculateFixedSwapByRatio(amountInAfterFees, packedRatio, zeroForOne);
```

Conceptually, this is trying to enforce the bound `amountOut <= ceil(amountInAfterFees * price)` for a fixed price.

The root cause is a "double rounding" discrepancy introduced by the fixed pool's height-splitting execution path. Internally, `FixedHelper.swapByInput` computes an initial output estimate using `calculateFixedSwapByRatioRoundingDown(amountInAfterFees, packedRatio, zeroForOne)` and then delegates to height update logic that can decompose a single swap into two internal legs (one associated with the input height and one associated with the output height). Each leg performs its own conversion between input and output units with its own rounding rules. In particular, the output-height leg can round up (it uses `FullMath.mulDivRoundingUp` in the `numerator > denominator` branch), and the input-height leg can also effectively round up when it computes a share boundary delta. When both legs round up, the

L-04 Fixed Pool amountOut Exceeds Ceil Bound

CONTINUED

sum of the two rounded outputs can exceed the single-shot ceiling computed over the total input.

This is not an exotic edge case; it is a known mathematical property of rounding that $\text{ceil}(a) + \text{ceil}(b)$ can be greater than $\text{ceil}(a + b)$ by 1 when both a and b have fractional parts. A realistic example in wei makes this concrete. Assume a fixed price ratio of $3/2$ (so $\text{price} = 1.5$), and assume the swap's net input after fees is exactly 2000000000000000000 wei. A single-shot ceiling quote gives $\text{ceil}(2000000000000000000 * 3 / 2) = 3000000000000000000$ wei of output. Now assume the height-splitting logic allocates the input across two legs as $A = 9999999999999999$ and $B = 1000000000000000001$ (both are plausible in practice because the split is driven by proportional share math and boundary crossings). If each leg converts input to output using rounding up, the outputs become $\text{ceil}(A * 3 / 2) = 14999999999999999$ and $\text{ceil}(B * 3 / 2) = 1500000000000000002$, which sum to 3000000000000000001. In other words, splitting plus per-leg rounding can overpay the swap recipient by 1 wei relative to the global ceiling bound for the same total input and price ratio.

If this behavior is reachable in production configurations, it represents a systematic accounting deviation where `swapByInput` can overpay `tokenOut` relative to the pool price, transferring value from the pool (and ultimately LPs) to swappers. Even if the deviation is typically 1 wei per swap, it breaks conservative quoting assumptions and can accumulate into measurable loss under repeated interactions.

— RECOMMENDATION

In the fixed pool swap path, ensure the final `amountOut` produced by `swapByInput` cannot exceed the ratio-based ceiling derived from the same post-fee input and `packedRatio`. One straightforward fix is to compute `maxAmountOut = FixedHelper.calculateFixedSwapByRatio(amountInAfterFees, packedRatio, zeroForOne)` and clamp `totalAmountOutFilled` to `maxAmountOut`, treating any excess as dust that remains in the pool (for example by recording it in `dust0` or `dust1` for later attribution to LP withdrawals) rather than paying it to the swap recipient. Alternatively, adjust the height-splitting math so that rounding never increases the aggregate output beyond the global ceiling bound. If the protocol intentionally allows this overpayment, then the fuzzing suite's bound should be updated to reflect the intended tolerance; however, doing so should be accompanied by a protocol-level justification that the extra output cannot be exploited to extract value.

PROOF OF CONCEPT <https://github.com/GuardianOrg/lbamm-core-fuzz-1763932869200/blob/5edd5222ae3603a5be6e5b79254294cc241b6274/test/fuzzing/FoundryPlayground.sol#L522>

L-04 Fixed Pool amountOut Exceeds Ceil Bound

CONTINUED

— RESOLUTION

Status: Resolved.

LOW **L-05**

RESOLVED

Zero Output Swap Becomes One Output

CATEGORY Rounding
LOCATION FixedHelper.sol
REVIEW Remediation Review 4

DESCRIPTION

When ratio < 1

`calculateShareDeltaForLiquidityReturn()` can return a shareDelta of 1 for returning only 1 unit of input, even when the true fixed price floor output should be 0

This means a trade with `amount0out = 0` can be upgraded to `amount0out = 1` inside

`_splitAmountsAndFeesByHeight()`

That function computes

```
newShare = floor(totalConsumedLiquidity * numerator / denominator)
shareDelta = currentShare - newShare
```

This is a difference of floors

share delta = the floor of (total consumed liquidity × ratio) - the floor of ((total consumed liquidity – returned liquidity) × ratio)

For a small liquidity delta, this can be 0 or 1, but the 1 happens at boundaries

That boundary jump produces a marginal exchange rate larger than the pool ratio

In `swapByInput`,

`_splitAmountsAndFeesByHeight` overwrites `swapCache.amount0out` with `totalAmountOutFilled`

and never clamps it to the precomputed floor output, so a 0 output swap can become 1 output

<https://gist.github.com/GuardianAudits/ca6c1c71edaa67cb1a0f33b54ea74cfe>

RECOMMENDATION

Consider treating the precomputed floor quote in `swapByInput` as a hard upper bound, and don't let `_splitAmountsAndFeesByHeight()` increase it.

L-05 Zero Output Swap Becomes One Output

CONTINUED

— **RESOLUTION**

Status: Resolved.

MEDIUM **M-01**

ACKNOWLEDGED

Hook Price Bounds Require Both Swap Hooks

CATEGORY Configuration
LOCATION AMMStandardHook.sol
REVIEW Remediation Review 5

DESCRIPTION

`AMMStandardHook` price bounds only work correctly when both `beforeSwap` and `afterSwap` are enabled.

The hook now uses a two-step flow:

1. `beforeSwap()` caches the pre-swap price or direct-swap amount.
2. `afterSwap()` validates the executed post-swap price against that cached value.

But `AMMStandardHook` marks both swap hooks as optional by setting `_requiredHookFlags = 0`. Core therefore accepts configurations with only one swap hook enabled.

This lets price-bound tokens be configured so their `minSqrtPriceX96` / `maxSqrtPriceX96` protections are bypassed or evaluated using a missing cache value.

RECOMMENDATION

Require both swap hook flags whenever price bounds are used.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-02**

RESOLVED

Hook call memory is underallocated

CATEGORY Unexpected Behavior
LOCATION AMModule.sol
REVIEW Remediation Review 5

DESCRIPTION

`_allocateHookMemory` reserves too little memory for the calldata buffers built by the optimized hook assembly. It computes the allocation as `HOOK_ALLOCATION_BASE + ceil32(swapCache.hookLongestData)`, and `HOOK_ALLOCATION_BASE` is `576` (`0x240`). That size does not cover the largest hook calldata layout.

`_executeSwapHook` writes the dynamic `hookData` length at `hookMemoryPointer + 0x260` and copies the padded hook data at `hookMemoryPointer + 0x280`. The external call then uses input data starting at `hookMemoryPointer + 0x1c` with length `0x264 + ceil32(hookData.length)`. Consequently, the calldata buffer extends through `hookMemoryPointer + 0x280 + ceil32(hookData.length)`.

The allocated region only extends through `hookMemoryPointer + 0x240 + ceil32(swapCache.hookLongestData)`. Since `swapCache.hookLongestData` is only the maximum hook data length, the swap hook writes and reads up to `0x40` bytes beyond the reserved memory.

`_executePoolFeeHook` has the same pattern with a smaller shortfall: it can reach `hookMemoryPointer + 0x260 + ceil32(hookData.length)`, which is `0x20` bytes beyond the current base allocation.

The affected code has the following shape:

```
uint16 constant HOOK_ALLOCATION_BASE = 576;
uint256 dataLength = _addAdjustedBytesLength(
    HOOK_ALLOCATION_BASE,
    swapCache.hookLongestData
);
mstore(0x40, add(hookMemoryPointer, dataLength))
mstore(add(hookMemoryPointer, 0x260), hookDataLength)
calldatacopy(add(hookMemoryPointer, 0x280), hookData.offset, hookDataCopyLength)
```

This is a memory safety bug. The hook assembly is annotated as `memory-safe`, but it accesses memory beyond the region that it manually allocated. Solidity allocations between hook calls can begin at the advertised free memory pointer and overlap the tail of the hook

M-02 Hook call memory is underallocated

CONTINUED

buffer. The buffer is also reused across multiple hooks during a swap, so later hook calls can write over memory that Solidity is entitled to use for other temporary values. The practical impact depends on compiler allocation choices and the exact swap sequence, but the current annotation gives the optimizer a memory-safety guarantee that the assembly does not satisfy.

The hook data length is supplied through user calldata. This lets a caller exercise the underallocated dynamic region in normal swap execution whenever token or pool hooks are enabled.

— RECOMMENDATION

Increase the shared hook allocation base so it covers the largest calldata layout. The smallest direct fix is:

```
uint16 constant HOOK_ALLOCATION_BASE = 640; // 0x280
```

This covers `_executeSwapHook`, which needs the largest buffer. Alternatively, compute the allocation size from the specific hook layout before each use and reserve `0x280 + ceil32(hookData.length)` for token swap hooks and `0x260 + ceil32(hookData.length)` for pool fee hooks.

Keep the `memory-safe` annotation only if every assembly block stays within Solidity-allocated memory, valid scratch space, or temporary memory beginning at the free memory pointer for that specific block.

PROOF OF CONCEPT <https://gist.github.com/r0bert-ethack/8c1a2c5ae6cb41e3a92597fd90595eb1>

— RESOLUTION

Status: Resolved.

<https://github.com/limit-break-inc/lbamm-core/pull/12/changes/452998e5feff6f28de359204d9edc6c5978e182c>

INFORMATIONAL **I-01**

RESOLVED

Stale Natspec in AMMModule

CATEGORY Documentation
LOCATION AMMModule.sol: 78
REVIEW Remediation Review 5

DESCRIPTION

AMMModule.sol (line 78) has stale Natspec: it still says tokens are ordered automatically, but the new behavior reverts when token0 > token1.

RECOMMENDATION

Update the Natspec accordingly.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-02

RESOLVED

Wrong Error Description

CATEGORY	Documentation
LOCATION	https://github.com/GuardianOrg/lbamm-core-team1-1776734469476/blob/7cfe7b09d6852b0093b6d847e02e010f1059f897/src/Errors.sol#L166
REVIEW	Remediation Review 5

DESCRIPTION

The description for the `LBAMM__TokensOutOfOrder` states out that it will throw if `token0 < token1` but the opposite is true.

RECOMMENDATION

Write `token0 > token1` instead.

RESOLUTION

Status: Resolved.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.